NeutronNova: Group-based folding done right

Abhiram Kothapalli and Srinath Setty

Microsoft Research

Abstract.

A folding scheme reduces the task of checking multiple NP instances into checking a single instance, providing an efficient route to incrementally verifiable computation (IVC). We identify five desirable properties of a practical folding scheme: constant recursion overheads, multi-folding, linear scaling with the number of instances, no extraneous commitments, and modularity. No existing scheme achieves all five simultaneously.

We introduce NeutronNova, the first folding scheme to achieve all five. The prover's work is dominated by the cost to commit to its witness—with no extraneous commitments—and the recursive verifier performs only three group scalar multiplications and a constant number of hash computations. We construct NeutronNova modularly using the reductions of knowledge (RoK) framework around a core relation called zero-check, and build folding schemes for circuit satisfiability (CCS), grand products, and lookups by reducing each to zero-check. In experiments, NeutronNova folds a SHA-256 circuit with 2^{20} constraints in 91 ms, a $\approx 10\times$ improvement over Nova. Several subsequent works have used NeutronNova to achieve new results, including space-efficient SNARKs with optimal prover time, packed sum-check protocols over small fields, and client-side zero-knowledge proofs with low end-to-end latency.

1 Introduction

A folding scheme [47] is a simple cryptographic protocol between a *prover* and a *verifier* that reduces the task of checking several NP instances into the task of checking a single instance. For example, suppose that a verifier holds two public inputs x_1 and x_2 and suppose that the prover wishes to prove that it knows two witnesses w_1 and w_2 such that they both satisfy a circuit C, that is, $C(w_1, x_1) = 1$ and $C(w_2, x_2) = 1$. Instead of proving both claims, the prover and the verifier can first invoke a folding scheme for circuit satisfiability to reduce both claims into a single claim of the same size about some witness w_3 and some public input x_3. Folding schemes can be used to amortize the cost of proving computations with repeating structure by first compressing each sub-computation into a single computation to be checked and then proving the compressed computation (e.g., as in Vega [42]). Furthermore, Kothapalli et al. [47,46] show that folding schemes when used in a recursive manner provide incrementally verifiable computation (IVC) [60], a powerful cryptographic primitive that allows producing a proof of correct execution of a "long running" computation in an incremental fashion—without having to prove the entire computation at once. Specifically, the prover takes as input a proof π_i proving the correct execution of the first i steps of a computation, and updates it to produce a proof π_{i+1} proving the first

$i + 1$ steps of the computation. Notably, the prover's per-step work to update a proof and the verifier's work to verify a proof are both independent of the number of steps executed.[1]

Prior to folding schemes, IVC was constructed using succinct non-interactive arguments of knowledge (SNARKs) [60,11], non-interactive arguments of knowledge (NARKs) with accumulation schemes [19,23], or NARKs with split-accumulation schemes [22]. Folding schemes avoid SNARKs and NARKs entirely and merely employ a particular type of reduction of knowledge [43]. Furthermore, folding schemes not only provide a clean abstraction that is not tied to SNARKs or NARKs, they also provide a significantly more efficient prover than their predecessors. The efficiency stems from not having to produce a SNARK (or a NARK), but rather directly fold instances in some relation. In state-of-the-art folding schemes [46,20,33], the prover merely commits to its witness and performs some finite field operations.

Motivating applications for folding schemes. Folding schemes (and more generally, IVC) target proving computations with repeated copies of the same sub-computation. As a result, they are naturally suited to prove statements that arise in systems such as zkRollups, which prove the correct execution of a batch of transactions to a verifier running on a blockchain [48,26]. Indeed, folding schemes have emerged as a high-throughput solution for these types of protocols [29]. Another motivating application is zkVMs, which prove the correct execution of an assembly program (e.g., RISC-V) so that programmers can write in a high-level language and compile using existing toolchains. A key challenge is ensuring that the prover's space complexity is independent of the length of the execution. As demonstrated by Ben-Sasson et al. [11], this can be achieved via IVC, proving execution cycle-by-cycle so that the prover's space requirements match those of the program itself. Given that folding schemes offer a more efficient route to IVC, they are a natural fit for constructing time- and space-efficient zkVMs.

Five desirable properties of a folding scheme. We identify five properties for getting a folding scheme right in practice. Below, we provide a case study illustrating why all five are necessary.

P1. Constant recursion overheads. The recursive verifier should perform a constant number of group scalar multiplications, field operations, and hash computations, in the number of constraints in the relation being folded. This is critical because the verifier circuit is on the critical path of IVC: the prover must prove the verifier's computation at each step, so a smaller verifier directly translates to a faster prover.

[1] A folding-based IVC can also be built in a tree fashion (e.g., [64,50]): the prover could first produce proofs $\pi_{i \to j}$ and $\pi_{j \to k}$ proving the correct execution of the computation from steps i to j and j to k respectively. The prover can then combine them to produce a proof $\pi_{i \to k}$ that proves the correct execution of the computation from steps i to k. Crucially, the cost to produce the combined proof does not depend on the number of steps proven in the incoming proofs.

P2. Multi-folding. The scheme should efficiently fold $k \geq 2$ instances at once. This is essential: for example, in zkVMs, the correctness of a sequence of CPU cycles is arithmetized not only with constraints (e.g., R1CS) but also with lookups and read-write memory checks, producing multiple instances that must be folded simultaneously. Multi-folding also enables proof-carrying data (PCD) [12,66], a generalization of IVC.

P3. Linear scaling with k. When folding k instances, the prover and verifier costs should scale linearly in k. Super-linear scaling (e.g., $O(k \log k)$) limits the practical benefit of multi-folding.

P4. No extraneous linear-sized commitments. The prover should only commit to its witness and additional helper terms sub-linear in the witness-size. The prover should entirely avoid committing to auxiliary linear-sized vectors (e.g., cross-terms [47]) containing random field elements. When the witness contains "small" (e.g., 32-bit) field elements, committing to small elements is an order of magnitude faster than committing to arbitrary field elements—a property widely exploited in practice [53,57,7].

P5. Modularity. The folding scheme should be designed around a core relation, so that folding schemes for more complex relations are obtained by composing reductions to the core relation with the core folding scheme—rather than designing a bespoke scheme for each target relation. This simplifies design and analysis, and enables folding a *heterogeneous* set of relations simultaneously, which is essential for building high-performance zkVMs (as we discuss below).

Despite a rich body of work on folding schemes [44,46,45,20,33,15,34,32,24,21,6,2,65], no existing scheme achieves all five properties simultaneously (see Section 1.1 for details). Nova [47] requires committing to cross-terms with random field elements and does not support multi-folding. Mova [32] avoids cross-terms but has a non-constant-sized verifier and does not support multi-folding. Protostar [20] achieves constant recursion overheads but cannot efficiently fold more than two instances at once. HyperNova [46] and ProtoGalaxy [33] support multi-folding, but their recursive verifiers incur logarithmic overheads; additionally, ProtoGalaxy's prover scales super-linearly in the number of instances. Finally, all of the above design bespoke protocols for each target relation, lacking a modular framework. Figure 1 summarizes which properties each scheme achieves.

Why all five properties matter: zkVMs as a case study. zkVMs provide a compelling case study for why all five properties are necessary. The Nexus project [4] built a folding-based zkVM by replacing SNARKs with folding schemes in the approach of [11]. Nexus achieves space efficiency via folding but encodes the entire RISC-V execution as a universal circuit in R1CS, resulting in a prover that is 3+ orders of magnitude slower than state-of-the-art zkVMs such as Jolt [7]. Jolt achieves its speedup through a carefully-designed set of reductions from machine execution to a *heterogeneous* collection of simpler problems—lookups, grand products, and R1CS—which are then proven with a Spartan-type protocol [53]. However, Jolt is a monolithic zkVM whose prover can only handle a limited

	P1	P2	P3	P4	P5
Nova [47]	✓		–		
Mova [32]			–	✓	
Ova [2]	✓		–		
HyperNova [46]		✓	✓	✓	
Protostar [20]	✓		–	✓†	
Protogalaxy [33]		✓		✓	
LatticeFold [15]		✓	✓		
NeutronNova	✓	✓	✓	✓	✓

†Protostar avoids cross-terms for high-degree constraints but falls back to Nova's error-term technique for degree-2 constraints, introducing additional commitment costs [20, §3.5].

Fig. 1. Comparison of folding schemes against the five desirable properties. ✓ = achieved; blank = not achieved; – = not applicable (e.g., linear scaling is moot without multi-folding). See Section 1.1 for detailed comparisons.

number of CPU cycles, bounded by available memory. A folding scheme satisfying all five properties could bridge this gap. Constant recursion overheads (P1) and no extraneous commitments (P4) ensure that the per-step prover cost remains low and matches that of Jolt. Multi-folding with linear scaling (P2 and P3) is needed because each cycle produces instances across multiple heterogeneous relations (constraints, lookups, and memory checks) that must be folded simultaneously. Finally, modularity (P5) is what makes Jolt-style reductions compatible with folding in the first place: rather than encoding everything into a single constraint system, a modular folding scheme can target each relation in Jolt's reductions directly.

Our work: NeutronNova. We provide the first folding scheme for an NP-complete relation that achieves all five desirable properties: the prover's work is dominated by the cost to commit to its witness (with no extraneous linear-size commitments) and a linear number of finite field operations; the recursive verifier performs a constant number of field operations and 3 group scalar multiplications; and it supports folding multiple instances at once with costs that scale linearly in the number of instances folded. The folding scheme is built in a modular fashion around a core relation (zero-check), composed together via the reductions of knowledge (RoK) framework [43]. Achieving this modularity involved carefully crafting expressive intermediate relations that are still simple and meaningful in a stand-alone context. Leveraging this modularity, we also design a suite of optimized folding schemes for lookups, grand products, and constraint satisfiability—the relations needed for Jolt-style zkVMs—by reducing each to zero-check. As such, NeutronNova serves as a starting point toward building a zkVM that is simultaneously space and time efficient. Like prior group-based folding schemes, NeutronNova is not post-quantum secure; achieving constant recursion overheads in the post-quantum setting remains an open problem.

New results using our work. Several recent works have used NeutronNova to achieve new results. First, Hobbit [5] uses our folding scheme (for sum-check instances) to achieve a space-efficient zkSNARK with optimal prover time. Second, Wei et al. [62] instantiate NeutronNova over a small field and use it to build a "packed" sum-check protocol (which in turn leads to a SNARK over small fields) that minimizes the prover's work over extension fields. Third, Vega [42] uses NeutronNova to achieve client-side zero-knowledge proofs with the lowest reported end-to-end latency for proving statements over existing credentials (e.g., mDL) in zero-knowledge.

Implementation and experimental evaluation. We implement NeutronNova in 5,000 lines of Rust, reusing infrastructure code from Nova's open source implementation [1]. To demonstrate improved performance from NeutronNova, we run experiments on an Azure VM containing 16 vCPUs and 64 GB of memory (Standard D16s v6). In each experiment, we measure the cost of the prover to fold two R1CS instance-witness pairs that hash a message of a certain number of bytes using SHA-256. We vary the size of the instances by varying the size of the message hashed. As a baseline, we use Nova (implementations of other folding schemes exist [3], but they are slower than Nova). We find that NeutronNova is up to an order of magnitude faster than Nova.

	2^{15}	2^{16}	2^{17}	2^{18}	2^{19}	2^{20}	2^{21}	2^{22}
Nova	48	86	146	275	494	891	1,666	3,138
NeutronNova	6	9	14	25	45	91	192	374

Fig. 2. Prover's performance (in ms) for varying R1CS instance sizes under NeutronNova and Nova. Each experiment hashes, with SHA-256, messages of different lengths, rounding up to the nearest power of 2. Hashing 64 bytes uses $\approx 26,000$ constraints.

1.1 Related work

We compare NeutronNova with related folding schemes against the five desirable properties identified in Section 1 (summarized in Figure 1), and provide detailed comparisons below.

HyperNova. HyperNova [46] is a folding scheme for CCS, where the prover runs the sum-check protocol [49] to reduce an instance-witness pair in CCS [56] into an instance-witness pair in linearized CCS, which admits a simple folding scheme based on random linear combination without any additional help from the prover. Beyond committing to the witness, the HyperNova prover's work is merely finite field operations in the sum-check protocol. Furthermore, HyperNova is a multi-folding scheme that can fold $k \geq 2$ instances at once. Leveraging this, Zhou et al. [66] show that HyperNova naturally extends to provide a generalization of IVC called PCD [12]. One can apply their transformation to NeutronNova's folding scheme for CCS to obtain PCD.

NeutronNova improves upon HyperNova in two ways. First, NeutronNova avoids running the sum-check protocol in entirety. Second, and more importantly, NeutronNova features a dramatically smaller verifier circuit: HyperNova's verifier circuit incurs $O(d \cdot \log m)$ hash computations and field operations (where d is the degree and m the number of CCS constraints) to verify $\log m$ rounds of the sum-check protocol, whereas NeutronNova's verifier incurs only $O(d)$. To handle CCS, NeutronNova reduces it to zero-check and this is free: it requires no additional commitments and does not invoke any proving system as an intermediate step (Section 6.1).

HyperPlonk. HyperPlonk [27, Section 3.8] presents a batching scheme for polynomial evaluation claims over different evaluation points, which exhibits a special case of the non-linearity issue addressed by our folding scheme for sum-check, SumFold (Section 4). Both HyperPlonk and SumFold invoke the sum-check protocol on a "packed" polynomial that includes contributions from input polynomials, but they differ in how the sum-check protocol is used. HyperPlonk involves running the *full* sum-check protocol involving $\log(k) + \log(m)$ rounds for batching k claims involving $\log(m)$-variate polynomials. In SumFold, the sum-check protocol is run only partially over the $\log(k)$ variables introduced during packing. This is because of SumFold's key observation that the verifier can homomorphically compute a folded sum-check instance (given only commitments) regardless of the degree of the involved multivariate polynomial.

Protostar. Protostar [20] provides special-sound protocols for various relations including Plonkish, CCS, and lookups, and then demonstrates how the verifier checks for these protocols can be naturally reduced to zero-check and then folded, thereby providing a folding scheme for the original relations. NeutronNova removes this indirection by directly translating a reduction from the original relation to zero-check into a folding scheme for the original relation.

Focusing on their folding scheme for CCS, the prover's work and the verifier circuit size under NeutronNova's folding scheme for CCS are strictly lower than those of Protostar. Unfortunately, Protostar cannot efficiently fold more than two instances at once. To fold $k > 2$ instances, Protostar has to fold them one-by-one increasing the verifier circuit size by a factor of $O(k)$ compared to folding two instances. Alternatively, one can attempt to fold all k instances at once, but the costs grow exponentially in k [33, §1.2]. Furthermore, Protostar does not use the sum-check protocol, and it relies on the error-term technique from Nova to fold low-degree constraints (this introduces additional commitment costs). In contrast, NeutronNova employs the sum-check protocol as a black box, so it can leverage recent optimizations [40,30] to the sum-check protocol.

Recently, Bünz and Chen [21] extend Protostar with access to a global read-write memory. One can replace Protostar with NeutronNova in their construction to achieve better efficiency, especially due to NeutronNova's support for folding multiple instances at once.

Protogalaxy. Protogalaxy [33] extends Protostar [20] to support folding $k > 2$ instances at once. In Protogalaxy, an instance is logarithmic in the number of constraints folded, whereas in NeutronNova, the instances are constant-sized. In Protogalaxy, the verifier circuit performs $O(d + \log m)$ finite field operations and hash computations when folding two instances, whereas with NeutronNova this is only $O(d)$. Their prover time scales super-linearly with the number of instances folded, i.e., $O(k \log k)$, whereas in NeutronNova, the prover time scales linearly with k. Protogalaxy describes alternate schemes that scale better for larger values of k and handle the more general case of multiple running instances, but remarks that it may have worse constants than their main protocol. In any case, the instances are still logarithmic in the number of constraints folded. Recent work [34] applies Protogalaxy to build a relaxed version of IVC, where they apply Protogalaxy to a relation that appears closely related to our zero-check relation. One can replace Protogalaxy with NeutronNova in their construction and achieve better efficiency due to our improved folding scheme.

Neither Protostar nor Protogalaxy achieves P5 (modularity): both utilize different protocols to handle different sets of parameters (e.g., Protostar falls back to Nova's error-term technique for degree-2 constraints; Protogalaxy describes separate protocols for different shapes of multi-folding). In contrast, NeutronNova achieves all five properties by reducing everything to sum-check instances.

LatticeFold. LatticeFold [15] describes a HyperNova-like folding scheme in the lattice setting. Unlike most folding schemes including NeutronNova, LatticeFold provides plausible post-quantum security. However, it runs the full sum-check protocol over a polynomial ring, so it incurs higher prover costs and recursion overheads than NeutronNova. Furthermore, their prover must commit to low-norm versions of witness vectors and prove range checks ensuring each entry is in $[b]$. Since the degree of the sum-check is $\min(2b, d)$, there is an inherent tension: small b forces an excessive number of committed vectors, while large b increases the prover's work and verifier circuit size.

Following a preprint of this work, LatticeFold+ [16] improve upon the norm-check process of LatticeFold, but the sum-check protocol is still run over a polynomial ring, so it is far more expensive than NeutronNova. In a similar vein, Neo [51] improves upon LatticeFold, where the scheme involves running the sum-check protocol over an extension of a small field; it also provides a far more efficient commitment scheme. However, it retains the need to prove norm checks. It remains an open problem to construct a folding scheme with the efficiency of NeutronNova in the lattice setting.

Folding schemes for non-homomorphic commitments. Recent work generalizes Protostar-type folding schemes to non-homomorphic commitments (e.g., Merkle commitments to codewords) [24]. Compared to NeutronNova, they provide plausible post-quantum security. Unfortunately, they still require excessive amounts of hashing to verify Merkle membership proofs. Furthermore, the resulting IVC schemes are limited to a bounded number of steps (i.e., concrete attacks exist for non-constant recursion depth). Following a preprint of this work,

Arc [25] remove the depth restriction, but their recursion overheads remain large (orders of magnitude higher than NeutronNova's) [51].

Mova and Ova. Mova [32] is a recent folding scheme that improves on Nova [47] by avoiding a commitment to the cross-term. NeutronNova (like its predecessors HyperNova and Protostar) naturally achieves this property. Furthermore, unlike NeutronNova, Mova is limited to folding two instances at once. Moreover, when folding two instances, Mova requires a logarithmic number of hashes and finite field operations in the verifier circuit, whereas NeutronNova requires only a constant number of hashes and finite field operations.

Ova [2] reduces the number of group scalar multiplications in Nova by 1. This is an improvement atop Nova in terms of verifier circuit sizes (by about 10–13%). Unfortunately, the prover still needs to compute and commit to a cross-term, which can contain arbitrary field elements even when witness elements are "small", a problem solved by prior works [46,20,33] including this work.

Nebula. Nebula [6] provides an efficient read-write memory primitive in folding schemes, globally accessible across steps of IVC. It introduces commitment-carrying IVC (CC-IVC), which extends HyperNova's compiler so that a proof carries an incremental commitment to non-deterministic witness at each step, enabling offline memory checking [13,54,57,7] within IVC. Since they use a folding scheme as a black box, one can adapt their techniques to our setting and directly reduce read-write memory checking to zero-check and use NeutronNova's folding scheme for zero-check. We leave this to future work.

2 A technical overview of NeutronNova

We overview NeutronNova, a suite of folding schemes for relations such as lookups, grand products, and constraint satisfiability.

We design NeutronNova in a modular fashion to enable a rapid development of folding schemes for new relations that may arise in the future. To enable such extensibility, we design a folding scheme for a highly-expressive core-relation, zero-check, to which we can naturally reduce a variety of complex relations such as CCS [56], lookups, and grand-products. We believe that this modular framework also eases the formal verification of a zkVM based on NeutronNova.

In more detail, an instance-witness pair is in the zero-check relation if a prescribed multivariate polynomial, with coefficients that are a function of the instance and witness, evaluates to zero for all values over a suitable hypercube. As shown implicitly in prior work, a vast number of complex relations can be *interactively* reduced to the zero-check relation, often with minimal computation and communication [9,8,58,14,53,36,35,55,27,57,7]. For instance, the circuit satisfiability relation reduces to checking that a set of multiplication and addition gate constraints [9,58,8] evaluate to zero, the grand product relation reduces to checking that a set of constraints between coefficients of polynomials at neighboring monomials evaluate to zero [55,35], and modern NP-complete relations, such as R1CS [37], CCS [56], and Plonkish [36] reduce to checking that entry-wise

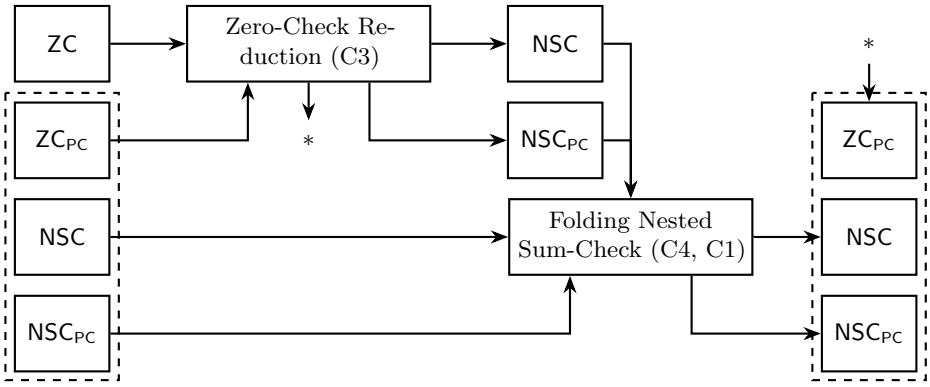

Fig. 3. An overview of ZeroFold, our folding scheme for the zero-check relation underlying NeutronNova. An instance in the zero-check relation, ZC, is folded into a running instance consisting of instances in the power-check relation, $\mathsf{ZC_{PC}}$, that checks that a vector commits to the powers of a random challenge, a variant of the sum-check relation NSC, and a variant of the sum-check relation that enforces the power-check relation $\mathsf{NSC_{PC}}$. In the diagram, $*$ indicates that one of the outputs of the zero-check reduction is a fresh $\mathsf{ZC_{PC}}$ instance included in the output running instance.

constraints on a fixed set of vectors evaluate to zero. By formally modeling such interactive reductions as *reductions of knowledge* [43], a folding scheme for the zero-check relation would imply a folding scheme for all of these relations.

ZeroFold: A new folding scheme for the zero-check relation

We introduce ZeroFold, a two-round folding scheme for the zero-check relation. This folding scheme internally invokes a *single* round of the sum-check protocol [49]. The prover's work in the folding scheme is the cost to commit to its witness and some finite field operations in the single round of the sum-check protocol. Notably, if the witness contains "small" field elements, the prover only commits to "small" field elements. This makes these commitments very fast to compute (an order of magnitude faster than committing to arbitrary field elements), a property also leveraged in prior works like Spartan [53,56], HyperNova [46], Protostar [21], Lasso [57], and Jolt [7]. Furthermore, our prover benefits from recent efficiency improvements to the sum-check prover [30]. The verifier's work is a constant number of group scalar multiplications, field operations, and hash computations. Our folding scheme naturally extends to folding an arbitrary number of instances at once (i.e., ZeroFold is a multi-folding scheme [46]), while ensuring that costs grow only linearly for the prover and the verifier in the number of instances. Note that the communication within the single round of the sum-check protocol and the verifier's work for sum-check messages do *not* depend on the number of instances folded. When folding k fresh instances with k running instances, ZeroFold requires only $1 + \log k$ rounds of the sum-check protocol.

Theorem 1 (Folding zero-check (Informal)). *There exists a folding scheme for folding k instances, with m constraints of degree d in the zero-check relation, ZC, with k running instances with $2 + \log k$ rounds, $O(\log k)$ communication, $O(mdk \log^2 d)$ prover time, and $O(k + d \log k)$ verifier time.*

As a concrete example, suppose that the verifier holds two linearly homomorphic commitments $\overline{w}_0$ and $\overline{w}_1$, and would like to fold the task of checking that the prover knows openings $w_0 \in \mathbb{F}^n$ and $w_1 \in \mathbb{F}^n$ such that, say, all the elements are 0 or 1. This can be reduced to the task of folding two zero-check statements

$$0 = w_0(x) \cdot w_0(x) - w_0(x)$$
$$0 = w_1(x) \cdot w_1(x) - w_1(x)$$

for all $x \in \{0,1\}^\ell$, where $w_i(x)$ indicates evaluating the polynomial representation of w_i at location x and $\ell = \log n$.

A prior approach [8,14,53,27], notably from [14], for encoding zero-check is to embed each of the constraints into coefficients of a Lagrange polynomial, and then check that this polynomial is zero when evaluated at a random point. In particular the verifier first samples a challenge τ and instead checks that

$$0 = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot (w_0(x) \cdot w_0(x) - w_0(x)) \tag{1}$$
$$0 = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot (w_1(x) \cdot w_1(x) - w_1(x)), \tag{2}$$

where X is the corresponding decimal representation if x is treated as the bit representation, i.e., $X = \sum_{i=0}^{\ell-1} 2^i \cdot x_i$. This essentially amounts to folding the sum-check problem, which checks the sum of evaluations of a polynomial over a suitable hypercube. While sum-check is decidedly an easier problem to work with, it is still not clear how to fold the above instances: the verifier cannot simply output a folded instance as a random linear combination of the input instances due to the inherent non-linearity of the summand polynomial.

To solve the non-linearity issue, we devise a new folding technique that utilizes a *single* round of the sum-check protocol, which we refer to as $\mathsf{SumFold}$. The essential idea in $\mathsf{SumFold}$ is that we can devise new polynomials $f(b, x)$ such that $f(0, x) = w_0(x)$ and $f(1, x) = w_1(x)$. Then, checking Equations (1), and (2) is equivalent to checking that

$$Q(B) = \sum_{b \in \{0,1\}} \mathsf{eq}(b, B) \cdot \sum_{x \in \{0,1\}^\ell} \tau^X \cdot (f(b, x) \cdot f(b, x) - f(b, x)) \tag{3}$$

is the zero polynomial, where eq is a Lagrange polynomial such that $\mathsf{eq}(b, b') = 1$ if $b = b'$ and 0 otherwise for $b, b' \in \{0, 1\}$. Then, the verifier can check Equation (3) with overwhelming probability by sampling a random challenge $\beta \in \mathbb{F}$ and checking that $0 = Q(\beta)$.

Then, by running the sum-check protocol [49] on the outer sum for a *single* round, the verifier can reduce the task of checking Equation (3) to the task of checking

$$T = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot (f(r_b, x) \cdot f(r_b, x) - f(r_b, x)),$$

for some claimed sum T and verifier challenge $r_b \in \mathbb{F}$. By the construction of f, for $w \leftarrow (1 - r_b) \cdot w_0 + r_b \cdot w_1$, this amounts to checking

$$T = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot (w(x) \cdot w(x) - w(x)).$$

The verifier can accordingly output a folded instance $\overline{w} \leftarrow (1 - r_b) \cdot \overline{w}_0 + r_b \cdot \overline{w}_1$.

Hence, the verifier has *linearly* combined two zero-check instances over *non-linear* constraints into a single sum-check instance. The verifier can then fold in new zero-check instances by first reducing to a sum-check instance (the zero-check reduction in Figure 3), and then repeating the above procedure (the nested sum-check reduction in Figure 3).

However, several challenges remain. First, with each additional zero-check instance, a new challenge τ must be sampled, and polynomials encoding the powers of τ (and the corresponding commitments) must also be folded. As it is too expensive for the verifier to compute these commitments in each folding step, this task must be outsourced to the prover. This places an additional burden on the verifier in each step to check that the claimed commitment to the powers of τ indeed agrees with τ. In applications such as IVC, where the verifier is represented as an arithmetic circuit, this is overly expensive and infeasible. Our idea here is to encode this check itself as *another* zero-check instance, which we refer to as $\mathsf{ZC}_{\mathsf{PC}}$, and bootstrap the existing folding scheme to fold these checks alongside. Moreover, even on the prover's end, committing to the full vector of the powers of τ (which scales with 2^ℓ) is *concretely* expensive. In particular, random field elements are about 10-20× more expensive than committing to witness field elements from a small set (e.g., 32-bit elements). We show how this can be circumvented by having the prover commit to two $\sqrt{n}$-sized vectors, one with the powers of τ and the other with a strided powers of τ (a similar pattern appears in tensor polynomial commitment schemes [39]). The prover and the verifier can then consider a variant of the sum-check relation, which we refer to as the *nested sum-check relation* (NSC), that computes the tensor product of these two vectors to produce the full powers of τ on the fly. Figure 3 summarizes the resulting folding scheme.

In several applications succinctness and zero-knowledge are needed for the final folded instance. For example, for applications such as IVC and zkVMs this ensures that intermediate inputs are not revealed in the final proof, and that the final proof is succinct, even in the size of a single step of execution. In Section 5.4, we discuss how the final folded instance can be proven (in zero-knowledge) using the sum-check protocol and polynomial evaluation arguments.

NeutronNova: Folding schemes for more complex relations

NeutronNova is the resulting suite of folding schemes that result from sequentially composing a reduction of knowledge (RoK) from a relation $\mathcal{R}$ to any number of zero-check instances with the folding scheme for zero-check.

This immediately provides a new folding scheme with attractive efficiency characteristics for customizable constraint systems (CCS), an NP-complete relation that generalizes R1CS [37], Plonkish [36], and AIR [10] and naturally reduces to zero-check. We provide a comparison between NeutronNova's folding scheme for CCS and prior work in Section 1.1.

Lemma 1 (Folding CCS (Informal)). *There exists a folding scheme for the CCS relation,* CCS*, with the same round complexity, communication complexity, prover time complexity, and verifier time complexity as ZeroFold.*

Moreover, by the results of Kothapalli and Setty [46, Lemma 4], we have that a folding scheme for CCS induces a corresponding non-uniform IVC (NIVC) scheme (i.e., IVC that supports different functions in each step of execution) over arbitrary degree constraints with a matching cost profile.

Beyond CCS, we formally describe a reduction of knowledge (RoK) from the grand-product relation, where a witness is a vector, and an instance is a commitment to a vector and a claimed product of all entries in the witness, to the zero-check relation. This RoK is based on the grand-product argument of Setty and Lee [55] and it is a non-interactive RoK.

Lemma 2 (Folding grand-product (Informal)). *There exists a folding scheme for folding k instances in the grand-product relation,* GP*, with k running instances with $3 + \log k$ rounds and the same communication complexity, prover time complexity, and verifier time complexity as ZeroFold.*

We additionally describe a RoK from an indexed lookup relation to four instance-witness pairs in the grand product relation. This RoK is based on Lasso [57] and consists of a single round of interaction. By sequentially composing the prior two RoKs, we get a RoK from indexed lookups to zero-checks and hence a folding scheme for indexed lookups. We compare this with prior lookup arguments for folding schemes in Section 1.1.

Lemma 3 (Folding lookup (Informal)). *There exists a folding scheme for folding k instances in the lookup relation,* LKP*, with k running instances with $4+\log 4k$ rounds, and the same communication complexity, prover time complexity, and verifier time complexity as ZeroFold.*

In Section 6.4, we describe an alternative reduction from lookups to two zero-check instances and a sum-check instance based on the logarithmic derivatives technique of Haböck [41], with a similar cost profile.

3 Preliminaries

In this section, we fix our notation and recall reductions of knowledge, which we use throughout our development. In Appendix A, we formally present multilinear polynomials and relevant properties, commitment schemes, arguments of knowledge, and IVC.

3.1 Notation

We let λ to denote the security parameter. We let $\mathsf{negl}(\lambda)$ to denote a negligible function in λ. We write $\Pr[X] \approx \epsilon$ to mean that $|\Pr[X] - \epsilon| = \mathsf{negl}(\lambda)$. Throughout the paper, the depicted asymptotics depend on λ, but we elide this for brevity. We let PPT denote probabilistic polynomial time and let EPT denote expected probabilistic polynomial time. We let $[n]$ denote the set $\{1, \ldots, n\}$. We let $\{u_i\}_{i \in [n]}$ denote the set $\{u_1, \ldots, u_n\}$.

We let $\mathbb{F}$ to denote a finite field (e.g., the prime field $\mathbb{F}_p$ for a large prime p) and let $\mathbb{F}^n$ denote vectors of length n over elements in $\mathbb{F}$. We write $\mathbb{F}^d[X_1, \ldots, X_n]$ to denote multivariate polynomials over field $\mathbb{F}$ in the variables $(X_1, \ldots, X_n)$ with degree bound d for each variable. We omit the superscript if there is no degree bound. We denote vectors as $\vec{v} = (v_1, \ldots, v_n)$. Given a vector of polynomials $\vec{g}$, we let $\vec{g}(x) = (g_1(x), \ldots, g_n(x))$. We let $\mathsf{eq}(x, y) \in \mathbb{F}^1[X_1, \ldots, X_\ell, Y_1, \ldots, Y_\ell]$ denote the polynomial that outputs 1 if $x = y$ and 0 otherwise for $x, y \in \{0,1\}^\ell$. For vector $v \in \mathbb{F}^n$ we let $\widetilde{v} \in \mathbb{F}^1[X_1, \ldots, X_{\log n}]$ denote the multilinear polynomial extension of v (i.e., $\widetilde{v}(i) = \sum_j \mathsf{eq}(i, j) \cdot v_j$).

3.2 Reductions of Knowledge

We now recall the reductions of knowledge framework, introduced by Kothapalli and Parno [43]. Reductions of knowledge are a generalization of arguments of knowledge, in which a verifier interactively *reduces* checking a prover's knowledge of a witness in a relation $\mathcal{R}_1$ to checking the prover's knowledge of a witness in another (simpler) relation $\mathcal{R}_2$. In particular, both parties take as input a claimed instance u_1 to be checked, and the prover additionally takes as input a corresponding witness w_1 such that $(u_1, w_1) \in \mathcal{R}_1$. After interaction, the prover and verifier together output a new statement u_2 to be checked in place of the original statement, and the prover additionally outputs a corresponding witness w_2 such that $(u_2, w_2) \in \mathcal{R}_2$.

We modify the original definition to account for the preprocessed setting, in which a deterministic *encoder* algorithm preprocesses a portion of the statement called the structure (typically encoding a circuit or set of constraints) *once* and outputs a prover and verifier key which can be used to verify any number of witnesses (e.g., variable assignments) against this structure. A similar extension is considered by Bünz et al. [25]. We enable the encoder to additionally output a new structure to check the output instance-witness pair against (this can be preprocessed by a subsequent encoder). We formally define this variant of reductions of knowledge below.

Definition 1 (Reduction of Knowledge [43]). *Consider relations $\mathcal{R}_1$ and $\mathcal{R}_2$ over public parameters, structure, instance, and witness tuples. A reduction of knowledge from $\mathcal{R}_1$ to $\mathcal{R}_2$ is defined by PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and deterministic algorithm $\mathcal{K}$, called the generator, the prover, the verifier and the encoder respectively with the following interface.*

- $\mathcal{G}(\lambda, n) \to \mathsf{pp}$*: Takes as input security parameter λ and size parameters n. Outputs public parameters* pp.
- $\mathcal{K}(\mathsf{pp}, \mathsf{s}_1) \to (\mathsf{pk}, \mathsf{vk}, \mathsf{s}_2)$*: Takes as input public parameters* pp *and structure* s_1*. Outputs prover key* pk*, verifier key* vk*, and updated structure* s_2.
- $\mathcal{P}(\mathsf{pk}, u_1, w_1) \to (u_2, w_2)$*: Takes as input public parameters* pp*, and an instance-witness pair (u_1, w_1). Interactively reduces the task of checking $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1$ to the task of checking $(\mathsf{pp}, \mathsf{s}, u_2, w_2) \in \mathcal{R}_2$.*
- $\mathcal{V}(\mathsf{pk}, u_1) \to u_2$*: Takes as input public parameters* pp*, and an instance u_1 in $\mathcal{R}_1$. Interactively reduces the task of checking instance u_1 to the task of checking a new instance u_2 in $\mathcal{R}_2$.*

Let $\langle \mathcal{P}, \mathcal{V} \rangle$ denote the interaction between $\mathcal{P}$ and $\mathcal{V}$. We treat $\langle \mathcal{P}, \mathcal{V} \rangle$ as a function that takes as input $((\mathsf{pk}, \mathsf{vk}), u_1, w_1)$ and runs the interaction on the prover's input (pk, u_1, w_1) and the verifier's input (pp, u_1). At the end of the interaction, $\langle \mathcal{P}, \mathcal{V} \rangle$ outputs the verifier's instance u_2 and the prover's witness w_2. A reduction of knowledge $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ satisfies the following conditions.

(i) Completeness: For any PPT adversary $\mathcal{A}$, given $\mathsf{pp} \leftarrow \mathcal{G}(\lambda, n)$, $(\mathsf{s}_1, u_1, w_1) \leftarrow \mathcal{A}(\mathsf{pp})$ such that $(\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1$ and $(\mathsf{pk}, \mathsf{vk}, \mathsf{s}_2) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}_1)$ we have that the prover's output instance is equal to the verifier's output instance u_2, and that

$$(\mathsf{pp}, \mathsf{s}_2, \langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, w_1)) \in \mathcal{R}_2.$$

(ii) Knowledge soundness: For any expected polynomial-time adversaries $\mathcal{A}$ and $\mathcal{P}^*$, there exists an expected polynomial-time extractor $\mathcal{E}$ such that given $\mathsf{pp} \leftarrow \mathcal{G}(\lambda, n)$, $(\mathsf{s}_1, u_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp})$, and $(\mathsf{pk}, \mathsf{vk}, \mathsf{s}_2) \leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}_1)$, we have that

$$\Pr[(\mathsf{pp}, \mathsf{s}_1, u_1, \mathcal{E}(\mathsf{pp}, \mathsf{s}, u_1, \mathsf{st})) \in \mathcal{R}_1] \approx \Pr[(\mathsf{pp}, \mathsf{s}_2, \langle \mathcal{P}^*, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st})) \in \mathcal{R}_2].$$

(iii) Public reducibility: There exists a deterministic polynomial-time function φ such that for any PPT adversary $\mathcal{A}$ and expected polynomial-time adversary $\mathcal{P}^*$, given

$$\begin{aligned} \mathsf{pp} &\leftarrow \mathcal{G}(\lambda, n), \\ (\mathsf{s}_1, u_1, \mathsf{st}) &\leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathsf{pk}, \mathsf{vk}, \mathsf{s}_2) &\leftarrow \mathcal{K}(\mathsf{pp}, \mathsf{s}_1), \end{aligned}$$

and $(u_2, w_2) \leftarrow \langle \mathcal{P}^*, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), u_1, \mathsf{st})$ with the interaction transcript tr*, we have that $\varphi(\mathsf{pp}, \mathsf{s}_1, u_1, \mathsf{tr}) = u_2$.*

As with arguments of knowledge, we can define various additional properties for reductions of knowledge such as succinctness (the communication is sublinear in the witness size), non-interactivity (the interaction consists of a single message from the prover), public-coin (the verifier only sends random challenges), and tree-extractability (there exists an extractor that can produce a satisfying witness given a tree of accepting transcripts). We define these properties in Appendix A.3.

Typically, we are interested in reducing several relations at once. We can interpret several relations as a single relation using the following product operator.

Definition 2 (Relation product). *For relations $\mathcal{R}_1$ and $\mathcal{R}_2$ over public parameter, structure, instance, and witness pairs we define the relation product as follows.*

$$\mathcal{R}_1 \times \mathcal{R}_2 = \left\{ (\mathsf{pp}, \mathsf{s}, (u_1, u_2), (w_1, w_2)) \,\middle|\, (\mathsf{pp}, \mathsf{s}, u_1, w_1) \in \mathcal{R}_1, (\mathsf{pp}, \mathsf{s}, u_2, w_2) \in \mathcal{R}_2 \right\}.$$

We let $\mathcal{R}^n$ denote $\mathcal{R} \times \ldots \times \mathcal{R}$ for n times.

A motivating property of reductions of knowledge is that they are composable, allowing us to build complex reductions by stitching together simpler ones. In particular, given reductions $\Pi_1 : \mathcal{R}_1 \to \mathcal{R}_2$ and $\Pi_2 : \mathcal{R}_2 \to \mathcal{R}_3$ we have that $\Pi_2 \circ \Pi_1$ (that is, running Π_1 first and then running Π_2 on the outputs) is a reduction of knowledge from R_1 to $\mathcal{R}_3$. Similarly, given reductions $\Pi_1 : \mathcal{R}_1 \to \mathcal{R}_2$ and $\Pi_2 : \mathcal{R}_3 \to \mathcal{R}_4$ we have that $\Pi_1 \times \Pi_2$ (that is, independently running Π_1 and Π_2 on pairs of inputs) is a reduction of knowledge from $\mathcal{R}_1 \times \mathcal{R}_3$ to $\mathcal{R}_2 \times \mathcal{R}_4$. We formally define sequential and parallel composition in Appendix A.3.

In this work we are chiefly interested in building folding schemes, a particular type of reduction of knowledge that reduces the task of checking several instances in some relation $\mathcal{R}_2$ into a running instance in a relation $\mathcal{R}_1$.

Definition 3 (Folding scheme). *A folding scheme for $\mathcal{R}_1^m$ and $\mathcal{R}_2^n$ is a reduction of knowledge of type $\mathcal{R}_1^m \times \mathcal{R}_2^n \to \mathcal{R}_1$ where the encoder outputs its input structure. We call a reduction of type $\mathcal{R}^n \to \mathcal{R}$ simply as a folding scheme for $\mathcal{R}$.*

3.3 The sum-check protocol

Recall that the standard sum-check relation checks that the sum of evaluations of an ℓ-variate polynomial Q (under a commitment) on the Boolean hypercube results in some value T. Formally, the sum-check relation is defined as follows.

Definition 4 (Unstructured sum-check relation). *Let $(\mathsf{Gen}, \mathsf{Commit})$ denote an additively homomorphic commitment scheme. Consider a size bound $\ell \in \mathbb{N}$. The unstructured sum-check relation USC over public parameter, instance, witness pairs is defined as follows.*

$$\mathsf{USC} = \left\{ (\mathsf{pp}, (\overline{Q}, T), Q) \,\middle|\, \begin{array}{l} Q \in \mathbb{F}[X_1, \ldots, X_\ell], \\ \overline{Q} = \mathsf{Commit}(\mathsf{pp}, Q), \\ T = \sum_{x \in \{0,1\}^\ell} Q(x) \end{array} \right\}$$

Central to our development is the sum-check protocol [49], which, when recast as a reduction of knowledge [18], reduces from the sum-check relation to the polynomial evaluation relation, which we define below.

Definition 5 (Polynomial evaluation relation). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively homomorphic commitment scheme. Consider size bound* $\ell \in \mathbb{N}$*. We define the polynomial evaluation relation,* PE*, as follows.*

$$\mathsf{PE} = \left\{ (\mathsf{pp}, (\overline{Q}, x, y), Q) \;\middle|\; \begin{array}{l} Q \in \mathbb{F}[X_1, \ldots, X_\ell], \\ \overline{Q} = \mathsf{Commit}(\mathsf{pp}, Q), \\ y = Q(x) \end{array} \right\}.$$

Lemma 4 (The sum-check protocol). *There exists a succinct, public-coin, tree-extractable reduction of knowledge (Lemma 17) from* USC *to* PE *compatible with all encoder, generator, and commitment algorithms where the output commitment is the same as the input commitment. For polynomials in* $\mathbb{F}^d[X_1, \ldots, X_\ell]$ *the communication complexity is* $O(d \cdot \ell)$ *elements in* $\mathbb{F}$*.*

4 SumFold: A folding scheme for the sum-check relation

In this section, we design a folding scheme for instance-witness pairs in the sum-check relation by using the sum-check protocol itself as a core building block. In the next section, we show that this enables a folding scheme for zero-check.

A strawman folding scheme for the unstructured sum-check relation (Definition 4) is quite natural. By linearity, to interactively fold two unstructured sum-check instances $(T_1, \overline{Q}_1)$ and $(T_2, \overline{Q}_2)$ the verifier can send a random challenge ρ and check instead that the prover knows a polynomial Q to the folded instance $(T_1 + \rho \cdot T_2, \overline{Q}_1 + \rho \cdot \overline{Q}_2)$. which indeed the prover can compute so long as it knows polynomials Q_1 and Q_2 that satisfy $(T_1, \overline{Q}_1)$ and $(T_2, \overline{Q}_2)$.

Sum-check over structured polynomials. Challenges arise in contexts of interest, where the verifier does not explicitly hold commitments to Q_1 and Q_2. In particular, when encoding instances in NP-complete relations, the polynomial Q is more concisely represented as a set of multilinear polynomials $(g_0, \ldots, g_{t-1})$ which we denote as $\vec{g}$ and a polynomial $F \in \mathbb{F}[Y_1, \ldots, Y_t]$ such that $Q(x) = F(g_0(x), \ldots, g_{t-1}(x))$ for all x in $\{0,1\}^\ell$. The multilinear polynomials $\vec{g}$ themselves are typically a linear function G of some (smaller) set of underlying vectors $\vec{w} = (w_0, \ldots, w_{s-1})$ and sometimes a public vector x. For example, when encoding circuit satisfiability, polynomials g_1, g_2, and g_3 could represent left, right, and output gate values selected from a single vector (w, x) encoding the full list of intermediate wire values and inputs. Hence, in practice, the commitment to the polynomial Q is much more concisely represented as commitments to polynomials $\vec{w}$ alongside public vector x and functions F and G. With this the following more accurately captures the relation we aim to fold.

Definition 6 (Sum-check relation). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively-homomorphic commitment scheme for vectors over finite field* $\mathbb{F}$*. Consider size*

bounds $n, m, \ell, d, s, t \in \mathbb{N}$. We define the sum-check relation, SC, as follows

$$\mathsf{SC} = \left\{ (\mathsf{pp}, (F, G), (T, \vec{u}, \mathsf{x}), \vec{w}) \;\middle|\; \begin{array}{l} T \in \mathbb{F}, F \in \mathbb{F}^d[Y_1, \ldots, Y_t], \vec{w} \in (\mathbb{F}^n)^s, \mathsf{x} \in \mathbb{F}^m, \\ \mathsf{Commit}(\mathsf{pp}, w_i) = u_i, \\ \vec{g} \in (\mathbb{F}^1[X_1, \ldots, X_\ell])^t \leftarrow G(\vec{w}, \mathsf{x}) \\ T = \sum_{x \in \{0,1\}^\ell} F(\vec{g}(x)) \end{array} \right\}$$

where G is linear over the input vector $(\vec{w}, \mathsf{x})$.

Challenges with folding sum-check. The simple folding scheme that worked for the basic sum-check relation no longer works for SC due to the potential non-linearity of F. To demonstrate this, consider the following. Concretely, suppose that G simply outputs multilinear extensions of its inputs (e.g., the multilinear extension of the witness and the public input concatenated) and F multiplies all its inputs. Suppose then that the verifier holds two sets of commitments $(\overline{g}_0, \ldots, \overline{g}_{t-1})$ and $(\overline{h}_0, \ldots, \overline{h}_{t-1})$ and would like to check that the prover knows two sets of ℓ-variate multilinear polynomial openings $(g_0, \ldots, g_{t-1})$ and $(h_0, \ldots, h_{t-1})$ such that

$$T_0 = \sum_{x \in \{0,1\}^\ell} g_0(x) \cdot g_1(x) \cdot \ldots \cdot g_{t-1}(x) \tag{4}$$

$$T_1 = \sum_{x \in \{0,1\}^\ell} h_0(x) \cdot h_1(x) \cdot \ldots \cdot h_{t-1}(x). \tag{5}$$

Suppose that the verifier samples a challenge ρ (as before), and computes the folded instance:

$$(T_0 + \rho \cdot T_1, (\overline{g}_0 + \rho \cdot \overline{h}_0), \ldots, (\overline{g}_{t-1} + \rho \cdot \overline{g}_{t-1})).$$

Then, while the prover can compute the corresponding openings $(g_0 + \rho \cdot h_0), \ldots, (g_{t-1} + \rho \cdot g_{t-1})$, they no longer satisfy the sum-check relation because

$$\begin{aligned} T_0 + \rho \cdot T_1 &= \sum_{x \in \{0,1\}^\ell} g_0(x) \cdot \ldots \cdot g_{t-1}(x) + \rho \cdot h_0(x) \cdot \ldots \cdot h_{t-1}(x) \\ &\neq \sum_{x \in \{0,1\}^\ell} (g_0(x) + \rho \cdot h_0(x)) \cdot \ldots \cdot (g_{t-1}(x) + \rho \cdot h_{t-1}(x)). \end{aligned}$$

Our solution: SumFold. We are now ready to describe NeutronNova's folding scheme for the sum-check relation, which we refer to as $\mathsf{SumFold}$. Recall that the crux of $\mathsf{SumFold}$ is that we embed polynomials g_i and h_i from two instances as Lagrange coefficients in a single-variable linear polynomial f_i (and likewise for T_0 and T_1). The prover and the verifier then run the sum-check protocol [49] for a *single* round to bind this new variable to a random value.

Indeed, for all $j \in [t]$, define f_i as a multilinear polynomial in $\ell + 1$ variables such that for all $x \in \{0,1\}^\ell$, $f_j(0, x) = g_j(x)$ and $f_j(1, x) = h_j(x)$. Specifically,

$$f_j(b, x) = (1 - b) \cdot g_j(x) + b \cdot h_j(x).$$

Then, checking Claims (4) and (5) is equivalent to checking

$$T_0 = \sum_{x \in \{0,1\}^\ell} f_0(0,x) \cdot f_1(0,x) \cdot \ldots \cdot f_{t-1}(0,x)$$
$$T_1 = \sum_{x \in \{0,1\}^\ell} f_0(1,x) \cdot f_1(1,x) \cdot \ldots \cdot f_{t-1}(1,x)$$

Embedding the above polynomials as Lagrange coefficients, the verifier can equivalently check

$$\sum_{b \in \{0,1\}} \mathsf{eq}(X,b) \cdot T_b = \sum_{b \in \{0,1\}} \mathsf{eq}(X,b) \cdot \sum_{x \in \{0,1\}^\ell} f_0(b,x) \cdot f_1(b,x) \cdot \ldots \cdot f_{t-1}(b,x).$$

Then, the verifier picks a random challenge $\beta \in \mathbb{F}$, and by the Schwartz-Zippel Lemma (Lemma 14), it reduces to checking

$$\sum_{b \in \{0,1\}} \mathsf{eq}(\beta,b) \cdot T_b = \sum_{b \in \{0,1\}} \mathsf{eq}(\beta,b) \cdot \sum_{x \in \{0,1\}^\ell} f_0(b,x) \cdot f_1(b,x) \cdot \ldots \cdot f_{t-1}(b,x).$$

The prover and verifier then apply the sum-check protocol [49] for a *single* round to the outer sum to bind the variable b to a random challenge $r_b \in \mathbb{F}$. Then, for some $T' \in \mathbb{F}$, the verifier is left to check the following claim about the inner sum

$$T' = \sum_{x \in \{0,1\}^\ell} f_0(r_b,x) \cdot f_1(r_b,x) \cdot \ldots \cdot f_{t-1}(r_b,x).$$

At this point, for $j \in [t]$, the verifier computes commitments to multilinear polynomials $f_j(r_b)$ (i.e., f_j with the first variable set to r_b), homomorphically as

$$\begin{aligned}\mathsf{Commit}(\mathsf{pp}, f_j(r_b)) &= (1-r_b) \cdot \mathsf{Commit}(\mathsf{pp}, f_j(0)) + r_b \cdot \mathsf{Commit}(\mathsf{pp}, f_j(1)) \\ &= (1-r_b) \cdot \overline{g}_j + r_b \cdot \overline{h}_j\end{aligned}$$

Finally, the verifier produces the folded instance

$$(T', ((1-r_b) \cdot \overline{g}_0 + r_b \cdot \overline{h}_0), \ldots, ((1-r_b) \cdot \overline{g}_{t-1} + r_b \cdot \overline{h}_{t-1})).$$

Furthermore, the prover can produce the satisfying folded witness $((1-r_b) \cdot g_0 + r_b \cdot h_0), \ldots, ((1-r_b) \cdot g_{t-1} + r_b \cdot h_{t-1})$.

SumFold generalizes naturally to fold *any* number of sum-check instances. If there are k sum-check instances, then the verifier samples $r_b \in \mathbb{F}^{\log k}$ and run the sum-check protocol for $\log k$ rounds. At the end of the protocol, the commitments and witnesses are folded using the evaluations of $\mathsf{eq}(r_b, i)$ for all $i \in \{0,1\}^{\log k}$ as weights. In the case of $k = 2$, these weights are simply $(1 - r_b)$ and r_b as described above. We now formally describe our generalized construction.

Construction 1 (SumFold). We construct a folding scheme for SC. Consider size bounds $\ell, d, s, t \in \mathbb{N}$ and let $\nu = \log k$. Let $(\mathcal{P}_{\mathsf{sc}}, \mathcal{V}_{\mathsf{sc}})$ denote the sum-check protocol (Lemma 4). Consider an arbitrary generator algorithm and an underlying additively homomorphic commitment scheme. We define the encoder, prover, and verifier as follows.

$\underline{\mathcal{K}(\mathsf{pp}, (F, G))}$: Output $(\mathsf{pk}, \mathsf{vk}) \leftarrow ((F, G), \perp)$ and structure (F, G).

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), \{(T_i, \vec{u}_i, \mathsf{x}_i)\}_{i \in [k]}, \{\vec{w}_i\}_{i \in [k]})}$:

1. $\mathcal{V}$: Sample and send $\rho \xleftarrow{\$} \mathbb{F}^{\nu}$.
2. $\mathcal{P}, \mathcal{V}$: Compute $(c, r_b) \leftarrow \langle \mathcal{P}_{\mathsf{sc}}, \mathcal{V}_{\mathsf{sc}} \rangle((\mathsf{pk}, \mathsf{vk}), (\overline{Q}, T), Q)$, where
$$T \leftarrow \sum_{i \in \{0,1\}^{\nu}} \mathsf{eq}(\rho, i) \cdot T_i$$
$$f_j(b, x) \leftarrow \sum_{i \in \{0,1\}^{\nu}} \mathsf{eq}(b, i) \cdot g_{i,j}(x) \text{ where } \vec{g}_i \leftarrow G(\vec{w}_i, \mathsf{x}_i)$$
$$Q(b) \leftarrow \mathsf{eq}(\rho, b) \cdot \left(\sum_{x \in \{0,1\}^{\ell}} F(f_1(b, x), \ldots, f_t(b, x)) \right)$$
$$\overline{Q} \leftarrow ((F, G), \rho, (\vec{u}_i, \mathsf{x}_i)_{i \in [k]}).$$
3. $\mathcal{P}, \mathcal{V}$: Output the folded instance-witness pair (the verifier only outputs the instance) $((T', \vec{u}, \mathsf{x}), \vec{w})$, where
$$T' \leftarrow c \cdot \mathsf{eq}(\rho, r_b)^{-1}$$
$$u_j \leftarrow \sum_{i \in \{0,1\}^{\nu}} \mathsf{eq}(r_b, i) \cdot u_{i,j}$$
$$\mathsf{x} \leftarrow \sum_{i \in \{0,1\}^{\nu}} \mathsf{eq}(r_b, i) \cdot \mathsf{x}_i$$
$$w_j \leftarrow \sum_{i \in \{0,1\}^{\nu}} \mathsf{eq}(r_b, i) \cdot w_{i,j}$$

We formally prove the following in Appendix B.1.

Theorem 2 (SumFold). *Construction 1 is a folding scheme for* SC *with* $1 + \log k$ *rounds, a communication complexity of* $O(d \log k)$ *field elements, a prover time complexity of* $O(ktd \log^2 d \cdot 2^{\ell})$ *field operations*[2]*, and a verifier time complexity of* $O(k + d \log k)$ *field operations.*

[2] The $\log^2 d$ factor is the cost of forming each degree-d sum-check round polynomial from d linear factors via an FFT. In practice, d is a small constant (e.g., 2 for R1CS), making the naive d^2 multiplication approach more efficient.

Proof (Intuition). We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct PPT extractor χ that outputs a satisfying input witness with probability $1-\mathsf{negl}(\lambda)$ given a tree of accepting transcripts and the corresponding output instance-witness pairs in SC. We do so by leveraging the tree-extractor χ_{sc} guaranteed by the tree extractability of the sum-check protocol (Lemma 4).

In particular, χ can produce a sufficient number of accepting sub-trees for the underlying sum-check protocol by using the provided instance-witness pairs in SC to solve for satisfying instance-witness pairs output by the sum-check protocol in the polynomial evaluation relation PE. Then, the sum-check tree extractor χ_{sc} can produce a satisfying input witness in the unstructured sum-check relation USC for each sub-tree. By the binding property of the commitment scheme, these witnesses must all be identical. But, by interpolating over the verifier's initial challenge, this means that the witness produced by χ_{sc} must also be a satisfying witness for the input relation SC^k. □

5 ZeroFold: A folding scheme for the zero-check relation

In this section, we design $\mathsf{ZeroFold}$, a folding scheme for the zero-check relation, which asserts that a polynomial is zero over a prescribed set of points. We use $\mathsf{SumFold}$ (Construction 1), which is a folding scheme for the sum-check relation, as the central engine underlying our construction. In Appendix E, we provide the full unrolled protocol for completeness. In the next section, we show that this enables folding schemes for a variety of relations that reduce to zero-check.

As with folding unstructured sum-check instances, unstructured zero-check instances can be folded with a standard random linear combination. However, as with the sum-check relation, in practice, commitments to the involved polynomials are typically represented as commitments to a set of (smaller) witness vectors that generate the aforementioned polynomials. Hence, we are more accurately interested in folding the following relation.

Definition 7 (Zero-check relation). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively-homomorphic commitment scheme for vectors over finite field* $\mathbb{F}$*. Consider size bounds* $n, m, \ell, d, s, t \in \mathbb{N}$*. We define the zero-check relation,* ZC*, as follows*

$$\mathsf{ZC} = \left\{ (\mathsf{pp}, (F, G), (\vec{u}, \mathsf{x}), \vec{w}) \;\middle|\; \begin{array}{l} F \in \mathbb{F}^d[Y_1, \ldots, Y_t], \vec{w} \in (\mathbb{F}^n)^s, \mathsf{x} \in \mathbb{F}^m, \\ \mathsf{Commit}(\mathsf{pp}, w_i) = u_i, \\ \vec{g} \in (\mathbb{F}^1[X_1, \ldots, X_\ell])^t \leftarrow G(\vec{w}, \mathsf{x}), \\ \forall x \in \{0,1\}^\ell.\ 0 = F(\vec{g}(x)) \end{array} \right\}$$

where G *is linear over the input vector* $(\vec{w}, \mathsf{x})$*.*

Once again, we can no longer use a standard random linear combination due to non-linearity. Our goal then is to *reduce* zero-check to the sum-check relation and then invoke $\mathsf{SumFold}$ from Section 4. This introduces various difficulties in terms of efficiency, which we methodically address.

5.1 From zero-check to (nested) sum-check

To illustrate core ideas, as in the previous section, suppose that the linear map G simply outputs multilinear extensions of its inputs and the polynomial F multiplies all its inputs. Suppose then that the verifier holds two sets of commitments $(\overline{g}_0, \ldots, \overline{g}_{t-1})$ and $(\overline{h}_0, \ldots, \overline{h}_{t-1})$, and would like to check that the prover knows two sets of ℓ-variate multilinear polynomial openings $(g_0, \ldots, g_{t-1})$ and $(h_0, \ldots, h_{t-1})$ such that

$$0 = g_0(x) \cdot \ldots \cdot g_{t-1}(x)$$
$$0 = h_0(x) \cdot \ldots \cdot h_{t-1}(x)$$

for all $x \in \{0,1\}^\ell$. To fold these instances, our goal is to reduce them to a corresponding set of sum-check instances.

Recall that, from prior work [14], this can be done by having the verifier sample a challenge τ and instead check for all $x \in \{0,1\}^\ell$ that

$$0 = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot g_0(x) \cdot \ldots \cdot g_{t-1}(x)$$
$$0 = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot h_0(x) \cdot \ldots \cdot h_{t-1}(x)$$

where $X = \sum_{i=0}^{\ell-1} 2^i \cdot x_i$ (i.e., X is the corresponding decimal representation if x is treated as the bit representation). [3]

To match the interface of the sum-check folding scheme, we define the Lagrange polynomial $e(x) = \sum_{z \in \{0,1\}^\ell} \mathsf{eq}(z, x) \cdot \tau^Z$ encoding the powers of τ (i.e., e is the multilinear extension of the vector $[\tau^0, \ldots, \tau^{2^\ell}]$), and have the verifier check for all $x \in \{0,1\}^\ell$ that

$$0 = \sum_{x \in \{0,1\}^\ell} e(x) \cdot g_0(x) \cdot \ldots \cdot g_{t-1}(x)$$
$$0 = \sum_{x \in \{0,1\}^\ell} e(x) \cdot h_0(x) \cdot \ldots \cdot h_{t-1}(x)$$

Recall that in $\mathsf{SumFold}$, the verifier samples a challenge ρ and the prover computes the folded witness as $g_j + \rho \cdot h_j$ for $j \in [t]$ for which the verifier homomorphically computes the corresponding folded commitment $\overline{g}_j + \rho \cdot \overline{h}_j$. Then, by extension

[3] We use the reduction from zero-check to sum-check from Clover [14], rather than from Spartan [53] although the latter is more widely adopted [55,27,56,57,7,31,30]. Clover's variant contributes a soundness error of $2^\ell/|\mathbb{F}|$ whereas Spartan's variant contributes at most $\ell/|\mathbb{F}|$. Both are acceptable as $|\mathbb{F}| \approx 2^{256}$ in our concrete instantiation. Furthermore, Clover's variant requires sampling a single challenge $\tau \in \mathbb{F}$ whereas Spartan's variant requires sampling a challenge from $\mathbb{F}^\ell$. We adopt the variant from Clover to keep NeutronNova's verifier circuit size independent of ℓ.

the prover will similarly need to fold e and the verifier will need to fold a corresponding commitment to e.

A key question is the following: how do the verifier and the prover represent a commitment to e, which is highly structured. Naturally, the prover can directly send a commitment to a vector of powers of τ, which for now we will assume is trusted. However, this requires committing to a vector of size equal to the number of constraints 2^ℓ. Furthermore, these vector entries are completely random. If the witness for the zero-check instance contains "small" field elements (e.g., witness elements are from the set $\{0, \ldots, 2^b - 1\}$, for $b = 32$), the cost to commit to e is at least an order of magnitude more expensive than the cost to commit to the witness, which is highly undesirable.

As a first attempt, leveraging tensor structure, τ itself could serve as a commitment to e. However, because e loses its tensor structure after folding, we will no longer have that $\tau + \rho \cdot \tau$ represents a commitment to $e + \rho \cdot e$. In particular, we have that

$$\begin{aligned} e + \rho \cdot e &= \sum_{z \in \{0,1\}^\ell} (1 + \rho) \cdot \mathsf{eq}(z, x) \cdot \tau^Z \\ &\neq \sum_{z \in \{0,1\}^\ell} \mathsf{eq}(z, x) \cdot (\tau + \rho \cdot \tau)^Z \end{aligned}$$

Our solution starts with the observation that

$$\begin{aligned} e(x) &= \sum_{z \in \{0,1\}^\ell} \mathsf{eq}(z, x) \cdot \tau^Z \\ &= \left(\sum_{z_1 \in \{0,1\}^{\ell/2}} \mathsf{eq}(z_1, x_1) \cdot \tau^{Z_1} \right) \cdot \left(\sum_{z_2 \in \{0,1\}^{\ell/2}} \mathsf{eq}(z_2, x_2) \cdot \tau^{\sqrt{2^\ell} \cdot Z_2} \right) \end{aligned}$$

where z_1 and z_2 represent the first and second half of z and for Z_i is the decimal representation of z_i for $i \in \{1, 2\}$ (i.e., $Z_i = \prod_{j=0}^{\ell/2-1} 2^j \cdot z_{i,j}$). [4] Following this, we define

$$\begin{aligned} e_1(x_1) &= \sum_{z_1 \in \{0,1\}^{\ell/2}} \mathsf{eq}(z_1, x_1) \cdot \tau^{Z_1} \\ e_2(x_2) &= \sum_{z_2 \in \{0,1\}^{\ell/2}} \mathsf{eq}(z_2, x_2) \cdot \tau^{\sqrt{2^\ell} \cdot Z_2}. \end{aligned}$$

[4] A recent work by Dao and Thaler [30] leverages a similar tensor decomposition applied to the multilinear extension of the equality function. Their focus is to speed up the task of *proving* a zero-check instance rather than folding multiple instance together. Although our purpose for decomposition is different, our prover also reaps benefits from lower field operation counts observed in [30] for the single round of the sum-check protocol in NeutronNova's folding scheme.

Now, we have that $e(x) = e_1(x_1) \cdot e_2(x_2)$. Hence, the verifier can instead check

$$0 = \sum_{x \in \{0,1\}^\ell} e_1(x_1) \cdot e_2(x_2) \cdot g_0(x) \cdot \ldots \cdot g_{t-1}(x)$$
$$0 = \sum_{x \in \{0,1\}^\ell} e_1(x_1) \cdot e_2(x_2) \cdot h_0(x) \cdot \ldots \cdot h_{t-1}(x)$$

Now, the prover can directly provide commitments to e_1 and e_2, or, for efficiency, a single commitment $\overline{e}$ to the vector $e = (e_1, e_2)$, which only commits to a vector of size $2 \cdot \sqrt{2^\ell}$. By linearity, the verifier can compute the folded commitment $\overline{e} + \rho \cdot \overline{e}$ for which the prover can compute the folded witness $(e_1 + \rho \cdot e_1, e_2 + \rho \cdot e_2)$. Formally, the prover and the verifier have reduced to folding the following variant of sum-check. Note that the verifier still needs to check the correctness of the provided commitments (which we address below).

Definition 8 (Nested sum-check relation). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively-homomorphic commitment scheme for vectors over finite field* $\mathbb{F}$*. Consider size bounds* $n, m, \ell, d, s, t \in \mathbb{N}$*. We define the nested sum-check relation,* NSC*, as follows*

$$\mathsf{NSC} = \left\{ \begin{pmatrix} (\mathsf{pp}, \\ (F, G), \\ (T, \vec{u}, \mathsf{x}, \overline{e}), \\ (\vec{w}, e)) \end{pmatrix} \middle| \begin{array}{l} T \in \mathbb{F}, F \in \mathbb{F}^d[Y_1, \ldots, Y_t], \vec{w} \in (\mathbb{F}^n)^s, \mathsf{x} \in \mathbb{F}^m, \\ \mathsf{Commit}(\mathsf{pp}, w_i) = u_i, \mathsf{Commit}(\mathsf{pp}, e) = \overline{e}, \\ \vec{g} \in \mathbb{F}^1[X_1, \ldots, X_\ell] \leftarrow G((\vec{w}, \mathsf{x})) \\ T = \sum_{x \in \{0,1\}^\ell} e_1(x_1) \cdot e_2(x_2) \cdot F(\vec{g}(x)) \end{array} \right\}$$

where G *is linear over the input vector* (w, x) *and* x_1 *and* x_2 *(likewise,* e_1 *and* e_2*) represent the first and second half of* x *(likewise,* e*).*

It appears that we can now apply $\mathsf{SumFold}$, but several problems remain. First, $\mathsf{SumFold}$ requires that the incoming sum-check instances are summing over the same domain. This is solved naturally by padding all polynomials to the same size with zero-evaluations, at no additional cost to the protocol. Second, unlike the sum-check relation, the nested sum-check relation enforces an additional stipulation that e_1 is only evaluated on the first half of x and e_2 is only evaluated on the second half of x. In the next section, we will address this by introducing a technique to safely extend the domain of e_1 and e_2. Third, when reducing from zero-check to the nested sum-check relation, the verifier must still ensure that e_1 and e_2 indeed contain appropriate powers of τ. Formally, the verifier is tasked with checking the following relation.

Definition 9 (Power-check relation). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively-homomorphic commitment scheme for vectors over finite field* $\mathbb{F}$*. Consider size bound* m*. We define the power-check relation* PC *over public parameter, instance, witness tuples as follows where* e_1 *and* e_2 *represent the first and second half of* e*.*

$$\mathsf{PC} = \left\{ (\mathsf{pp}, (\overline{e}, \tau), e) \middle| \begin{array}{l} \mathsf{Commit}(\mathsf{pp}, e) = \overline{e}, \\ e_1 = (\tau^0, \tau^1, \tau^2, \ldots, \tau^{\sqrt{m}-1}) \\ e_2 = (\tau^0, \tau^{\sqrt{m}}, \tau^{2\sqrt{m}}, \ldots, \tau^{(\sqrt{m}-1)\cdot\sqrt{m}}) \end{array} \right\}$$

From power-check to zero-check. Recall that for most applications it is too expensive for the verifier to check a power-check instance in each folding step. To circumvent this, our solution is for the prover and the verifier to also fold the power-check instances alongside the zero-check instances. Instead of designing a new folding scheme for the power-check relation, we reinterpret the power-check relation as a particular type of zero-check relation. This allows us the bootstrap our eventual folding scheme for zero-check for the purpose of power-check.

Let $m = 2^\ell$. We observe that the power-check relation can be enforced with $2 \cdot \sqrt{m}$ quadratic constraints: The first half of e can be checked by checking if $e_0 = 1$ and that each subsequent entry is the result of multiplying the previous entry by τ. The second half of e can be checked by checking that $e_{\sqrt{m}} = 1$, checking that $e_{\sqrt{m}+1}$ (presumably $\tau^{\sqrt{m}}$) is the result of multiplying the last entry of the first half of e (presumably $\tau^{\sqrt{m}-1}$) by τ, checking that the subsequent entry $e_{\sqrt{m}+2}$ (presumably $\tau^{2\cdot\sqrt{m}}$) is the square of the previous entry, and then check that all subsequent entries result from multiplying the previous entry by $e_{\sqrt{m}+1}$. However, for many applications, even checking $2 \cdot \sqrt{m}$ constraints too expensive for the verifier in each folding step. To circumvent this, we observe that instead of designing new techniques to fold power-check instances, we can natively reinterpret the power-check relation as a particular type of zero-check relation. This allows us to uniformly fold the power-check constraints alongside the original constraints with no additional rounds while maintaining key performance characteristics such as avoiding committing to any cross-terms related to the power-check instance, and folding many power-check instances at once.

Specifically, we observe that each entry of e is checked to be the result of multiplying two other entries in (e, τ). Then, given e and τ we can fix the structure polynomial G_{PC} to set $g_1 \leftarrow \widetilde{e}$ and set g_2 and g_3 to contain the left and right inputs respectively to the multiplication operation. Then, F_{PC} can compute $g_1(x) - g_2(x) \cdot g_3(x)$, for all $x \in \{0,1\}^\ell$ thereby enforcing each multiplication constraint in the zero-check. Formally, we encode a power-check instance as a zero-check instance as follows.

Construction 2 (From power-check to zero-check). We have that $(\mathsf{pp}, (\overline{e}, \tau), e) \in \mathsf{PC}$ if and only if $(\mathsf{pp}, (F_{\mathsf{PC}}, G_{\mathsf{PC}}), (\overline{e}, \tau), e) \in \mathsf{ZC}$ where $G_{\mathsf{PC}}(e, \tau)$ produces $g_1 \leftarrow \widetilde{e}$, g_2, and g_3 such that

$$g_2(i) = \begin{cases} 1 & i = 0 \\ e_{i-1} & 1 \le i < \sqrt{m} \\ 1 & i = \sqrt{m} \\ e_{i-2} & i = \sqrt{m}+1 \\ e_{i-1} & i = \sqrt{m}+2 \\ e_{\sqrt{m}+1} & \sqrt{m}+2 < i < 2\sqrt{m} \end{cases} \qquad g_3(i) = \begin{cases} 1 & i = 0 \\ \tau & 1 \le i < \sqrt{m} \\ 1 & i = \sqrt{m} \\ \tau & i = \sqrt{m}+1 \\ e_{i-1} & i = \sqrt{m}+2 \\ e_{i-1} & \sqrt{m}+2 < i < 2\sqrt{m} \end{cases}$$

and $F_{\mathsf{PC}}(y_1, y_2, y_3) = y_1 - y_2 \cdot y_3$. Note that g_1, g_2, and g_3 can be padded with evaluations to zero to any desired length. We let $\mathsf{ZC}_{\mathsf{PC}}$ denote the zero-check

relation with $(F_{\mathsf{PC}}, G_{\mathsf{PC}})$ fixed for the structure. Similarly, we let $\mathsf{NSC}_{\mathsf{PC}}$ denote the nested sum-check relation with $(F_{\mathsf{PC}}, G_{\mathsf{PC}})$ fixed for the structure.

Putting everything together, given any number of zero-check instances, and any number of power-check instances (represented as zero-check instances), the verifier first samples a challenge τ for which the prover responds with an (unverified) commitment $\overline{e}$ to the powers of τ. Then, the zero-check instances, and similarly the power-check instances, can be reduced to the corresponding nested sum-check instances over the corresponding structures with respect to the challenge τ. Alongside, the verifier outputs a new power-check instance $(\overline{e}, \tau)$ to be checked (in the future). The following construction formally captures this aggregate reduction.

Construction 3 (Zero-check reduction). We construct a reduction of knowledge of type

$$\mathsf{ZC}^k \times \mathsf{ZC}_{\mathsf{PC}}^m \to \mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^m \times \mathsf{ZC}_{\mathsf{PC}}.$$

Consider size bounds $\ell, d, t \in \mathbb{N}$. Consider an arbitrary generator algorithm, an arbitrary encoder algorithm that outputs its input structure and an arbitrary underlying additively homomorphic commitment scheme. We define the prover and the verifier as follows.

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), (\vec{\mathsf{u}}, \vec{\mathsf{u}}_{\mathsf{pc}}), (\vec{\mathsf{w}}, \vec{\mathsf{w}}_{\mathsf{pc}}))}$:

1. $\mathcal{V}$: Sample and send $\tau \xleftarrow{\$} \mathbb{F}$.
2. $\mathcal{P}$: Compute $((\overline{e}, \tau), e) \in \mathsf{ZC}_{\mathsf{PC}}$ with size parameter $m = 2^{\ell/2}$ and send $\overline{e}$.
3. $\mathcal{P}, \mathcal{V}$: Output the following instance-witness pairs (the verifier only outputs the instances):

$$\begin{aligned} \left(\{(0, \mathsf{u}_i, \overline{e})\}_{i \in [k]}, \{(\mathsf{w}_i, e)\}_{i \in [k]}\right) &\in \mathsf{NSC}^k \\ \left(\{(0, \mathsf{u}_{\mathsf{pc},j}, \overline{e})\}_{j \in [m]}, \{(\mathsf{w}_{\mathsf{pc},j}, e)\}_{j \in [m]}\right) &\in \mathsf{NSC}_{\mathsf{PC}}^m \\ ((\overline{e}, \tau), e) &\in \mathsf{ZC}_{\mathsf{PC}} \end{aligned}$$

We formally prove the following lemma in Appendix B.2.

Lemma 5 (Zero-check reduction). *Construction 3 is a reduction of knowledge of type* $\mathsf{ZC}^k \times \mathsf{ZC}_{\mathsf{PC}}^m \to \mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^m \times \mathsf{ZC}_{\mathsf{PC}}$ *with* 1 *round, a communication complexity of* 1 *field element and* 1 *element in the message space of the commitment scheme over size* $2^{\ell/2+1}$ *vectors, an* $O(2^{\ell/2+1})$ *prover time complexity, and an* $O(1)$ *verifier time complexity.*

Proof (Intuition). We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct PPT extractor χ that outputs a satisfying input witness given a tree of accepting transcripts and corresponding output instance-witness pairs in $\mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^m \times \mathsf{ZC}_{\mathsf{PC}}$.

Indeed, in such a tree, consider an output satisfying NSC instance-witness pair

$$((0, \mathsf{u}_i, \overline{e}), (\mathsf{w}_i, e)) \in \mathsf{NSC}.$$

By the satisfiability condition of NSC, for $(\vec{u}_i, \mathsf{x}_i) \leftarrow \mathsf{u}_i$ and $\vec{w}_i \leftarrow \mathsf{w}_i$ we have that

$$0 = \sum_{x \in \{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(G(\vec{w}_i, \mathsf{x}_i)).$$

But from the fact that e is a satisfying $\mathsf{ZC}_{\mathsf{PC}}$ instance, it must contain powers of some $\tau \in \mathbb{F}$. Substituting, we have that

$$0 = \sum_{x \in \{0,1\}^\ell} \tau^X \cdot F(G(\vec{w}_i, \mathsf{x}_i)(x)),$$

where X is the corresponding decimal representation if x is treated as the bit representation.

By the binding property of the commitment scheme, and the verifier's construction, we must have that $\vec{w}_i$ and x_i remain identical across all transcripts. Then, with enough such transcripts, by interpolation we have that

$$0 = F(G(\vec{w}_i, \mathsf{x}_i)(x))$$

for all $x \in \{0,1\}^\ell$. But, this means that

$$(\mathsf{u}_i, \mathsf{w}_i) \in \mathsf{ZC}.$$

By an identical line of reasoning χ can produce satisfying witnesses for the input $\mathsf{ZC}_{\mathsf{PC}}$ instances as well. □

5.2 A folding scheme for the nested sum-check relation

Given the reductions in the prior subsection, our goal is to fold a set of nested sum-check instances over the original structure (F, G), and a set of nested sum-check instances over the power-check structure $(F_{\mathsf{PC}}, F_{\mathsf{PC}})$. This involves recasting nested sum-check instances as sum-check instances.

Indeed, as stated earlier, one technicality is that unlike the sum-check relation, which performs a check of the form $T = \sum_x F(G(\vec{w}, \mathsf{x})(x))$, the nested sum-check relation performs a more structured check of the form:

$$T = \sum_{x \in \{0,1\}^\ell} e_1(x_1) \cdot e_2(x_2) \cdot F(G(\vec{w}, \mathsf{x})(x)).$$

The nested sum-check instance can be reinterpreted as a sum-check instance by first defining polynomials h_1 and h_2, which extend the domain of e_1 and e_2 to $\{0,1\}^\ell$ as follows

$$h_1(x) = e_1(x_1) \cdot \widetilde{(1, 1, \ldots, 1)}(x_2) = e_1(x_1)$$
$$h_2(x) = \widetilde{(1, 1, \ldots, 1)}(x_1) \cdot e_2(x_2) = e_2(x_2)$$

Intuitively, h_1 replicates e_1 column-wise and h_2 replicates e_2 row-wise ensuring that x_2 and x_1 are respectively ignored.[5]

Next, we can define the structure G' to additionally produce h_1 and h_2 given an additional vector e in the nested sum-check witness

$$G'((\vec{w}, e), \mathsf{x}) = (G(\vec{w}, \mathsf{x}), h_1, h_2)$$

Crucially, we have that h_1 and h_2 are produced linearly from e_1 and e_2, which is stipulated by the sum-check relation. Then, we can define the polynomial F' to account for these additionally produced polynomials

$$F'(\vec{g}(x), h_1(x), h_2(x)) = h_1(x) \cdot h_2(x) \cdot F(\vec{g}(x))$$

Then, we have that

$$\begin{aligned} T &= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(G(\vec{w}, \mathsf{x})(x)) \\ &= \sum_{x \in \{0,1\}^\ell} F'(G'((\vec{w}, e), \mathsf{x})(x)) \end{aligned}$$

Hence, the prover and the verifier can fold a set of nested sum-check instances over structure (F, G) by using $\mathsf{SumFold}$ to fold equivalent sum-check instances with respect to structure (F', G').

We can similarly define the structure $(F'_{\mathsf{PC}}, G'_{\mathsf{PC}})$ to fold the nested sum-check instances over the power-check structure. At this point the prover and the verifier can run two independent invocations of $\mathsf{SumFold}$, once to fold over structure (F', G') and once to fold over structure $(F'_{\mathsf{PC}}, G'_{\mathsf{PC}})$. Alternatively, for concrete efficiency, the prover and the verifier can run a single invocation of $\mathsf{SumFold}$ by taking a random linear combination of the original structures. In particular, consider two sets of nested sum-check instance-witness pairs

$$\begin{aligned} \{(T_i, (\vec{u}_i, \overline{e}_i, \mathsf{x}_i), (\vec{w}_i, e_i))\}_{i \in [k]} &\in \mathsf{NSC}^k \\ \{(T_{\mathsf{pc},i}, (\vec{u}_{\mathsf{pc},i}, \overline{e}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}), (\vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}))\}_{i \in [k]} &\in \mathsf{NSC}^k_{\mathsf{PC}} \end{aligned}$$

to be checked over structures (F, G) and $(F_{\mathsf{PC}}, G_{\mathsf{PC}})$ respectively. The verifier begins by sending a challenge $\gamma \in \mathbb{F}$. Then, for each $i \in [k]$, the prover and the verifier can reduce to checking that

$$T_i + \gamma \cdot T_{\mathsf{pc},i} = \sum_{x \in \{0,1\}^\ell} F'(G'((\vec{w}_i, e_i), \mathsf{x}_i)(x)) + \gamma \cdot F'_{\mathsf{PC}}(G'_{\mathsf{PC}}((\vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}), \mathsf{x}_{\mathsf{pc},i})(x))$$

Once again, we can recast the above instance to a sum-check instance by defining the following structure:

$$\begin{aligned} F''(\vec{g}, \vec{g}_{\mathsf{pc}}) &= F(\vec{g}) + \gamma \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}}) \\ G''(\vec{w}, e, \vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}, (\mathsf{x}, \mathsf{x}_{\mathsf{pc}})) &= (G'((\vec{w}, e), \mathsf{x}), G'_{\mathsf{PC}}((\vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}), \mathsf{x}_{\mathsf{pc}})). \end{aligned}$$

[5] While this seemingly increases the degree of the summand polynomial by 1 as both x_1 and x_2 appear in one extra polynomial, the modification is free as $(1, \widetilde{1, \ldots, 1})(x_i) = 1$.

Then, for the aggregated instance-witness pair

$$(T_i + \gamma \cdot T_{\mathsf{pc},i}, (\vec{u}_i, \overline{e}_i, \vec{u}_{\mathsf{pc},i}, \overline{e}_{\mathsf{pc},i}, (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i})), (\vec{w}_i, e_i, \vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i})),$$

the verifier can equivalently check

$$T_i + \gamma \cdot T_{\mathsf{pc},i} = \sum_{x \in \{0,1\}^\ell} F''(G''((\vec{w}_i, e_i, \vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}), (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i}))(x)) \tag{6}$$

for all $i \in [k]$. Note that because we are pairing off NSC and $\mathsf{NSC}_{\mathsf{PC}}$ instances, the above optimization specifically requires the same number of instances from each relation. This is not a problem in practice as we can always pad either side with trivially satisfying instance-witness pairs.

Then, the prover and the verifier can run $\mathsf{SumFold}$ over all the instance-witness pairs in Equation (6) to reduce to checking a single instance

$$\begin{aligned} T_\gamma &= \sum_{x \in \{0,1\}^\ell} F''(G''((\vec{w}, e, \vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}), (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc}}))(x)) \\ &= \sum_{x \in \{0,1\}^\ell} F'(G'((\vec{w}, e), \mathsf{x})(x)) + \gamma \cdot F'_{\mathsf{PC}}(G'_{\mathsf{PC}}((\vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}), \mathsf{x}_{\mathsf{pc}})(x)) \end{aligned}$$

At this point, the prover can send

$$T = \sum_{x \in \{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(G(\vec{w}, \mathsf{x})(x)) \tag{7}$$

$$T_{\mathsf{pc}} = \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{\mathsf{pc},1}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2}(x_2) \cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}})(x)). \tag{8}$$

The verifier checks that indeed $T_\gamma = T + \gamma \cdot T_{\mathsf{pc}}$, and reduces to checking Equations (7) and (8). But, by definition, this is precisely equivalent to checking an instance in NSC and an instance in $\mathsf{NSC}_{\mathsf{PC}}$. Hence, the above interaction gives us a reduction of knowledge from $\mathsf{NSC}^k \times \mathsf{NSC}^k_{\mathsf{PC}}$ to $\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}}$. We formally describe this reduction below.

Construction 4 (Folding nested sum-check). We construct a reduction of knowledge of type

$$\mathsf{NSC}^k \times \mathsf{NSC}^k_{\mathsf{PC}} \to \mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}}.$$

Indeed, let $(\mathcal{G}_{\mathsf{SF}}, \mathcal{K}_{\mathsf{SF}}, \mathcal{P}_{\mathsf{SF}}, \mathcal{V}_{\mathsf{SF}})$ be $\mathsf{SumFold}$ (Construction 1). Consider size bounds $\ell, d, t \in \mathbb{N}$. Consider an arbitrary generator algorithm and an underlying additively homomorphic commitment scheme. We define the encoder, the prover, and the verifier as follows.

$\underline{\mathcal{K}(\mathsf{pp}, (F, G))}$: Output $(\mathsf{pk}, \mathsf{vk}) \leftarrow ((F, G), \bot)$ and structure (F, G).

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), \{((T_i, \mathsf{u}_i, \mathsf{x}_i), (T_{\mathsf{pc},i}, \mathsf{u}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}))\}_{i \in [k]}, \{(\mathsf{w}_i, \mathsf{w}_{\mathsf{pc},i})\}_{i \in [k]})}$:

1. $\mathcal{V}$: Sample and send $\gamma \xleftarrow{\$} \mathbb{F}$.

2. $\mathcal{P}$: Compute the updated prover key $\mathsf{pk}' \leftarrow (F', G')$ where

$$F'(\vec{g}, h_1, h_2, \vec{g}_{\mathsf{pc}}, h_{\mathsf{pc},1}, h_{\mathsf{pc},2}) = h_1 \cdot h_2 \cdot F(\vec{g}) + \gamma \cdot h_{\mathsf{pc},1} \cdot h_{\mathsf{pc},2} \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}})$$
$$G'(\vec{w}, e, \vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}, (\mathsf{x}, \mathsf{x}_{\mathsf{pc}})) = (G(\vec{w}, \mathsf{x}), h_1, h_2, G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}}), h_{\mathsf{pc},1}, h_{\mathsf{pc},2})$$

where $(F_{\mathsf{PC}}, G_{\mathsf{PC}})$ are defined as in Construction 2 with an appropriate padding, and for $j \in \{1, 2\}$, $h_j(x) = \widetilde{e}_j(x_j)$ and $h_{\mathsf{pc},j}(x) = \widetilde{e}_{\mathsf{pc},j}(x_j)$, where (e_1, e_2), $(e_{\mathsf{pc},1}, e_{\mathsf{pc},2})$, and (x_1, x_2) represent the first and second half of the original vector.

3. $\mathcal{P}, \mathcal{V}$: Run the sum-check folding reduction $\langle \mathcal{P}_{\mathsf{SF}}, \mathcal{V}_{\mathsf{SF}} \rangle$ on the updated prover and verifier keys $(\mathsf{pk}', \mathsf{vk})$, and the updated instance-witness pairs for $i \in [k]$

$$\left((T_i + \gamma \cdot T_{\mathsf{pc},i}, (\mathsf{u}_i, \mathsf{u}_{\mathsf{pc},i}), (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i})), (\mathsf{w}_i, \mathsf{w}_{\mathsf{pc},i}) \right)$$

to produce a folded instance-witness pair $(T_\gamma, (\mathsf{u}, \mathsf{u}_{\mathsf{pc}}), (\mathsf{x}, \mathsf{x}_{\mathsf{pc}})), (\mathsf{w}, \mathsf{w}_{\mathsf{pc}})$.

4. $\mathcal{P}$: Parse $(\vec{w}, e) \leftarrow \mathsf{w}$, $(\vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}) \leftarrow \mathsf{w}_{\mathsf{pc}}$, compute $\vec{g} \leftarrow G(\vec{w}, \mathsf{x})$, $\vec{g}_{\mathsf{pc}} \leftarrow G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}})$, and send to $\mathcal{V}$

$$T \leftarrow \sum_{x \in \{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(\vec{g}(x))$$
$$T_{\mathsf{pc}} \leftarrow \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{\mathsf{pc},1}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2}(x_2) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}}(x)).$$

5. $\mathcal{V}$: Check that $T_\gamma = T + \gamma \cdot T_{\mathsf{pc}}$.

6. $\mathcal{P}, \mathcal{V}$: Output the following instance-witness pairs (the verifier only outputs the instances).

$$((T, \mathsf{u}, \mathsf{x}), \mathsf{w}) \in \mathsf{NSC}$$
$$((T_{\mathsf{pc}}, \mathsf{u}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}}), \mathsf{w}_{\mathsf{pc}}) \in \mathsf{NSC}_{\mathsf{PC}}$$

We formally prove the following lemma in Appendix B.3.

Lemma 6 (Folding nested sum-check). *Construction 4 is a reduction of knowledge of type* $\mathsf{NSC}^k \times \mathsf{NSC}^k_{\mathsf{PC}} \to \mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}}$ *with* $2 + \log k$ *rounds, a communication complexity of* $O(d \log k)$ *field elements, a prover time complexity of* $O(ktd \log^2 d \cdot 2^\ell)$ *field operations, and a verifier time complexity of* $O(k + d \log k)$ *field operations.*

Proof (Intuition). We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct PPT extractor χ that outputs a satisfying input witness given a tree of accepting transcripts and corresponding output instance-witness pairs in $\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}}$. We do so by leveraging the tree-extractor χ_{FSC} guaranteed by the tree extractability of $\mathsf{SumFold}$ (Lemma 19).

In particular, χ can produce a sufficient number of accepting sub-trees for the underlying $\mathsf{SumFold}$ by using the provided instance witness pairs in $\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}}$ to solve for corresponding satisfying output instance-witness pairs in SC. Then, $\mathsf{SumFold}$'s tree extractor χ_{FSC} can produce a satisfying input witness in SC^k for each sub-tree. By the binding property of the commitment scheme, these witnesses must all be identical. Then, by interpolating over the verifier's initial challenge, this means that the witness produced by χ_{FSC} must also be a satisfying witness for the input relation $\mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^k$. □

5.3 ZeroFold: Putting everything together

So far, we have constructed the zero-check reduction $\Pi_{\mathsf{ZCR}} : \mathsf{ZC}^k \times \mathsf{ZC}_{\mathsf{PC}}^k \to \mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^k \times \mathsf{ZC}_{\mathsf{PC}}$ (Construction 3) and a folding scheme for the nested sum-check relation $\Pi_{\mathsf{FNSC}} : \mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^k \to \mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}}$ (Construction 4). We can use these two reductions to produce a folding scheme for ZC with a running instance type of $\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}}$. Indeed, suppose that a verifier would like to check k zero-check instances $\vec{\mathsf{u}} = (\mathsf{u}_1, \ldots, \mathsf{u}_k)$ and k running instances $\vec{\mathsf{U}} = (\mathsf{U}_1, \ldots, \mathsf{U}_k)$ in $\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}}$. The verifier first collects k instances in $\mathsf{ZC}_{\mathsf{PC}}$ from $\vec{\mathsf{U}}$ and reduces these instances along with $\vec{\mathsf{u}}$ via the zero-check reduction Π_{ZCR} into k instances in NSC, k instances $\mathsf{NSC}_{\mathsf{PC}}$, and a fresh instance in $\mathsf{ZC}_{\mathsf{PC}}$. Now the verifier is left with $2k$ instances in NSC and $2k$ instances in $\mathsf{NSC}_{\mathsf{PC}}$ (half from the original k running instances and half freshly generated by Π_{ZCR}). The verifier reduces all these instances via the folding scheme for the nested sum-check relation Π_{FNSC} into a single instance in NSC and a single instance in $\mathsf{NSC}_{\mathsf{PC}}$. The verifier concludes by outputting these two instances alongside the fresh instance in $\mathsf{ZC}_{\mathsf{PC}}$ generated by Π_{ZCR} as the folded instance. We formally capture this construction in the following theorem. We let $1_{\mathcal{R}}$ denote the identity reduction (i.e. the prover and the verifier output their inputs) for the relation $\mathcal{R}$. In Appendix D, we discuss how to reduce the number of rounds to $2 + \log k$.

Theorem 3 (ZeroFold). *Given reductions of knowledge*

$$\Pi_{\mathsf{ZCR}} : \mathsf{ZC}^k \times \mathsf{ZC}_{\mathsf{PC}}^k \to \mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^k \times \mathsf{ZC}_{\mathsf{PC}} \qquad \textit{(Construction 3)},$$
$$\Pi_{\mathsf{FNSC}} : \mathsf{NSC}^{2k} \times \mathsf{NSC}_{\mathsf{PC}}^{2k} \to \mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \qquad \textit{(Construction 4)}$$

we have that

$$(\Pi_{\mathsf{FNSC}} \times 1_{\mathsf{ZC}_{\mathsf{PC}}}) \circ (1_{\mathsf{NSC}^k \times \mathsf{NSC}_{\mathsf{PC}}^k} \times \Pi_{\mathsf{ZCR}})$$

is a reduction of knowledge of type

$$(\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}})^k \times \mathsf{ZC}^k \to (\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}})$$

with $4 + \log k$ rounds, a communication complexity of $O(d \log k)$ field elements and 1 element in the message space of the commitment scheme over size $2^{\ell/2+1}$ vectors, a prover time complexity of $O(ktd \log^2 d \cdot 2^\ell)$ field operations and a commitment over a size $2^{\ell/2+1}$ vector, and a verifier time complexity of $O(k + d \log k)$ field operations.

Proof. The result follows by composing the zero-check reduction (Construction 3) with the nested sum-check folding reduction (Construction 4). The field-operation portion of the prover time is dominated by the invocation of $\mathsf{SumFold}$ in Construction 4, which takes $O(ktd \log^2 d \cdot 2^\ell)$ field operations. The zero-check reduction additionally requires the stated commitment over a size-$2^{\ell/2+1}$ vector. □

5.4 Proving the final folded relation

The final folded relation, $\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}}$, can be directly proven using the sum-check protocol (Lemma 4) and polynomial evaluation argument (Definition 5). This ensures that no additional reductions or emulation of checks within other constraint systems are necessary.

In particular, $\mathsf{ZC}_{\mathsf{PC}}$ can be reduced to $\mathsf{NSC}_{\mathsf{PC}}$ via Lemma 5. Then, the resulting $\mathsf{NSC}_{\mathsf{PC}}$ and NSC instance-witness pairs can be reduced to evaluation claims for the committed polynomials via the sum-check protocol. These can then be succinctly proven using standard polynomial evaluation arguments. Standard batching techniques can be applied to reduce to a single sumcheck protocol invocation and a single polynomial evaluation argument. The asymptotic costs can be found in [56, Figure 3] and concrete costs will be comparable to [53, Figures 7 and 8].

A zero-knowledge variant of the sum-check protocol [28,63] and polynomial evaluation arguments can be used to ensure a zero-knowledge argument for the final folded instance. Alternatively, using $\mathsf{NovaBlindFold}$ from HyperNova [46], the prover and verifier can fold in random instance-witness pairs into the NSC and $\mathsf{NSC}_{\mathsf{PC}}$ instance-witness pairs and then use the standard sum-check and polynomial evaluation arguments (not necessarily zero-knowledge).

6 Folding relations targeting zero-check

In this section, we design reductions of knowledge [43] from various relations to zero-check, thereby deriving folding schemes for these relations. Concretely, we reduce an NP-complete relation, CCS [56], the grand-product relation, and the lookup relation to zero-check. In all these cases, the reductions are simple, constant-round, and involve minimal verifier work.

To tie together the results of this section, we start with a natural corollary of Theorem 3, which formally states that if there exists a reduction of knowledge from a relation $\mathcal{R}$ to any number of zero-check instances, then we can fold instances in $\mathcal{R}$ by first reducing them to instances in the zero-check relation and then applying $\mathsf{ZeroFold}$, NeutronNova's folding scheme for zero-check.

Corollary 1 (Folding relations targeting zero-check). *Let Π_{FZC} represent the zero-check folding scheme (Theorem 3). A reduction of knowledge $\Pi : \mathcal{R} \to \mathsf{ZC}^m$ implies*

$$\Pi_{\mathsf{FZC}} \circ (1_{(\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}})^{nm}} \times \Pi^n)$$

is a reduction of knowledge of type

$$(\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}})^{nm} \times \mathcal{R}^n \to (\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}})$$

where the round complexity, communication complexity, prover time complexity and verifier time complexity are the combined total of running Π for n times and Π_{FZC} over nm zero-check instances.

6.1 Reducing CCS to zero-check

We first reduce the customizable constraint system (CCS) [56], to zero-check without any interaction or work by the verifier. Crucially, this reduction requires no additional commitments beyond the witness and does not invoke any proving system as an intermediate step; it simply re-represents CCS terms as zero-check terms. Since CCS is an NP-complete language that simultaneously generalizes R1CS [37], AIR [10], and Plonkish [36], we obtain a folding scheme for all of these relations whose efficiency matches the efficiency of our folding scheme for zero-check.

Recall that CCS checks arbitrary degree constraints represented using selector matrices $M_1, \ldots, M_t$ over z (which is the concatenation of a witness vector w, a public vector x, and a constant of 1) by first computing the matrix-vector products $M_j \cdot w$ for all $j \in [t]$ and then checking a sum of Hadamard products over the resulting vectors. Formally, CCS is defined as follows.

Definition 10 (CCS [56]). *Let $(\mathsf{Gen}, \mathsf{Commit})$ denote an additively homomorphic commitment scheme for vectors over finite field $\mathbb{F}$. Consider size bounds $m, n, N, \ell, t, q, d \in \mathbb{N}$ where $n > \ell$. We define the customizable constraint system (CCS) relation, CCS, over structure, instance, witness tuples as follows.*

$$\mathsf{CCS} = \left\{ (\mathsf{pp}, (\vec{M}, \vec{S}, \vec{c}), (\overline{w}, \mathsf{x}), w) \;\middle|\; \begin{array}{l} M \in (\mathbb{F}^{m \times n})^t, S_i \subseteq [t] \text{ for } i \in [q], \vec{c} \in \mathbb{F}^q \\ \mathsf{x} \in \mathbb{F}^\ell, w \in \mathbb{F}^{m-\ell-1}, \\ \mathsf{Commit}(\mathsf{pp}, w) = \overline{w}, \\ \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} M_j z = 0 \end{array} \right\}$$

where $z = (w, \mathsf{x}, 1)$ and each matrix M_i contains at most $\Omega(m)$ non-zero entries.

We reduce CCS to zero-check as follows: we design an inner zero-check structure polynomial $G_{\vec{M}}$ that encodes the CCS selector matrices $M_1, \ldots, M_t$. Specifically, on input witness vector w and a public vector x, $G_{\vec{M}}$ outputs the multilinear extensions of $M_1 z, \ldots, M_t z$, where $z = (w, \mathsf{x}, 1)$. Then, for each row of entries in these vectors, we design an outer zero-check structure polynomial $F_{\vec{S},\vec{c}}$ that takes a sum of products exactly as the original CCS structure dictates. Formally we have the following reduction from CCS to zero-check, which, by Corollary 1, implies a folding scheme for CCS.

Lemma 7 (From CCS to zero-check [56]). *Consider size bounds m, n, N, ℓ, t, q, and d where $n > \ell$. We have that $(\mathsf{pp}, (\vec{M}, \vec{S}, \vec{c}), (\overline{w}, \mathsf{x}), w) \in \mathsf{CCS}$ if and*

only if $(\mathsf{pp}, (F, G), (\overline{w}, \mathsf{x}), w) \in \mathsf{ZC}$ *where for* $z = (w, \mathsf{x}, 1)$

$$F_{\vec{S},\vec{c}}(m_1, \ldots, m_t) = \sum_{i \in [q]} c_i \cdot \prod_{j \in S_i} m_j$$
$$G_{\vec{M}}(w, \mathsf{x}) = (\widetilde{(M_1 z)}, \ldots, \widetilde{(M_t z)}).$$

This naturally induces an equivalent reduction *of knowledge* (with no interaction) by defining the encoder, prover, and verifier to map the structure, instance, and witness identically.

6.2 Reducing grand-product to zero-check

In this section, we reduce the grand-product relation to the zero-check relation, thereby providing a folding scheme for grand-products. We rely on results from Lasso [57], which itself builds on its predecessors [53,55].

Recall that the grand-product relation checks that taking the product of elements of a committed vector v results in some prescribed value p.

Definition 11 (Grand-product relation). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively-homomorphic commitment scheme for vectors over finite field* $\mathbb{F}$*. Consider size bound* $n, m \in \mathbb{N}$*. We define the grand-product relation* GP *over public parameter, instance, witness tuples as follows*

$$\mathsf{GP} = \left\{ (\mathsf{pp}, (p, \overline{v}), v) \,\middle|\, v \in \mathbb{F}^n, \mathsf{Commit}(\mathsf{pp}, v) = \overline{v}, v = \textstyle\prod_{i \in [m]} v_i \right\}.$$

Lasso provides two options for *proving* grand products: (1) the approach presented in Section 6.2, which in their setting is then proven with the sum-check protocol; and (2) the protocol of Thaler [59], a specialization of the GKR proof system [38]. The main trade-off is that the first option requires committing to an additional vector of size N, where N is the number of entries in v, whereas the second protocol requires a logarithmic number of invocations of the sum-check protocol, resulting in a proof length of $O(\log^2 N)$. To mitigate this trade-off, Setty and Lee [55, §6] also describe a hybrid solution where their grand product argument is applied on an instance of size $N/2^k$ (for some chosen parameter k) and Thaler's protocol is applied to reduce a claim about that instance to a multilinear evaluation claim about $\widetilde{v}$. For simplicity, we focus on option (1). However, both option (2) and the hybrid solution induce alternative compatible reductions from grand-product to zero-check.

At a high level, Setty and Lee consider a new polynomial f, which contains the original vector v as well as all the intermediate products computed in a tree-like fashion. Then, the relationship between v and the final claimed product p can be checked using quadratic constraints.

Lemma 8 (Grand-product arithmetization [55]). *Consider* $p \in \mathbb{F}$ *and* $v \in \mathbb{F}^n$*. There exists an efficiently computable* $f \in \mathbb{F}[X_1, \ldots, X_{\log n + 1}]$ *such that*

$p = \prod_{i \in [n]} v_i$ *if and only if* $f(0, x) = \widetilde{v}(x)$, $f(1, \ldots, 1, 0) = p$ *and*

$$f(1, x) = f(x, 0) \cdot f(x, 1) \tag{9}$$

for all $x \in \{0,1\}^{\log n}$.

In light of this arithmetization, we get a natural reduction from grand-product to zero-check: The prover begins by committing to an intermediate product vector v' which is the portion of f not including v. Together v and v' are treated as the new zero-check witness and a vector with a single entry of p is treated as the public vector. Then, we design an inner zero-check structure polynomial G_{GP}, which produces polynomials $g_1(x) = f(1, x)$, $g_2(x) = f(x, 0)$, and $g_3(x) = f(x, 1)$ by selecting elements from v, v', and p. For each evaluation of these polynomials we design an outer zero-check structure polynomial F_{GP} that computes Equation 9. Formally, we have the following reduction.

Construction 5 (From grand-product to zero-check). We construct a reduction of knowledge of type

$$\mathsf{GP} \to \mathsf{ZC}.$$

Consider $n \in \mathbb{N}$. Let $(\mathsf{Gen}, \mathsf{Commit})$ denote an arbitrary additively homomorphic vector commitment scheme. We define $\mathcal{G}(\lambda, n) = \mathsf{Gen}(\lambda, n)$ and the encoder, prover, and verifier as follows.

$\underline{\mathcal{K}(\mathsf{pp})}$:

1. Define $F_{\mathsf{GP}}(g_1, g_2, g_3) = g_1 - g_2 \cdot g_3$ and define G_{GP} on input $((v, v'), p)$ to output

$$\begin{aligned} g_1(x) &\leftarrow f(1, x) \\ g_2(x) &\leftarrow f(x, 0) \\ g_3(x) &\leftarrow f(x, 1) \end{aligned}$$

   where f is the multilinear extension of $(v, (v'_1, \ldots, v'_{n-2}, p, v'_n))$.

2. Output $(\mathsf{pk}, \mathsf{vk}) \leftarrow (\mathsf{pp}, \perp)$ and the updated zero-check structure $(F_{\mathsf{GP}}, G_{\mathsf{GP}})$.

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), (p, \overline{v}), v)}$:

1. $\mathcal{P}$: Compute a multilinear polynomial $\widetilde{f}$ as guaranteed by Lemma 8. Define $v' \in \mathbb{F}^n$ such that $\widetilde{v}' = \widetilde{f}(1, x)$ and send a commitment $\overline{v}' \leftarrow \mathsf{Commit}(\mathsf{pp}, v')$.
2. $\mathcal{P}, \mathcal{V}$: Produce the following zero-check instance-witness pair (the verifier only outputs the instance)

$$(((\overline{v}, \overline{v}'), p), (v, v')) \in \mathsf{ZC}.$$

We formally prove the following lemma in Appendix B.5, which, by Corollary 1 implies a folding scheme for GP.

Lemma 9 (From grand-product to zero-check). *Construction 5 is an RoK of type* $\mathsf{GP} \to \mathsf{ZC}$ *with a single round, a communication complexity of one element in the message space of the commitment scheme for vectors of size* n*. The prover's time complexity is* $O(n)$ *and the verifier's time complexity is* $O(1)$*.*

6.3 Reducing lookups to grand-product

In this section, we reduce a lookup instance to several grand product instances, which, from the previous subsection, can be reduced to zero-check instances. Hence, we get a folding scheme for the lookup relation.

Our starting point is Lasso [57], a state-of-the-art lookup argument. Lasso supports unstructured tables (for which the verifier holds a commitment) as well as very large Spark-only structured (SOS) tables that do not need to be materialized either by the prover nor the verifier. Crucially, Lasso's lookup argument for very large structured tables of size N is obtained by essentially running c copies of the lookup argument for unstructured tables on tables of size $N^{1/c}$ and then assembling results of sub-table lookups (e.g., with CCS). A similar observation appears in prior work [31]. For this reason, the rest of the section focuses on lookups over unstructured tables, but our results naturally generalize to Lasso's SOS tables.

Recall that the lookup relation checks that a committed vector v only consists of elements found in a committed table t. Lasso considers a more general form, which they refer to as indexed lookups, where an additional committed vector a indicates which index of t each entry of v can be found. All lookups used in Jolt [7] are indexed lookups, so we focus on this general version.

Definition 12 (Lookup relation [57]). *Let* $(\mathsf{Gen}, \mathsf{Commit})$ *denote an additively-homomorphic commitment scheme for vectors over finite field* $\mathbb{F}$*. Consider size parameters* $n, m \in \mathbb{N}$*. We define the lookup relation* LKP *over public parameter, instance, witness tuples as follows*

$$\mathsf{LKP} = \left\{ (\mathsf{pp}, (\overline{t}, \overline{a}, \overline{v}), (t, a, v)) \;\middle|\; \begin{array}{l} v, a \in \mathbb{F}^m, t \in \mathbb{F}^n, \forall i \in [m].\ v_i = t_{a_i}, \\ \mathsf{Commit}(\mathsf{pp}, t) = \overline{t}, \mathsf{Commit}(\mathsf{pp}, a) = \overline{a}, \\ \mathsf{Commit}(\mathsf{pp}, v) = \overline{v} \end{array} \right\}.$$

Instead of directly proving the above relation, Lasso reformulates the lookup relation as the following bivariate polynomial identity test [57, Claim 3].

Lemma 10 (Lookup arithmetization [53,57]). *Consider vectors* $v, a \in \mathbb{F}^m$ *and* $t \in \mathbb{F}^n$*. There exist efficiently computable vectors* $c \in \mathbb{F}^m$ *and* $f \in \mathbb{F}^n$ *such*

that $v_i = t_{a_i}$ *for all* $i \in [m]$ *if and only if*

$$\left(\prod_{i\in[n]} (i + t_i \cdot X - Y)\right) \cdot \left(\prod_{i\in[m]} (a_i + v_i \cdot X + (c_i + 1) \cdot X^2 - Y)\right) = \left(\prod_{i\in[m]} (a_i + v_i \cdot X + c_i \cdot X^2 - Y)\right) \cdot \left(\prod_{i\in[n]} (i + t_i \cdot X + f_i \cdot X^2 - Y)\right). \tag{10}$$

Checking Equation (10) can be reduced to checking four grand product instances (Definition 11). In particular, the prover first computes c and f as guaranteed by Lemma 10 and send the corresponding commitments $\overline{c}$ and $\overline{f}$. The verifier samples and sends challenges $r_X \in \mathbb{F}$ and $r_Y \in \mathbb{F}$ to check Equation (10) at a single random point. By the Schwartz-Zippel lemma, this implies checking the original polynomial identity with overwhelming probability. The prover then sends claimed products p_1, p_2, p_3, and p_4 for each of the grand products in Equation (10) The verifier checks that $p_1 \cdot p_2 = p_3 \cdot p_4$, and then homomorphically compute and output commitments to each of the inner vectors to be checked in constant time. Formally, we have the following reduction.

Construction 6 (From Lookup to grand-product). We construct a reduction of knowledge of type

$$\mathsf{LKP} \to \mathsf{GP}^4.$$

Consider $n, m \in \mathbb{N}$. Let $(\mathsf{Gen}, \mathsf{Commit})$ denote an arbitrary additively homomorphic commitment scheme. We define the $\mathcal{G}(\lambda, (n, m)) = \mathsf{Gen}(\lambda, \max(n, m))$ and the encoder, prover, and verifier as follows.

$\underline{\mathcal{K}(\mathsf{pp})}$:

1. Compute vector commitments

$$\begin{aligned} G_n &\leftarrow \mathsf{Commit}(\mathsf{pp}, 1^n) \\ G_m &\leftarrow \mathsf{Commit}(\mathsf{pp}, 1^m) \\ H &\leftarrow \mathsf{Commit}(\mathsf{pp}, (1, \ldots, n)). \end{aligned}$$

   $G_n \leftarrow \mathsf{Commit}(\mathsf{pp}, 1^n)$, $G_m \leftarrow \mathsf{Commit}(\mathsf{pp}, 1^m)$, $H \leftarrow \mathsf{Commit}(\mathsf{pp}, (1, \ldots, n))$.

2. Output $(\mathsf{pk}, \mathsf{vk}) \leftarrow ((\mathsf{pp}, (G_n, G_m, H)), (G_n, G_m, H))$.

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), (\overline{t}, \overline{a}, \overline{v}), (t, a, v))}$:

1. $\mathcal{P}$: Compute $c \in \mathbb{F}^m$ and $f \in \mathbb{F}^n$ as guaranteed by Lemma 10 and send $\overline{c} \leftarrow \mathsf{Commit}(\mathsf{pp}, c)$ and $\overline{f} \leftarrow \mathsf{Commit}(\mathsf{pp}, f)$.

2. $\mathcal{V}$: Send random challenges $r_X \in \mathbb{F}$ and $r_Y \in \mathbb{F}$.

3. $\mathcal{P}$: Compute the underlying grand-product witnesses

$$\begin{aligned}
v_1 &\leftarrow \{(i + t_i \cdot r_X - r_Y)\}_{i\in[n]} \\
v_2 &\leftarrow \{(a_i + v_i \cdot r_X + (c_i + 1) \cdot r_X^2 - r_Y)\}_{i\in[m]} \\
v_3 &\leftarrow \{(a_i + v_i \cdot r_X + c_i \cdot r_X^2 - r_Y)\}_{i\in[m]} \\
v_4 &\leftarrow \{(i + t_i \cdot r_X + f_i \cdot r_X^2 - r_Y)\}_{i\in[n]}.
\end{aligned}$$

4. $\mathcal{P}$: Compute and send the claimed products $p_k \leftarrow \prod_i v_{k,i}$ for $k \in \{1, 2, 3, 4\}$.
5. $\mathcal{V}$: Check that $p_1 \cdot p_2 = p_3 \cdot p_4$.
6. $\mathcal{P}, \mathcal{V}$: Compute the corresponding vector commitments

$$\begin{aligned}
\overline{v}_1 &\leftarrow H + \overline{t} \cdot r_X - G_n \cdot r_Y \\
\overline{v}_2 &\leftarrow \overline{a} + \overline{v} \cdot r_X + (\overline{c} + G_m) \cdot r_X^2 - G_m \cdot r_Y \\
\overline{v}_3 &\leftarrow \overline{a} + \overline{v} \cdot r_X + \overline{c} \cdot r_X^2 - G_m \cdot r_Y \\
\overline{v}_4 &\leftarrow H + \overline{t} \cdot r_X + \overline{f} \cdot r_X^2 - G_n \cdot r_Y.
\end{aligned}$$

7. $\mathcal{P}, \mathcal{V}$: Output the following grand-product instance-witness pairs (the verifier only outputs instances)

$$\{((p_k, \overline{v}_k), v_k)\}_{k\in[4]} \in \mathsf{GP}^4$$

We formally prove the following lemma in Appendix B.4, which, by Lemma 9, and Corollary 1 induces the desired folding scheme for the lookup relation.

Lemma 11 (From lookup to grand-product). *Construction 6 is an RoK of type* $\mathsf{LKP} \to \mathsf{GP}^4$ *with* 2 *rounds, a communication cost of* 6 *field elements, and* 2 *elements in the message space of the commitment scheme over vectors of size* n *and* m*. The prover's time complexity is* $O(n + m)$ *field operations, time to compute commitments to vectors of size* n *and* m*, and* $O(1)$ *operations in the message space of the commitment scheme. The verifier's time complexity is* $O(1)$*.*

By Lemma 9, we have a reduction of knowledge from lookup to four zero-check instances. By Corollary 1, we get the desired folding scheme for the lookup relation. However, by our optimization in Section 5, to combine two sum-check reductions into one, we can only fold the same number of "fresh" instances and "running" instances. However, in our setting, we must fold k lookup instances into $4k$ running instances. This is not an issue as we can always generate trivially satisfying running instances or forgo the optimization to remove this dependence.

6.4 Reducing lookups to zero-check and sum-check via logarithmic derivatives

We present an alternative reduction from lookups to zero-check, based on the logarithmic derivatives technique of Haböck [41]. Compared to the grand-product-based reduction in Section 6.3, this approach avoids grand-products entirely and instead reduces a lookup to two zero-check instances and one sum-check instance.

The key observation is that the lookup relation $v_i = t_{a_i}$ can be reformulated as an equality of rational sums.

Lemma 12 (Lookup arithmetization via logarithmic derivatives [41]). *Consider vectors $v, a \in \mathbb{F}^m$ and $t \in \mathbb{F}^n$. Define the count vector $\mathsf{cnt} \in \mathbb{F}^n$ where cnt_j is the number of times the j-th table entry was accessed, i.e., $\mathsf{cnt}_j = |\{i \in [m] \mid a_i = j\}|$ for each $j \in [n]$. Then $v_i = t_{a_i}$ for all $i \in [m]$ if and only if*

$$\sum_{i \in [m]} \frac{1}{a_i + v_i \cdot X + Y} = \sum_{j \in [n]} \frac{\mathsf{cnt}_j}{j + t_j \cdot X + Y} \tag{11}$$

as a polynomial identity in formal variables X and Y.

To check Equation (11), we clear the denominators using committed inverse vectors. Specifically, the prover first computes and commits to the count vector cnt. The verifier then samples random challenges $r_X, r_Y \in \mathbb{F}$ to evaluate Equation (11) at a single random point. By the Schwartz-Zippel lemma, this implies checking the original identity with overwhelming probability. The prover then computes and commits to two vectors of inverses: $h \in \mathbb{F}^m$ where $h_i = (a_i + r_X \cdot v_i + r_Y)^{-1}$, and $e \in \mathbb{F}^n$ where $e_j = (j + r_X \cdot t_j + r_Y)^{-1}$. The correctness of these inverses is verified with two zero-check instances:

$$h_i \cdot (a_i + r_X \cdot v_i + r_Y) - 1 = 0 \quad \text{for all } i \in [m], \tag{12}$$

$$e_j \cdot (j + r_X \cdot t_j + r_Y) - 1 = 0 \quad \text{for all } j \in [n]. \tag{13}$$

Finally, the grand-sum equality from Equation (11) becomes a sum-check instance with target value $T = 0$:

$$\sum_{x \in \{0,1\}^\ell} \left(\widetilde{h}(x) - \widetilde{\mathsf{cnt}}(x) \cdot \widetilde{e}(x) \right) = 0 \tag{14}$$

where $\ell = \max(\log m, \log n)$ and vectors are zero-padded as needed.

Formally, we have the following reduction.

Construction 7 (From lookup to zero-check and sum-check via logarithmic derivatives). We construct a reduction of knowledge of type

$$\mathsf{LKP} \to \mathsf{ZC}^2 \times \mathsf{SC}.$$

Consider $n, m \in \mathbb{N}$. Let $\ell = \max(\log m, \log n)$. Let $(\mathsf{Gen}, \mathsf{Commit})$ denote an arbitrary additively homomorphic commitment scheme. We define $\mathcal{G}(\lambda, (n, m)) = \mathsf{Gen}(\lambda, \max(n, m))$ and the encoder, prover, and verifier as follows.

$\underline{\mathcal{K}(\mathsf{pp})}$:

1. Compute $G_n \leftarrow \mathsf{Commit}(\mathsf{pp}, 1^n)$, $G_m \leftarrow \mathsf{Commit}(\mathsf{pp}, 1^m)$, $H \leftarrow \mathsf{Commit}(\mathsf{pp}, (1, \ldots, n))$.
2. Define the zero-check structure polynomials $F_1(g_1, g_2) = g_1 \cdot g_2 - 1$ and $F_2(g_3, g_4) = g_3 \cdot g_4 - 1$.

3. Define the sum-check structure polynomial $F_{\mathsf{GS}}(g_5, g_6, g_7) = g_5 - g_6 \cdot g_7$.
4. Output $(\mathsf{pk}, \mathsf{vk}) \leftarrow ((\mathsf{pp}, G_n, G_m, H), (G_n, G_m, H))$ and the structure polynomials $(F_1, F_2, F_{\mathsf{GS}})$.

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), (\overline{t}, \overline{a}, \overline{v}), (t, a, v))}$:

1. $\mathcal{P}$: Compute the count vector $\mathsf{cnt} \in \mathbb{F}^n$ as defined in Lemma 12 and send $\overline{\mathsf{cnt}} \leftarrow \mathsf{Commit}(\mathsf{pp}, \mathsf{cnt})$.
2. $\mathcal{V}$: Send random challenges $r_X, r_Y \in \mathbb{F}$.
3. $\mathcal{P}$: Compute the inverse vectors $h \in \mathbb{F}^m$ where $h_i = (a_i + r_X \cdot v_i + r_Y)^{-1}$ and $e \in \mathbb{F}^n$ where $e_j = (j + r_X \cdot t_j + r_Y)^{-1}$. Send $\overline{h} \leftarrow \mathsf{Commit}(\mathsf{pp}, h)$ and $\overline{e} \leftarrow \mathsf{Commit}(\mathsf{pp}, e)$.
4. $\mathcal{P}, \mathcal{V}$: Define the inner structure polynomials (which depend on r_X, r_Y): G_1 on input $((h, a, v, 1^m), \perp)$ outputs $g_1(x) \leftarrow \widetilde{h}(x)$ and $g_2(x) \leftarrow \widetilde{a}(x) + r_X \cdot \widetilde{v}(x) + r_Y$; G_2 on input $((e, t, \iota, 1^n), \perp)$ outputs $g_3(x) \leftarrow \widetilde{e}(x)$ and $g_4(x) \leftarrow \widetilde{\iota}(x) + r_X \cdot \widetilde{t}(x) + r_Y$, where $\iota = (1, \ldots, n)$; G_{GS} on input $((h, \mathsf{cnt}, e), \perp)$ outputs $g_5(x) \leftarrow \widetilde{h}(x)$, $g_6(x) \leftarrow \widetilde{\mathsf{cnt}}(x)$, and $g_7(x) \leftarrow \widetilde{e}(x)$, all over the Boolean hypercube $\{0,1\}^\ell$ (padding vectors with zeros to length 2^ℓ as needed).
5. $\mathcal{P}, \mathcal{V}$: Output the following instance-witness pairs (the verifier only outputs instances):
   (a) Zero-check: $(((\overline{h}, \overline{a}, \overline{v}, G_m), \perp), (h, a, v, 1^m)) \in \mathsf{ZC}$ with structure (F_1, G_1).
   (b) Zero-check: $(((\overline{e}, \overline{t}, H, G_n), \perp), (e, t, \iota, 1^n)) \in \mathsf{ZC}$ with structure (F_2, G_2).
   (c) Sum-check: $((0, (\overline{h}, \overline{\mathsf{cnt}}, \overline{e}), \perp), (h, \mathsf{cnt}, e)) \in \mathsf{SC}$ with structure $(F_{\mathsf{GS}}, G_{\mathsf{GS}})$ and target $T = 0$.

Lemma 13 (From lookup to zero-check and sum-check). *Construction 7 is an RoK of type* $\mathsf{LKP} \to \mathsf{ZC}^2 \times \mathsf{SC}$ *with* 2 *rounds, a communication cost of* 2 *field elements and* 3 *elements in the message space of the commitment scheme over vectors of size* n *and* m*. The prover's time complexity is* $O(n + m)$ *field operations and the time to compute commitments to vectors of sizes* n*,* n*, and* m*. The verifier's time complexity is* $O(1)$*.*

Acknowledgments. We thank Justin Thaler for helpful conversations and comments on a prior version of this paper. Abhiram Kothapalli was supported by the Carnegie Mellon University (CMU) Secure Blockchain Initiative and Anaxi Labs through the CMU CyLab partnership program.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Nova: Recursive SNARKs without trusted setup. https://github.com/Microsoft/Nova
2. Ova: A slightly better Nova. https://hackmd.io/V4838nnlRKal9ZiTHiGYzw
3. sonobe. https://github.com/privacy-scaling-explorations/sonobe/
4. The Nexus zkVM. https://github.com/nexus-xyz/nexus-zkvm (2024)
5. Hobbit: Space-efficient zkSNARK with optimal prover time. Cryptology ePrint Archive (2025)
6. Arun, A., Setty, S.: Nebula: Efficient read-write memory and switchboard circuits for folding schemes. Cryptology ePrint Archive (2024)
7. Arun, A., Setty, S., Thaler, J.: Jolt: SNARKs for VMs via lookups. In: EUROCRYPT (2024)
8. Babai, L., Fortnow, L., Lund, C.: Non-deterministic exponential time has two-prover interactive protocols. Computational Complexity **2**(4) (Dec 1992)
9. Babai, L., Fortnow, L., Levin, L.A., Szegedy, M.: Checking computations in polylogarithmic time. In: STOC (1991)
10. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable zero knowledge with no trusted setup. In: CRYPTO (2019)
11. Ben-Sasson, E., Chiesa, A., Tromer, E., Virza, M.: Scalable zero knowledge via cycles of elliptic curves. In: CRYPTO (2014)
12. Bitansky, N., Canetti, R., Chiesa, A., Tromer, E.: Recursive composition and bootstrapping for SNARKs and proof-carrying data. In: STOC (2013)
13. Blum, M., Evans, W., Gemmell, P., Kannan, S., Naor, M.: Checking the correctness of memories. In: FOCS (1991)
14. Blumberg, A.J., Thaler, J., Vu, V., Walfish, M.: Verifiable computation using multiple provers. ePrint Report 2014/846 (2014)
15. Boneh, D., Chen, B.: LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. Cryptology ePrint Archive, Paper 2024/257 (2024)
16. Boneh, D., Chen, B.: LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. Cryptology ePrint Archive, Paper 2025/247 (2025)
17. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: EUROCRYPT (2016)
18. Bootle, J., Chiesa, A., Sotiraki, K.: Sumcheck arguments and their applications. In: CRYPTO. pp. 742–773 (2021)
19. Bowe, S., Grigg, J., Hopwood, D.: Halo: Recursive proof composition without a trusted setup. Cryptology ePrint Archive, Report 2019/1021 (2019)
20. Bünz, B., Chen, B.: Protostar: Generic efficient accumulation/folding for special sound protocols. In: ASIACRYPT (2023)
21. Bünz, B., Chen, J.: Proofs for deep thought: Accumulation for large memories and deterministic computations. In: ASIACRYPT (2024)
22. Bünz, B., Chiesa, A., Lin, W., Mishra, P., Spooner, N.: Proof-carrying data without succinct arguments. In: CRYPTO (2021)
23. Bünz, B., Chiesa, A., Mishra, P., Spooner, N.: Proof-carrying data from accumulation schemes. In: TCC (2020)
24. Bünz, B., Mishra, P., Nguyen, W., Wang, W.: Accumulation without homomorphism. In: ITCS (2024)
25. Bunz, B., Mishra, P., Nguyen, W., Wang, W.: Arc: Accumulation for reed–solomon codes. In: CRYPTO (2025)

26. Buterin, V.: On-chain scaling to potentially 500 tx/sec through mass tx validation. https://ethresear.ch/t/on-chain-scaling-to-potentially-500-tx-sec-through-mass-tx-validation/3477 (Sep 2018)
27. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In: EUROCRYPT (2023)
28. Chiesa, A., Forbes, M.A., Spooner, N.: A zero knowledge sumcheck and its applications. CoRR **abs/1704.02086** (2017)
29. Daix-Moreux, P., Zhang, C.: PlasmaFold: An efficient and scalable layer 2 with client-side proving. Cryptology ePrint Archive, Paper 2025/1300 (2025), https://eprint.iacr.org/2025/1300
30. Dao, Q., Thaler, J.: More optimizations to sum-check proving. Cryptology ePrint Archive, Paper 2024/1210 (2024)
31. Diamond, B.E., Posen, J.: Succinct arguments over towers of binary fields. Cryptology ePrint Archive, Paper 2023/1784 (2023)
32. Dimitriou, N., Garreta, A., Manzur, I., Vlasov, I.: Mova: Nova folding without committing to error terms. Cryptology ePrint Archive, Paper 2024/1220 (2024)
33. Eagen, L., Gabizon, A.: Protogalaxy: Efficient ProtoStar-style folding of multiple instances. Cryptology ePrint Archive, Paper 2023/1106 (2023)
34. Eagen, L., Gabizon, A., Sefranek, M., Towa, P., Williamson, Z.J.: Stackproofs: Private proofs of stack and contract execution using protogalaxy. Cryptology ePrint Archive, Paper 2024/1281 (2024)
35. Gabizon, A., Williamson, Z.J.: plookup: A simplified polynomial protocol for lookup tables. Cryptology ePrint Archive (2020)
36. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive (2019)
37. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct NIZKs without PCPs. In: EUROCRYPT (2013)
38. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating computation: Interactive proofs for muggles. In: STOC (2008)
39. Golovnev, A., Lee, J., Setty, S., Thaler, J., Wahby, R.: Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In: CRYPTO (2023)
40. Gruen, A.: Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108 (2024)
41. Haböck, U.: Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530 (2022)
42. Kaviani, D., Setty, S.: Vega: Low-latency zero-knowledge proofs over existing credentials. Cryptology ePrint Archive (2025), https://eprint.iacr.org/2025/2094
43. Kothapalli, A., Parno, B.: Algebraic reductions of knowledge. In: CRYPTO (2023)
44. Kothapalli, A., Setty, S.: SuperNova: Proving universal machine executions without universal circuits. Cryptology ePrint Archive (2022)
45. Kothapalli, A., Setty, S.: Cyclefold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. Cryptology ePrint Archive, Paper 2023/1192 (2023), https://eprint.iacr.org/2023/1192, https://eprint.iacr.org/2023/1192
46. Kothapalli, A., Setty, S.: Hypernova: Recursive arguments for customizable constraint systems. In: CRYPTO. pp. 345–379 (2024)
47. Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In: CRYPTO (2022)

48. Lee, J., Nikitin, K., Setty, S.: Replicated state machines without replicated execution. In: S&P (2020)
49. Lund, C., Fortnow, L., Karloff, H., Nisan, N.: Algebraic methods for interactive proof systems. In: FOCS (Oct 1990)
50. Nguyen, W., Datta, T., Chen, B., Tyagi, N., Boneh, D.: Mangrove: A scalable framework for folding-based SNARKs. In: CRYPTO (2024)
51. Nguyen, W., Setty, S.: Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Paper 2025/294 (2025), https://eprint.iacr.org/2025/294
52. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. J. ACM **27**(4) (1980)
53. Setty, S.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: CRYPTO (2020)
54. Setty, S., Angel, S., Gupta, T., Lee, J.: Proving the correct execution of concurrent services in zero-knowledge. In: OSDI (Oct 2018)
55. Setty, S., Lee, J.: Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275 (2020)
56. Setty, S., Thaler, J., Wahby, R.: Customizable constraint systems for succinct arguments. Cryptology ePrint Archive (2023)
57. Setty, S., Thaler, J., Wahby, R.: Unlocking the lookup singularity with Lasso. In: EUROCRYPT (2024)
58. Shamir, A.: IP = PSPACE. J. ACM **39**(4), 869–877 (1992)
59. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: CRYPTO (2013)
60. Valiant, P.: Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In: TCC. pp. 552–576 (2008)
61. Vu, V., Setty, S., Blumberg, A.J., Walfish, M.: A hybrid architecture for verifiable computation. In: S&P (2013)
62. Wei, Y., Wang, K., Xiang, B., Zhang, X., Wang, H., Deng, Y., Zhu, X., Lin, L.: Packed sumcheck over fields of small characteristic with application to verifiable FHE. Cryptology ePrint Archive (2025), https://eprint.iacr.org/2025/719
63. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct zero-knowledge proofs with optimal prover computation. In: CRYPTO (2019)
64. Zhang, Y., Vark, A.: Parallelizing Nova – Visualizations and Mental Models behind Paranova. https://zkresear.ch/t/parallelizing-nova-visualizations-and-mental-models-behind-paranova/198 (2023)
65. Zhao, J., Setty, S., Cui, W., Zaverucha, G.: MicroNova: Folding-based arguments with efficient (on-chain) verification. In: S&P (2025)
66. Zhou, Z., Zhang, Z., Dong, J.: Proof-carrying data from multi-folding schemes. Cryptology ePrint Archive, Paper 2023/1282 (2023)

A Additional Background

A.1 Polynomials and multilinear extensions

We adapt this subsection from prior work [53]. We recall several definitions and results regarding multivariate polynomials.

Definition 13 (Multilinear polynomial). *A multivariate polynomial is called a multilinear polynomial if the degree of the polynomial in each variable is at most one.*

Definition 14 (Multilinear polynomial extension). *Given a vector $v \in \mathbb{F}^n$ a* multilinear polynomial extension *of v is an $(\log n)$-variate multilinear polynomial, denoted $\widetilde{v}$, such that $\widetilde{v}(x) = v_x$ for all $x \in \{0,1\}^{\log n}$. Specifically, $\widetilde{v}$ can be computed as follows.*

$$\widetilde{v}(x) = \sum_{y \in \{0,1\}^\ell} v_y \cdot \mathsf{eq}(x, y)$$

where $\mathsf{eq}(x,y) = \prod_{i=1}^{\ell}(x_i \cdot y_i + (1-x_i)\cdot(1-y_i))$, outputs 1 if $x = y$ and 0 otherwise for $x, y \in \{0,1\}^{\log n}$.

For any $r \in \mathbb{F}^\ell$, $\widetilde{v}(r)$ can be computed in $O(2^\ell)$ operations in $\mathbb{F}$ [61,59].

Lemma 14 (Schwartz-Zippel [52]). *let $g : \mathbb{F}^\ell \to \mathbb{F}$ be an ℓ-variate polynomial of total degree at most d. Then, on any finite set $S \subseteq \mathbb{F}$,*

$$\Pr_{x \leftarrow S^\ell}[g(x) = 0] \leq d/|S|.$$

A.2 Commitment Schemes

Definition 15 (Commitment Scheme). *A commitment scheme is defined by polynomial-time algorithm $\mathsf{Gen} : \mathbb{N}^2 \to P$ that produces public parameters given the security parameter and size parameter, a deterministic polynomial-time algorithm $\mathsf{Commit} : P \times M \times R \to C$ that produces a commitment in C given a public parameters, message, and randomness tuple such that binding holds. That is, for any PPT adversary $\mathcal{A}$, given $\mathsf{pp} \leftarrow \mathsf{Gen}(\lambda, n)$, and given $((m_1, r_1), (m_2, r_2)) \leftarrow \mathcal{A}(\mathsf{pp})$ we have that*

$$\Pr[(m_1, r_1) \neq (m_2, r_2) \wedge \mathsf{Commit}(\mathsf{pp}, m_1, r_1) = \mathsf{Commit}(\mathsf{pp}, m_2, r_2)] \approx 0.$$

The commitment scheme is deterministic if Commit does not use its randomness.

Definition 16 (Homomorphic). *The commitment scheme $(\mathsf{Gen}, \mathsf{Commit})$ is homomorphic if the message space M, randomness space R, and commitment space C are groups and for all $n \in \mathbb{N}$, and $\mathsf{pp} \leftarrow \mathsf{Gen}(\lambda, n)$, we have that for any $m_1, m_2 \in M$ and $r_1, r_2 \in R$*

$$\mathsf{Commit}(\mathsf{pp}, m_1, r_1) + \mathsf{Commit}(\mathsf{pp}, m_2, r_2) = \mathsf{Commit}(\mathsf{pp}, m_1 + m_2, r_1 + r_2).$$

Definition 17 (Succinct Commitments). *A commitment scheme* $(\mathsf{Gen}, \mathsf{Commit})$*, over message space* M *and commitment space* R*, provides succinct commitments if for all* $\mathsf{pp} \leftarrow \mathsf{Gen}(1^\lambda)$*, and any* $m \in M$ *and* $r \in R$*, we have that* $|\mathsf{Commit}(\mathsf{pp}, m, r)| = O_\lambda(\mathsf{polylog}(|m|))$*.*

A.3 Reductions of Knowledge

First, we define additional properties of reductions of knowledge.

Definition 18 (Succinctness). *A reduction of knowledge is succinct if the communication complexity and the verifier time complexity is at most polylogarithmic in the size of the structure and witness.*

Definition 19 (Non-interactivity). *A reduction of knowledge is non-interactive if the interaction consists of a single message from the prover to the verifier. In this case, we denote this single message as the output of the prover, and as an input to the verifier.*

Definition 20 (Public-coin). *A reduction of knowledge is public-coin if the verifier only sends uniformly random challenges during the interaction.*

Next, we define the semantics of the sequential and parallel composition operators.

Lemma 15 (Sequential composition [43]). *For reductions* $\Pi_1 = (\mathcal{G}, \mathcal{K}_1, \mathcal{P}_1, \mathcal{V}_1) : \mathcal{R}_1 \to \mathcal{R}_2$ *and* $\Pi_2 = (\mathcal{G}, \mathcal{K}_2, \mathcal{P}_2, \mathcal{V}_2) : \mathcal{R}_2 \to \mathcal{R}_3$*, we have that* $\Pi_2 \circ \Pi_1 = (\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V}) : \mathcal{R}_1 \to \mathcal{R}_3$ *where* $\mathcal{K}(\mathsf{pp}, \mathsf{s}_1)$ *computes* $(\mathsf{pk}_1, \mathsf{vk}_1, \mathsf{s}_2) \leftarrow \mathcal{K}_1(\mathsf{pp}, \mathsf{s}_1)$*,* $(\mathsf{pk}_2, \mathsf{vk}_2, \mathsf{s}_3) \leftarrow \mathcal{K}_2(\mathsf{pp}, \mathsf{s}_2)$ *and outputs* $((\mathsf{pk}_1, \mathsf{pk}_2), (\mathsf{vk}_1, \mathsf{vk}_2), \mathsf{s}_3)$ *and where*

$$\mathcal{P}((\mathsf{pk}_1, \mathsf{pk}_2), u_1, w_1) = \mathcal{P}_2(\mathsf{pk}_2, \mathcal{P}_1(\mathsf{pk}_1, u_1, w_1))$$
$$\mathcal{V}((\mathsf{vk}_1, \mathsf{vk}_2), u_1) = \mathcal{V}_2(\mathsf{vk}_2, \mathcal{V}_1(\mathsf{vk}_1, u_1, w_1))$$

Lemma 16 (Parallel composition [43]). *Consider relations* $\mathcal{R}_1$*,* $\mathcal{R}_2$*,* $\mathcal{R}_3$*, and* $\mathcal{R}_4$*. For reductions of knowledge* $\Pi_1 = (\mathcal{G}, \mathcal{K}, \mathcal{P}_1, \mathcal{V}_1) : \mathcal{R}_1 \to \mathcal{R}_2$ *and* $\Pi_2 = (\mathcal{G}, \mathcal{K}, \mathcal{P}_2, \mathcal{V}_2) : \mathcal{R}_3 \to \mathcal{R}_4$ *we have that* $\Pi_1 \times \Pi_2 = (\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V}) : \mathcal{R}_1 \times \mathcal{R}_3 \to \mathcal{R}_2 \times \mathcal{R}_4$ *where*

$$\mathcal{P}(\mathsf{pk}, (u_1, u_3), (w_1, w_3)) = (\mathcal{P}_1(\mathsf{pk}, u_1, w_1), \mathcal{P}_2(\mathsf{pk}, u_3, w_3))$$
$$\mathcal{V}(\mathsf{vk}, (u_1, u_3)) = (\mathcal{V}_1(\mathsf{vk}, u_1), \mathcal{V}_2(\mathsf{vk}, u_3))$$

Next, we can define arguments of knowledge as a special type of reduction of knowledge.

Definition 21 (Argument of Knowledge). *Consider the boolean relation* $\mathsf{TRUE} = \{(\mathsf{true}, \bot)\}$*. A proof of knowledge for relation* $\mathcal{R}$ *is a reduction of knowledge of type* $\mathcal{R} \to \mathsf{TRUE}$*.*

When proving the security of protocols, reasoning about knowledge soundness directly is typically cumbersome. To alleviate this issue, Bootle et al. [17] show that to prove knowledge soundness for the vast majority of public-coin interactions, it is sufficient to show that there exists an extractor that can produce a satisfying witness when provided a tree of accepting transcripts with refreshed verifier randomness at each layer. This property is known as *tree-extractability* and we formally state the corresponding result for reductions of knowledge (proven by Kothapalli and Parno [43]).

Definition 22 (Tree of transcripts). *Consider an m-round public-coin interactive protocol $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ that satisfies the interface described in Definition 1. For a public parameter, structure, instance tuple $(\mathsf{pp}, \mathsf{s}, u_1)$, a $(n_1, \ldots, n_m)$-tree of accepting transcripts is a tree of depth m where each vertex at layer i has n_i outgoing edges such that (1) each vertex in layer $i \in [m]$ is labeled with a prover message for round i; (2) each outgoing edge from layer $i \in [m]$ is labeled with a different choice of verifier randomness for round i; (3) each leaf is labeled with an accepting statement-witness pair output by the prover and verifier corresponding to the interaction along the path.*

Lemma 17 (Tree extraction [43]). *Consider an m-round public-coin interactive protocol $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ that satisfies the interface described in Definition 1 and satisfies completeness. Then $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ is a reduction of knowledge if there exists a PPT extractor χ that outputs a satisfying witness w_1 with probability $1 - \mathsf{negl}(\lambda)$, given an $(n_1, \ldots, n_m)$-tree of accepting transcripts for public parameter, structure, instance tuple $(\mathsf{pp}, \mathsf{s}, u_1)$ where the verifier's randomness is sampled from space Q such that $|Q| = O(2^\lambda)$, and $\prod_i n_i = \mathsf{poly}(\lambda)$.*

A.4 Incrementally Verifiable Computation

Recall that incrementally verifiable computation (IVC), enables a prover to produce a proof π_{i+1} for $i+1$ steps of a computation given a proof π_i for i steps of the computation in a way that ensures that the proof size remains independent of the total steps of computation. As IVC is a major application of folding schemes [22,47,46], for completeness, we recall the formal definition below.

Definition 23 (Incrementally verifiable computation (IVC)). *An incrementally verifiable computation (IVC) scheme is defined by PPT algorithms $(\mathcal{G}, \mathcal{P}, \mathcal{V})$ and deterministic $\mathcal{K}$ denoting the generator, the prover, the verifier, and the encoder respectively, with the following interface*

- $\mathcal{G}(1^\lambda, N) \to \mathsf{pp}$*: on input security parameter λ and size bounds N, samples public parameters* pp*.*
- $\mathcal{K}(\mathsf{pp}, F) \to (\mathsf{pk}, \mathsf{vk})$*: on input public parameters* pp*, and polynomial-time function F, deterministically produces a prover key* pk *and a verifier key* vk*.*
- $\mathcal{P}(\mathsf{pk}, (i, z_0, z_i), \omega_i, \Pi_i) \to \Pi_{i+1}$*: on input a prover key* pk*, a counter i, an initial input z_0, a claimed output after i iterations z_i, a non-deterministic*

advice ω_i, and an IVC proof Π_i attesting to z_i, produces a new proof Π_{i+1} attesting to $z_{i+1} = F(z_i, \omega_i)$.

- $\mathcal{V}(\mathsf{vk}, (i, z_0, z_i), \Pi_i) \to \{0, 1\}$*: on input a verifier key* vk*, a counter* i*, an initial input* z_0*, a claimed output after* i *iterations* z_i*, and an IVC proof* Π_i *attesting to* z_i*, outputs* 1 *if* Π_i *is accepting, and* 0 *otherwise.*

An IVC scheme $(\mathcal{G}, \mathcal{K}, \mathcal{P}, \mathcal{V})$ satisfies the following requirements.

1. *Perfect Completeness: For any PPT adversary $\mathcal{A}$*

$$\Pr\left[\mathcal{V}(\mathsf{vk}, (i+1, z_0, z_{i+1}), \Pi_{i+1}) = 1 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda, N), \\ F, (i, z_0, z_i, \Pi_i) \leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, F), \\ z_{i+1} \leftarrow F(z_i, \omega_i), \\ \mathcal{V}(\mathsf{vk}, i, z_0, z_i, \Pi_i) = 1, \\ \Pi_{i+1} \leftarrow \mathcal{P}(\mathsf{pk}, (i, z_0, z_i), \omega_i, \Pi_i) \end{array}\right] = 1$$

   where F is a polynomial-time computable function represented as an arithmetic circuit.

2. *Knowledge Soundness: Consider constant $n \in \mathbb{N}$. For all expected polynomial-time adversaries $\mathcal{P}^*$ there exists an expected polynomial-time extractor $\mathcal{E}$ such that over all randomness ρ*

$$\Pr\left[\begin{array}{l} z_n = z \text{ where} \\ z_{i+1} \leftarrow F(z_i, \omega_i) \\ \forall i \in \{0, \ldots, n-1\} \end{array} \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathcal{G}(1^\lambda), \\ (F, (z_0, z_i), \Pi) \leftarrow \mathcal{P}^*(\mathsf{pp}, \rho), \\ (\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, F), \\ \mathcal{V}(\mathsf{vk}, (n, z_0, z), \Pi) = 1, \\ (\omega_0, \ldots, \omega_{n-1}) \leftarrow \mathcal{E}(\mathsf{pp}, \rho) \end{array}\right] \approx 1.$$

   Moreover, F is a polynomial-time computable function represented as an arithmetic circuit.

3. *Succinctness: The size of an IVC proof Π is independent of the number of iterations n.*

B Deferred Proofs

B.1 Proof of Theorem 2 (SumFold)

Lemma 18 (Completeness). *Construction 1 is complete.*

Proof. Consider an arbitrary generator and encoder algorithm $(\mathcal{G}, \mathcal{K})$, and an arbitrary PPT adversary $\mathcal{A}$. For an arbitrary size parameter N, and a structure (F, G), let $\mathsf{pp} \leftarrow \mathcal{G}(\lambda, N)$ and $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, (F, G))$. Suppose now that the adversary $\mathcal{A}$ on input pp generates input instance-witness pairs

$$(\{(T_i, \vec{u}_i, \mathsf{x}_i)\}_{i \in [k]}, \{\vec{w}_i\}_{i \in [k]}) \in \mathsf{SC}^k. \tag{15}$$

The prover and the verifier on input $(\mathsf{pk}, \mathsf{vk}), \{(T_i, \vec{u}_i, \mathsf{x}_i)\}_{i\in[k]}, \{\vec{w}_i\}_{i\in[k]}$ interactively produce the output instance-witness pairs

$$((T', \vec{u}, \mathsf{x}), \vec{w}).$$

We must show that $((T', \vec{u}, \mathsf{x}), \vec{w}) \in \mathsf{SC}$.

Indeed, by the linearity of the commitment scheme and by the satisfiability of the input instances, we have that for $j \in [s]$

$$\begin{aligned} u_j &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot u_{i,j} \\ &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot \mathsf{Commit}(\mathsf{pp}, w_{i,j}) \\ &= \mathsf{Commit}(\mathsf{pp}, \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot w_{i,j}) \\ &= \mathsf{Commit}(\mathsf{pp}, w_j). \end{aligned} \tag{16}$$

That is, we have that all output witnesses are satisfying openings to the output commitments.

Moreover, given $\vec{g}_i \leftarrow G(\vec{w}_i, \mathsf{x}_i)$, we have that

$$\begin{aligned} T &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(\rho, i) \cdot T_i && \text{By construction} \\ &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(\rho, i) \cdot \sum_{x\in\{0,1\}^\ell} F(\vec{g}_i(x)) && \text{By Precondition (15)} \\ &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(\rho, i) \cdot \sum_{x\in\{0,1\}^\ell} F(\vec{f}(i, x)) && \text{By construction} \\ &= \sum_{i\in\{0,1\}^\nu} Q(i) && \text{By construction} \end{aligned}$$

Therefore, by the completeness of the sum-check protocol, we have that

$$c = Q(r_b).$$

This means that

$$\begin{aligned} T' &= \mathsf{eq}(\rho, r_b)^{-1} \cdot c \\ &= \mathsf{eq}(\rho, r_b)^{-1} \cdot Q(r_b) \\ &= \sum_{x\in\{0,1\}^\ell} F(\vec{f}(r_b, x)). \end{aligned} \tag{17}$$

But, by the linearity of G (Definition 7), we have that

$$\begin{aligned} G(\vec{w}, \mathsf{x}) &= G(\sum_{i \in \{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot \vec{w}_i, \sum_{i \in \{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot \mathsf{x}_i) \\ &= \sum_{i \in \{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot G(\vec{w}_i, \mathsf{x}_i) \\ &= \sum_{i \in \{0,1\}^\nu} \mathsf{eq}(r_b, i) \cdot \vec{g}_i \\ &= \vec{f}(r_b, x). \end{aligned} \tag{18}$$

Thus, by derivations (16), (17), and (18), we have that $((T', \vec{u}, \mathsf{x}), \vec{w}) \in \mathsf{SC}$. □

Lemma 19 (Knowledge soundness). *Construction 1 is knowledge sound (and in particular tree-extractable).*

Proof. We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct a PPT extractor χ that outputs a satisfying input witness with probability $1 - \mathsf{negl}(\lambda)$ given a tree of accepting transcripts and the corresponding output instance-witness pairs.

Indeed, let χ_{sc} denote the tree-extractor for the sum-check protocol which reduces from the unstructured sum-check relation USC to the polynomial evaluation relation PE. Suppose that χ_{sc} can extract using an L-tree of accepting transcripts where $L \in \mathbb{N}^\nu$. Then, we provide χ with a $(2^\nu, L)$-tree of accepting transcripts for public parameter, structure, instance tuple $(\mathsf{pp}, (F, G), \{(T_i, \vec{u}_i, \mathsf{x}_i)\}_{i \in [k]})$. For $\kappa \in [2^\nu]$ and $l \in ([L_1], \ldots, [L_\nu])$, let $\rho^{(\kappa)}$ denote the verifier's first challenge, and for each such challenge let $r_b^{(\kappa,l)}$ denote the verifier's challenges during the sum-check protocol. Let

$$((T'^{(\kappa,l)}, \vec{u}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)}), \vec{w}^{(\kappa,l)}) \in \mathsf{SC} \tag{19}$$

denote the corresponding output instance-witness pairs.

For $\kappa \in [2^\ell]$, the extractor χ interpolates $\vec{w}_i^{(\kappa)}$ for $i \in [k]$ such that for $j \in [s]$

$$w_j^{(\kappa,l)} = \sum_{i \in \{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i) \cdot w_{i,j}^{(\kappa)}. \tag{20}$$

Now, by the satisfiability requirement of SC (Precondition 19), we have that

$$T'^{(\kappa,l)} = \sum_{x \in \{0,1\}^l} F(G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)})(x)) \tag{21}$$

$$u_j^{(\kappa,l)} = \mathsf{Commit}(\mathsf{pp}, w_j^{(\kappa,l)}) \tag{22}$$

Moreover, by the verifier's computation we have that

$$\begin{aligned} G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)}) &= G(\sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot \vec{w}_i^{(\kappa)}, \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot \mathsf{x}_i^{(\kappa)}) \quad \text{By (20)} \\ &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot G(\vec{w}_i^{(\kappa)}, \mathsf{x}_i^{(\kappa)}) \\ &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot \vec{g}_{i,1}^{(\kappa)} \\ &= \vec{f}'^{(\kappa)}(r_b^{(\kappa,l)}) \end{aligned}$$

Then, by the verifier's computation in Step 3 and Equation (21), this means that

$$\begin{aligned} c^{(\kappa,l)} &= \mathsf{eq}(\rho^{(\kappa)}, r_b^{(\kappa,l)})\cdot T'^{(\kappa,l)} \\ &= \mathsf{eq}(\rho^{(\kappa)}, r_b^{(\kappa,l)})\cdot \sum_{x\in\{0,1\}^\ell} F(G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)})(x)) \\ &= \mathsf{eq}(\rho^{(\kappa)}, r_b^{(\kappa,l)})\cdot \sum_{x\in\{0,1\}^\ell} F(\vec{f}'^{(\kappa)}(r_b^{(\kappa,l)}, x)) \\ &= Q^{(\kappa)}(r_b^{(\kappa,l)}) \end{aligned} \tag{23}$$

Moreover, by the verifier's computation for $i \in [k]$ and $j \in [s]$, we have that for all $r_b^{(\kappa,l)}$

$$\begin{aligned} \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot u_{i,j}^{(\kappa)} &= u_j^{(\kappa,l)} \\ &= \mathsf{Commit}(\mathsf{pp}, w_j^{(\kappa,l)}) && \text{By (22)} \\ &= \mathsf{Commit}(\mathsf{pp}, \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot w_{i,j}^{(\kappa)}) && \text{By (20)} \\ &= \sum_{i\in\{0,1\}^\nu} \mathsf{eq}(r_b^{(\kappa,l)}, i)\cdot \mathsf{Commit}(\mathsf{pp}, w_{i,j}^{(\kappa)}) && \text{By linearity} \end{aligned}$$

Then, by interpolation, we have that for all $i \in [k]$ and $j \in [s]$ that

$$u_{i,j}^{(\kappa)} = \mathsf{Commit}(\mathsf{pp}, w_{i,j}^{(\kappa)}). \tag{24}$$

Then, by Derivation (23) and Equation (24), we have that for each challenge $\rho^{(\kappa)}$, χ can produce an L-sub-tree of accepting transcripts for an input unstructured sum-check instance $(T^{(\kappa)}, \overline{Q}^{(\kappa)})$ where $\overline{Q}^{(\kappa)} = ((F, G), \rho^{(\kappa)}, (\vec{u}_i, \mathsf{x}_i)_{i\in[k]})$, with output polynomial evaluation instance-witness pairs

$$((\overline{Q}^{(\kappa)}, c^{(\kappa,l)}, r_b^{(\kappa,l)}), Q^{(\kappa)}) \in \mathsf{PE}$$

where $Q^{(\kappa)}$ is represented as the tuple $((F,G),\rho^{(\kappa)},(\vec{w}_i^{(\kappa)},\mathsf{x}_i)_{i\in[k]})$.

Now, for each challenge $\rho^{(\kappa)}$, given the corresponding L-sub-tree χ_{sc} can produce a witness $Q^{(\kappa)}$ represented as the tuple $((F,G),\rho^{(\kappa)},(\vec{w}_i^{(\kappa)},\mathsf{x}_i)_{i\in[k]})$ such that

$$((T^{(\kappa)},\overline{Q}^{(\kappa)}),Q^{(\kappa)})\in\mathsf{USC}. \tag{25}$$

Then, because $Q^{(\kappa)}$ must be a valid opening to $\overline{Q}^{(\kappa)}$ which contains the same commitment $\vec{u}_i$ across all transcripts by Equation (25), we must have that $\vec{w}_i^{(\kappa)}=\vec{w}_i^{(\kappa')}$ for all $\kappa,\kappa'\in[2^\ell]$. We denote this value across all transcripts simply as $\vec{w}_i$.

By the satisfiability condition of USC and the verifier's computation, this means that

$$\begin{aligned}\sum_{i\in\{0,1\}^\nu}\mathsf{eq}(\rho^{(\kappa)},i)\cdot T_i &= T^{(\kappa)}\\ &=\sum_{b\in\{0,1\}^\nu}Q^{(\kappa)}(\rho^{(\kappa)})\\ &=\sum_{b\in\{0,1\}^\nu}\mathsf{eq}(\rho^{(\kappa)},b)\cdot\sum_{x\in\{0,1\}^\ell}F(\vec{f}(b,x))\end{aligned}$$

Then, interpolating, we must have for $i\in\{0,1\}^\nu$

$$\begin{aligned}T_i &=\sum_{x\in\{0,1\}^\ell}(\vec{f}(i,x))\\ &=F(\vec{g}_i(x))\\ &=F(G(\vec{w}_i,\mathsf{x}_i)(x))\end{aligned}$$

This means that

$$(\{(T_i,\vec{u}_i,\mathsf{x}_i)\}_{i\in[k]},\{\vec{w}_i\}_{i\in[k]})\in\mathsf{SC}^k.$$

Thus, χ can compute and output a satisfying input witness $\vec{w}_i$ for $i\in[k]$. □

The prover-time bound follows from the cost of the sum-check protocol in Step 2. For each of the $kt2^\ell$ relevant evaluations, forming a degree-d round polynomial from d linear factors using a product tree and FFT-based polynomial multiplication takes $O(d\log^2 d)$ field operations. Thus, the prover uses $O(ktd\log^2 d\cdot 2^\ell)$ field operations.

B.2 Proof of Lemma 5 (Zero-check reduction)

Lemma 20 (Completeness). *Construction 3 is complete.*

Proof. Consider an arbitrary generator and encoder algorithms $(\mathcal{G},\mathcal{K})$, and an arbitrary PPT adversary $\mathcal{A}$. For an arbitrary size parameter N, let $\mathsf{pp}\leftarrow\mathcal{G}(\lambda,N)$.

Suppose now that the adversary $\mathcal{A}$ on input pp, generates input structure-instance-witness pairs

$$(F, G), (\vec{\mathsf{u}}, \vec{\mathsf{u}}_{\mathsf{pc}}), (\vec{\mathsf{w}}, \vec{\mathsf{w}}_{\mathsf{pc}}) \in \mathsf{ZC}^k \times \mathsf{ZC}^m_{\mathsf{PC}}. \tag{26}$$

Suppose that for $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, (F, G))$, the prover and the verifier on input $(\mathsf{pk}, \mathsf{vk}), (\vec{\mathsf{u}}, \vec{\mathsf{u}}_{\mathsf{pc}}), (\vec{\mathsf{w}}, \vec{\mathsf{w}}_{\mathsf{pc}})$ interactively produce the output instance-witness pairs

$$(\{(0, \mathsf{u}_i, \overline{e})\}_{i\in[k]}, \{(\mathsf{w}_i, e)\}_{i\in[k]}), (\{(0, \mathsf{u}_{\mathsf{pc},j}, \overline{e})\}_{j\in[m]}, \{(\mathsf{w}_{\mathsf{pc},j}, e)\}_{j\in[m]}), ((\overline{e}, \tau), e).$$

We must show that

$$(\{(0, \mathsf{u}_i, \overline{e})\}_{i\in[k]}, \{(\mathsf{w}_i, e)\}_{i\in[k]}) \in \mathsf{NSC}^k \tag{27}$$

$$(\{(0, \mathsf{u}_{\mathsf{pc},j}, \overline{e})\}_{j\in[m]}, \{(\mathsf{w}_{\mathsf{pc},j}, e)\}_{j\in[m]}) \in \mathsf{NSC}^m_{\mathsf{PC}} \tag{28}$$

$$((\overline{e}, \tau), e) \in \mathsf{ZC}_{\mathsf{PC}}. \tag{29}$$

Indeed, by the prover's construction (Step 2), we immediately have that Equation 29 holds.

Now, for each $i \in [k]$, we parse $(\vec{u}_i, \mathsf{x}_i) \leftarrow \mathsf{u}_i$, and $\vec{w}_i \leftarrow \mathsf{w}_i$. By Precondition (26) we have that

$$\mathsf{Commit}(\mathsf{pp}, w_{i,j}) = u_{i,j}. \tag{30}$$

Moreover, by the precondition, we have that for all $x \in \{0,1\}^\ell$

$$0 = F(G(\vec{w}_i, \mathsf{x}_i)(x)).$$

This means that for any choice of e

$$0 = \sum_{x\in\{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(G(\vec{w}_i, \mathsf{x}_i)(x)) \tag{31}$$

where x_1 and x_2 (likewise e_1 and e_2) represent the first and the second half of the original vector. Then, by Equations (30) and (31), we have that Equation (27) holds. By an identical line of reasoning, we have that Equation (28) holds as well. Thus, we have that completeness holds. □

Lemma 21 (Knowledge Soundness). *Construction 3 is knowledge sound.*

Proof. We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct a PPT extractor χ that outputs a satisfying input witness with probability $1 - \mathsf{negl}(\lambda)$ given a tree of accepting transcripts and the corresponding output instance-witness pairs.

Indeed, suppose χ is provided with a (2^ℓ)-tree of accepting transcripts for public parameter, structure, instance tuple $(\mathsf{pp}, (F, G), (\{\vec{\mathsf{u}}_i\}_{i\in[k]}, \{\vec{\mathsf{u}}_{\mathsf{pc},j}\}_{j\in[m]}))$. For $\kappa \in$

$[2^\ell]$, let $\tau^{(\kappa)}$ denote the verifier challenge in each of these transcripts, and let

$$(\{(0, \mathsf{u}_i^{(\kappa)}, \overline{e}^{(\kappa)})\}_{i\in[k]}, \{(\mathsf{w}_i^{(\kappa)}, e^{(\kappa)})\}_{i\in[k]}) \in \mathsf{NSC}^k \tag{32}$$

$$(\{(0, \mathsf{u}_{\mathsf{pc},j}^{(\kappa)}, \overline{e}^{(\kappa)})\}_{j\in[m]}, \{(\mathsf{w}_{\mathsf{pc},j}^{(\kappa)}, e^{(\kappa)})\}_{j\in[m]}) \in \mathsf{NSC}_{\mathsf{PC}}^m \tag{33}$$

$$((\overline{e}^{(\kappa)}, \tau^{(\kappa)}), e^{(\kappa)}) \in \mathsf{ZC}_{\mathsf{PC}} \tag{34}$$

denote the corresponding output instance-witness pairs.

By the verifier's construction (Step 3) we have that $\mathsf{u}_i^{(\kappa)}$ is the same as the input instance u_i for all $\kappa \in [2^\ell]$. Then, by the binding property of the commitment scheme, we have that $\mathsf{w}_i^{(\kappa)} = \mathsf{w}_i^{(\kappa')}$ for all $\kappa, \kappa' \in [2^\ell]$. Thus, we denote this value as w_i. Now, for each $i \in [k]$, we parse $(\vec{u}_i, \mathsf{x}_i) \leftarrow \mathsf{u}_i$, and $\vec{w}_i \leftarrow \mathsf{w}_i$. By Equation (32), for each $\kappa \in [2^\ell]$, given

$$\vec{g}_i \leftarrow G(\vec{w}_i, \mathsf{x}_i),$$

we have that

$$0 = \sum_{x\in\{0,1\}^\ell} \widetilde{e}_1^{(\kappa)}(x_1) \cdot \widetilde{e}_2^{(\kappa)}(x_2) \cdot F(\vec{g}_i(x)),$$

where x_1 and x_2 (likewise $e_1^{(\kappa)}$ and $e_2^{(\kappa)}$) represent the first and the second half of the original vector. But by Equation (34), we have that

$$e_1 = ((\tau^{(\kappa)})^0, (\tau^{(\kappa)})^1, (\tau^{(\kappa)})^2, \ldots, (\tau^{(\kappa)})^{\sqrt{m}-1})$$
$$e_2 = ((\tau^{(\kappa)})^0, (\tau^{(\kappa)})^{\sqrt{m}}, (\tau^{(\kappa)})^{2\sqrt{m}}, \ldots, (\tau^{(\kappa)})^{(\sqrt{m}-1)\cdot\sqrt{m}}).$$

Substituting, we have that

$$0 = \sum_{x\in\{0,1\}^\ell} (\tau^{(\kappa)})^{X_1+\sqrt{m}\cdot X_2} \cdot F(\vec{g}_i(x)),$$

where $X_i = \sum_{j\in[\ell/2]} x_{i,j} \cdot 2^{\ell/2-j}$ (i.e. the corresponding decimal representation if x_i is treated as the bit representation). Then, by interpolation, we have that the Lagrange polynomial

$$Q(Y) = \sum_{x\in\{0,1\}^\ell} Y^{X_1+\sqrt{m}\cdot X_2} \cdot F(\vec{g}_i(x))$$

is the zero polynomial. This means that

$$0 = F(\vec{g}_i(x))$$

for all $x \in \{0,1\}^\ell$. Hence, we have that

$$(\vec{\mathsf{u}}, \vec{\mathsf{w}}) \in \mathsf{ZC}^k.$$

Then, χ can produce $\vec{\mathsf{w}}_i$ for all $i \in [k]$ by reading the output witness from any one of the transcripts. By an identical line of reasoning χ can produce $\vec{\mathsf{w}}_{\mathsf{pc}}$ such that

$$(\vec{\mathsf{u}}_{\mathsf{pc}}, \vec{\mathsf{w}}_{\mathsf{pc}}) \in \mathsf{ZC}_{\mathsf{PC}}^m.$$

Thus, we have that tree-extractability holds. □

B.3 Proof of Lemma 6 (Folding nested sum-check)

Lemma 22 (Completeness). *Construction 4 is complete.*

Proof. Consider an arbitrary generator and encoder algorithm $(\mathcal{G}, \mathcal{K})$, and an arbitrary PPT adversary $\mathcal{A}$. For an arbitrary size parameter N, let $\mathsf{pp} \leftarrow \mathcal{G}(\lambda, N)$. Suppose now that the adversary $\mathcal{A}$ on input pp generates a structure (F, G), and input instance-witness pairs

$$\{((T_i, \mathsf{u}_i, \mathsf{x}_i), (T_{\mathsf{pc},i}, \mathsf{u}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}))\}_{i\in[k]}, \{(\mathsf{w}_i, \mathsf{w}_{\mathsf{pc},i})\}_{i\in[k]} \in \mathsf{NSC}^k \times \mathsf{NSC}^k_{\mathsf{PC}}. \quad (35)$$

Suppose for $(\mathsf{pk}, \mathsf{vk}) \leftarrow \mathcal{K}(\mathsf{pp}, (F, G))$, the prover and the verifier on input $(\mathsf{pk}, \mathsf{vk})$ and this instance-witness pair interactively produce the output instance-witness pair

$$(((T, \mathsf{u}, \mathsf{x}), (T_{\mathsf{pc}}, \mathsf{u}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}})), (\mathsf{w}, \mathsf{w}_{\mathsf{pc}})).$$

We must show that

$$((T, \mathsf{u}, \mathsf{x}), \mathsf{w}) \in \mathsf{NSC}$$
$$((T_{\mathsf{pc}}, \mathsf{u}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}}), \mathsf{w}_{\mathsf{pc}}) \in \mathsf{NSC}_{\mathsf{PC}}.$$

Indeed, for each $i \in [k]$, we parse $(\vec{w}_i, e_i) \leftarrow \mathsf{w}_i$ and $(\vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}) \leftarrow \mathsf{w}_{\mathsf{pc},i}$. Then, for $\vec{g}_i \leftarrow G(\vec{w}_i, \mathsf{x}_i)$ and $\vec{g}_{\mathsf{pc},i} \leftarrow G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i})$, by Precondition 35, we have that

$$\begin{aligned}
&T_i + \gamma \cdot T_{\mathsf{pc},i} \\
&= \sum_{x\in\{0,1\}^\ell} \widetilde{e}_{i,1}(x_1) \cdot \widetilde{e}_{i,2}(x_2) \cdot F(\vec{g}_i(x)) + \gamma \cdot \widetilde{e}_{\mathsf{pc},i,1}(x_1) \cdot \widetilde{e}_{\mathsf{pc},i,2}(x_2) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc},i}(x)) \\
&= \sum_{x\in\{0,1\}^\ell} h_{1,i}(x) \cdot h_{2,i}(x) \cdot F(\vec{g}_i(x)) + \gamma \cdot h_{\mathsf{pc},1,i}(x) \cdot h_{\mathsf{pc},2,i}(x) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc},i}(x)) \\
&= \sum_{x\in\{0,1\}^\ell} F'(\vec{g}_i(x), h_{1,i}(x), h_{2,i}(x), \vec{g}_{\mathsf{pc,i}}, h_{\mathsf{pc},1,i}(x), h_{\mathsf{pc},2,i}(x)) \\
&= \sum_{x\in\{0,1\}^\ell} F'(G'(\vec{w}_i, e_i, \vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}, (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i}))(x)),
\end{aligned}$$

where $h_{1,i}$, $h_{2,i}$, $h_{\mathsf{pc},1,i}$, and $h_{\mathsf{pc},2,i}$ are defined as in the construction. This implies that the completeness precondition holds for the sum-check folding reduction. That is, for $i \in [k]$

$$(\mathsf{pp}, (F', G'), (T_i + \gamma \cdot T_{\mathsf{pc},i}, (\mathsf{u}_i, \mathsf{u}_{\mathsf{pc},i}), (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i})), (\mathsf{w}_i, \mathsf{w}_{\mathsf{pc},i})) \in \mathsf{SC}.$$

Then, by the completeness of the sum-check folding reduction, we have that the instance-witness pair output from the folding sum-check reduction is satisfying. That is

$$(\mathsf{pp}, (F', G'), (T_\gamma, (\mathsf{u}, \mathsf{u}_{\mathsf{pc}}), (\mathsf{x}, \mathsf{x}_{\mathsf{pc}})), (\mathsf{w}, \mathsf{w}_{\mathsf{pc}})) \in \mathsf{SC}. \quad (36)$$

This implies that given $(\vec{w}, e) \leftarrow \mathsf{w}$, $(\vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}) \leftarrow \mathsf{w}_{\mathsf{pc}}$, $\vec{g} \leftarrow G(\vec{w}, e)$, and $\vec{g}_{\mathsf{pc}} \leftarrow G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}})$ we have that

$$\begin{aligned}
T_\gamma &= \sum_{x \in \{0,1\}^\ell} F'(G'(\vec{w}, e, \vec{w}_{\mathsf{pc}}, e_{\mathsf{pc}}, (\mathsf{x}, \mathsf{x}_{\mathsf{pc}}))(x)) \\
&= \sum_{x \in \{0,1\}^\ell} h_1(x) \cdot h_2(x) \cdot F(\vec{g}(x)) + \gamma \cdot h_{\mathsf{pc},1}(x) \cdot h_{\mathsf{pc},2}(x) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}}(x)) \\
&= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(\vec{g}(x)) + \gamma \cdot \widetilde{e}_{\mathsf{pc},1}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2}(x) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}}(x)) \\
&= T + \gamma \cdot T_{\mathsf{pc}}.
\end{aligned}$$

Therefore, we have that the verifier's check in Step 5 passes.

Moreover, by the prover's computation, we have that

$$\begin{aligned}
T &= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(\vec{g}(x)) \\
T_{\mathsf{pc}} &= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{\mathsf{pc},1}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2}(x) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}}(x))
\end{aligned}$$

Then, by the validity of the commitments implied by Equation 36, we have that

$$\begin{aligned}
((T, (\mathsf{u}, \mathsf{x})), \mathsf{w}) &\in \mathsf{NSC} \\
((T_{\mathsf{pc}}, (\mathsf{u}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}})), \mathsf{w}_{\mathsf{pc}}) &\in \mathsf{NSC}_{\mathsf{PC}}.
\end{aligned}$$

Therefore, completeness holds. □

Lemma 23 (Knowledge Soundness). *Construction 4 is knowledge sound.*

Proof. We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct a PPT extractor χ that outputs a satisfying input witness with probability $1 - \mathsf{negl}(\lambda)$ given a tree of accepting transcripts and the corresponding output instance-witness pairs.

Indeed, let χ_{FSC} denote the tree-extractor for the sum-check folding reduction, which reduces from SC^k to SC in Step 3. Suppose that χ_{FSC} can extract using an L-tree of accepting transcripts where $L \in \mathbb{N}^{\nu+1}$. Then, we provide χ with a $(2, L)$-tree of accepting transcripts for public parameter, structure, instance tuple

$$(\mathsf{pp}, (F, G), \{((T_i, \mathsf{u}_i, \mathsf{x}_i), (T_{\mathsf{pc},i}, \mathsf{u}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}))\}_{i \in [k]}).$$

For $\kappa \in [2]$ and $l \in ([L_1], \ldots, [L_{\nu+1}])$, let $\gamma^{(\kappa)}$ denote the verifier's first challenge, and for each such challenge let $r^{(\kappa,l)}$ denote the verifier's challenges during the sum-check folding reduction. Let

$$\begin{aligned}
((T^{(\kappa,l)}, \mathsf{u}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)}), \mathsf{w}^{(\kappa,l)}) &\in \mathsf{NSC} \\
((T_{\mathsf{pc}}^{(\kappa,l)}, \mathsf{u}_{\mathsf{pc}}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)}), \mathsf{w}_{\mathsf{pc}}^{(\kappa,l)}) &\in \mathsf{NSC}_{\mathsf{PC}}
\end{aligned}$$

denote the corresponding output instance-witness pairs.

Now, by the satisfiability condition of NSC and $\mathsf{NSC}_{\mathsf{PC}}$ for

$$\begin{aligned}(\vec{u}^{(\kappa,l)}, \overline{e}^{(\kappa,l)}) &\leftarrow \mathsf{u}^{(\kappa,l)}\\ (\vec{u}^{(\kappa,l)}_{\mathsf{pc}}, \overline{e}^{(\kappa,l)}_{\mathsf{pc}}) &\leftarrow \mathsf{u}^{(\kappa,l)}_{\mathsf{pc}}\\ (\vec{w}^{(\kappa,l)}, e^{(\kappa,l)}) &\leftarrow \mathsf{w}^{(\kappa,l)}\\ (\vec{w}^{(\kappa,l)}_{\mathsf{pc}}, e^{(\kappa,l)}_{\mathsf{pc}}) &\leftarrow \mathsf{w}^{(\kappa,l)}_{\mathsf{pc}},\end{aligned}$$

we have that

$$\begin{aligned}T^{(\kappa,l)} &= \sum_{x\in\{0,1\}^{\ell}} \widetilde{e}_1^{(\kappa,l)}(x_1)\cdot \widetilde{e}_2^{(\kappa,l)}(x_2)\cdot F(G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)})(x))\\ T^{(\kappa,l)}_{\mathsf{pc}} &= \sum_{x\in\{0,1\}^{\ell}} \widetilde{e}_{\mathsf{pc},1}^{(\kappa,l)}(x_1)\cdot \widetilde{e}_{\mathsf{pc},2}^{(\kappa,l)}(x_2)\cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}^{(\kappa,l)}_{\mathsf{pc}}, \mathsf{x}^{(\kappa,l)}_{\mathsf{pc}})(x))\end{aligned}$$

and that

$$\overline{e}^{(\kappa,l)} = \mathsf{Commit}(\mathsf{pp}, e^{(\kappa,l)}) \tag{37}$$

$$\overline{e}^{(\kappa,l)}_{\mathsf{pc}} = \mathsf{Commit}(\mathsf{pp}, e^{(\kappa,l)}_{\mathsf{pc}}) \tag{38}$$

$$u_j^{(\kappa,l)} = \mathsf{Commit}(\mathsf{pp}, w_j^{(\kappa,l)}) \tag{39}$$

$$u_{\mathsf{pc},j}^{(\kappa,l)} = \mathsf{Commit}(\mathsf{pp}, w_{\mathsf{pc},j}^{(\kappa,l)}) \tag{40}$$

for $j \in [s]$.

Then, by the verifier's check in Step 5, we have that

$$
\begin{aligned}
&T_{\gamma}^{(\kappa,l)} \\
&= T^{(\kappa,l)} + \gamma^{(\kappa,l)} \cdot T_{\mathsf{pc}}^{(\kappa,l)} \\
&= \sum_{x \in \{0,1\}^{\ell}} \widetilde{e}_1^{(\kappa,l)}(x_1) \cdot \widetilde{e}_2^{(\kappa,l)}(x_2) \cdot F(G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)})(x)) + \\
&\quad \gamma^{(\kappa,l)} \cdot \sum_{x \in \{0,1\}^{\ell}} \widetilde{e}_{\mathsf{pc},1}^{(\kappa,l)}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2}^{(\kappa,l)}(x_2) \cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)})(x)) \\
&= \sum_{x \in \{0,1\}^{\ell}} h_1^{(\kappa,l)}(x) \cdot h_2^{(\kappa,l)}(x) \cdot F(G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)})(x)) + \\
&\quad \gamma^{(\kappa,l)} \cdot \sum_{x \in \{0,1\}^{\ell}} h_{\mathsf{pc},1}^{(\kappa,l)}(x) \cdot h_{\mathsf{pc},2}^{(\kappa,l)}(x) \cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)})(x)) \\
&= \sum_{x \in \{0,1\}^{\ell}} F'((G(\vec{w}^{(\kappa,l)}, \mathsf{x}^{(\kappa,l)}), h_1^{(\kappa,l)}, h_2^{(\kappa,l)}, G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc}}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)}), h_{\mathsf{pc},1}^{(\kappa,l)}, h_{\mathsf{pc},2}^{(\kappa,l)})(x)) \\
&= \sum_{x \in \{0,1\}^{\ell}} F'(G'(\vec{w}^{(\kappa,l)}, e^{(\kappa,l)}, \vec{w}_{\mathsf{pc}}^{(\kappa,l)}, e_{\mathsf{pc}}^{(\kappa,l)}, (\mathsf{x}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)}))(x)) \\
&= \sum_{x \in \{0,1\}^{\ell}} F'(G'((\mathsf{w}^{(\kappa,l)}, \mathsf{w}_{\mathsf{pc}}^{(\kappa,l)}), (\mathsf{x}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)}))(x))
\end{aligned}
\tag{41}
$$

Therefore, by Derivation (41), and Equations (37), (38), (39), and (40) we have that

$$(\mathsf{pp}, (F', G'), (T_{\gamma}^{(\kappa,l)}, (\mathsf{u}^{(\kappa,l)}, \mathsf{u}_{\mathsf{pc}}^{(\kappa,l)}), (\mathsf{x}^{(\kappa,l)}, \mathsf{x}_{\mathsf{pc}}^{(\kappa,l)})), (\mathsf{w}^{(\kappa,l)}, \mathsf{w}_{\mathsf{pc}}^{(\kappa,l)})) \in \mathsf{SC}.$$

This means that for each challenge $\gamma^{(\kappa)}$, and the corresponding input instance to the sum-check folding reduction

$$(\mathsf{pp}, (F', G'), \{(T_i^{(\kappa)} + \gamma^{(\kappa)} \cdot T_{\mathsf{pc},i}^{(\kappa)}, (\mathsf{u}_i^{(\kappa)}, \mathsf{u}_{\mathsf{pc},i}^{(\kappa)}), (\mathsf{x}_i^{(\kappa)}, \mathsf{x}_{\mathsf{pc},i}^{(\kappa)}))\}_{i \in [k]}),$$

χ can produce a satisfying L-sub-tree of transcripts for the sum-check folding reduction with satisfying output instance-witness pairs in SC. Then, χ can run χ_{FSC} on each of these sub-trees to retrieve a corresponding satisfying input witness $\{(\mathsf{w}_i^{(\kappa)}, \mathsf{w}_{\mathsf{pc},i}^{(\kappa)})\}_{i \in [k]}$ such that

$$(\{(T_i^{(\kappa)} + \gamma^{(\kappa)} \cdot T_{\mathsf{pc},i}^{(\kappa)}, (\mathsf{u}_i^{(\kappa)}, \mathsf{u}_{\mathsf{pc},i}^{(\kappa)}), (\mathsf{x}_i^{(\kappa)}, \mathsf{x}_{\mathsf{pc},i}^{(\kappa)}))\}_{i \in [k]}, \{(\mathsf{w}_i^{(\kappa)}, \mathsf{w}_{\mathsf{pc},i}^{(\kappa)})\}_{i \in [k]}) \in \mathsf{SC}^k$$

with respect to structure (F', G').

By the verifier's construction, we have that for all $i \in [k]$, $T_i^{(\kappa)} = T_i$, $T_{\mathsf{pc},i}^{(\kappa)} = T_{\mathsf{pc},i}$, $\mathsf{x}_i^{(\kappa)} = \mathsf{x}_i$, $\mathsf{x}_{\mathsf{pc},i}^{(\kappa)} = \mathsf{x}_{\mathsf{pc},i}$, $\mathsf{u}_i^{(\kappa)} = \mathsf{u}_i$, and $\mathsf{u}_{\mathsf{pc},i}^{(\kappa)} = \mathsf{u}_{\mathsf{pc},i}$. Then, by the binding property of the commitment scheme, for all $i \in [k]$ we must have that for any $\kappa, \kappa' \in [2]$ $\mathsf{w}_i^{(\kappa)} = \mathsf{w}_i^{(\kappa')}$ and $\mathsf{w}_{\mathsf{pc},i}^{(\kappa)} = \mathsf{w}_{\mathsf{pc},i}^{(\kappa')}$. We denote these values simply as w_i and $\mathsf{w}_{\mathsf{pc},i}$.

Then, for

$$(\vec{w}_i, e_i) \leftarrow \mathsf{w}_i$$
$$(\vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}) \leftarrow \mathsf{w}_{\mathsf{pc},i},$$

by the satisfiability condition of SC we have that

$$\begin{aligned}
&T_i + \gamma^{(\kappa)} \cdot T_{\mathsf{pc},i} \\
&= \sum_{x \in \{0,1\}^\ell} F'(G'((\mathsf{w}_i, \mathsf{w}_{\mathsf{pc},i}), (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i}))(x)) \\
&= \sum_{x \in \{0,1\}^\ell} F'(G'((\vec{w}_i, e_i, \vec{w}_{\mathsf{pc},i}, e_{\mathsf{pc},i}), (\mathsf{x}_i, \mathsf{x}_{\mathsf{pc},i}))(x)) \\
&= \sum_{x \in \{0,1\}^\ell} F'((G(\vec{w}_i, \mathsf{x}_i), h_{1,i}, h_{2,i}, G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}), h_{\mathsf{pc},1,i}, h_{\mathsf{pc},2,i})(x)) \\
&= \sum_{x \in \{0,1\}^\ell} h_{1,i}(x) \cdot h_{2,i}(x) \cdot F(G(\vec{w}_i, \mathsf{x}_i)(x)) + \\
&\qquad \gamma^{(\kappa)} \cdot \sum_{x \in \{0,1\}^\ell} h_{\mathsf{pc},1,i}(x) \cdot h_{\mathsf{pc},2,i}(x) \cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i})(x)) \\
&= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{1,i}(x_1) \cdot \widetilde{e}_{2,i}(x_2) \cdot F(G(\vec{w}_i, \mathsf{x}_i)(x)) + \\
&\qquad \gamma^{(\kappa)} \cdot \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{\mathsf{pc},1,i}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2,i}(x_2) \cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i})(x)).
\end{aligned}$$

Then, by interpolation, we have that

$$\begin{aligned}
T_i &= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{1,i}(x_1) \cdot \widetilde{e}_{2,i}(x_2) \cdot F(G(\vec{w}_i, \mathsf{x}_i)(x)) \\
T_{\mathsf{pc},i} &= \sum_{x \in \{0,1\}^\ell} \widetilde{e}_{\mathsf{pc},1,i}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2,i}(x_2) \cdot F_{\mathsf{PC}}(G_{\mathsf{PC}}(\vec{w}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i})(x)).
\end{aligned}$$

This means that for $i \in [k]$

$$((T_i, \mathsf{u}_i, \mathsf{x}_i), \mathsf{w}_i) \in \mathsf{NSC}$$
$$((T_{\mathsf{pc},i}, \mathsf{u}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}), \mathsf{w}_{\mathsf{pc},i}) \in \mathsf{NSC}_{\mathsf{PC}}.$$

Therefore, χ can compute a satisfying input witness $(\mathsf{w}_i, \mathsf{w}_{\mathsf{pc},i})$ for $i \in [k]$. □

Construction 4 invokes $\mathsf{SumFold}$ once in Step 3. Its remaining prover computations are lower order, so the prover-time complexity remains $O(ktd \log^2 d \cdot 2^\ell)$ field operations.

B.4 Proof of Lemma 11 (From lookup to grand-product)

Lemma 24 (Completeness). *Construction 6 is complete.*

Proof. Consider an arbitrary PPT adversary $\mathcal{A}$. For arbitrary size parameters (n, m), let $\mathsf{pp} \leftarrow \mathcal{G}(\lambda, (n, m))$, and $\mathsf{pk}, \mathsf{vk} \leftarrow \mathcal{K}(\mathsf{pp})$. Suppose now that the adversary on input pp generates

$$((\overline{t}, \overline{a}, \overline{v}), (t, a, v)) \in \mathsf{LKP}.$$

The prover and verifier on input $(\mathsf{pk}, \mathsf{vk}), (\overline{t}, \overline{a}, \overline{v}), (t, a, v)$ interactively produce output instance-witness pairs

$$\{((p_k, \overline{v}_k), v_k)\}_{k \in [4]}$$

We must show that $\{((p_k, \overline{v}_k), v_k)\}_{k \in [4]} \in \mathsf{GP}^4$.

Indeed, by the linearity of the commitment scheme and by definitions of $G_n = \mathsf{Commit}(\mathsf{pp}, 1^n)$, $G_m = \mathsf{Commit}(\mathsf{pp}, 1^m)$, and $H = \mathsf{Commit}(\mathsf{pp}, (1, \ldots, n))$, we have that

$$\overline{v}_k = \mathsf{Commit}(\mathsf{pp}, v_k) \tag{42}$$

for $k \in [4]$.

Next, by the prover's computation of f and c and by Lemma 10, we have that

$$\left(\prod_{i \in [n]} (i + t_i \cdot X - Y)\right) \cdot \left(\prod_{i \in [m]} (a_i + v_i \cdot X + (c_i + 1) \cdot X^2 - Y)\right) =$$
$$\left(\prod_{i \in [m]} (a_i + v_i \cdot X + c_i \cdot X^2 - Y)\right) \cdot \left(\prod_{i \in [n]} (i + t_i \cdot X + f_i \cdot X^2 - Y)\right).$$

But, this means that for an arbitrary $r_X, r_Y \in \mathbb{F}$

$$\left(\prod_{i \in [n]} (i + t_i \cdot r_X - r_Y)\right) \cdot \left(\prod_{i \in [m]} (a_i + v_i \cdot r_X + (c_i + 1) \cdot r_X^2 - Y)\right) =$$
$$\left(\prod_{i \in [m]} (a_i + v_i \cdot r_X + c_i \cdot r_X^2 - r_Y)\right) \cdot \left(\prod_{i \in [n]} (i + t_i \cdot r_X + f_i \cdot r_X^2 - r_Y)\right).$$

Then, by the definition of $v_1, \ldots, v_4$ we have that

$$\prod_{i \in [n]} v_{1,i} \cdot \prod_{i \in [m]} v_{2,i} = \prod_{i \in [m]} v_{3,i} \cdot \prod_{i \in [n]} v_{4,i}.$$

Hence, by construction, we have that

$$p_k = \prod_i v_{k,i} \tag{43}$$

for $k \in [4]$, by substitution, we have that

$$p_1 \cdot p_2 = p_3 \cdot p_4.$$

Therefore, we have that the verifier's check in Step 5 passes.

Then, by Equations 42 and 43, we have that $\{((p_k, \overline{v}_k), v_k)\}_{k\in[4]} \in \mathsf{GP}^4$. □

Lemma 25 (Knowledge Soundness). *Construction 6 is knowledge sound.*

Proof. We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct a PPT extractor χ that outputs a satisfying input witness with probability $1-\mathsf{negl}(\lambda)$ given a tree of accepting transcripts and the corresponding output instance-witness pairs.

Indeed, for a sufficiently large $N = O(nm)$ we provide χ with an N-tree of accepting transcripts for the public parameter, instance tuple $(\mathsf{pp}, (\overline{t}, \overline{a}, \overline{v}))$. For $l \in [N]$, let $r_X^{(l)}$ and $r_Y^{(l)}$ denote the verifier's challenges in Step 2. Let

$$\{((p_k^{(l)}, \overline{v}_k^{(l)}), v_k^{(l)})\}_{k\in[4]} \in \mathsf{GP}^4 \tag{44}$$

denote the corresponding output instance-witness pairs.

Using an arbitrary but sufficiently large subsets of transcripts, the extractor χ interpolates t such that for $i \in [n]$

$$v_{1,i}^{(l)} = i + t_i \cdot r_X^{(l)} - r_Y^{(l)}, \tag{45}$$

interpolates a, v, and c such that for $i \in [m]$

$$v_{2,i}^{(l)} = a_i + v_i \cdot r_X^{(l)} + (c_i + 1) \cdot {r_X^{(l)}}^2 - r_Y^{(l)}, \tag{46}$$

interpolates a', v', and c' such that for $i \in [m]$

$$v_{3,i}^{(l)} = a'_i + v'_i \cdot r_X^{(l)} + c'_i \cdot {r_X^{(l)}}^2 - r_Y^{(l)} \tag{47}$$

interpolates t' and f such that for $i \in [n]$

$$v_{4,i}^{(l)} = i + t'_i \cdot r_X^{(l)} + f_i \cdot {r_X^{(l)}}^2 - r_Y^{(l)}. \tag{48}$$

We will now argue that regardless of the extractors choice of transcripts, due to the binding property of the commitment scheme, it will derive the same interpolated vectors t, a, v, c, and f. Indeed, starting with t, regardless of the extractors choice of $l \in [N]$ to derive t, we must have that

$$\begin{aligned}
\mathsf{Commit}(\mathsf{pp}, t) &= \mathsf{Commit}(\mathsf{pp}, (v_1^{(l)} - (1, \ldots, n) + (r_Y^{(l)}, \ldots, r_Y^{(l)})) \cdot {r_X^{(l)}}^{-1}) \\
&= (\overline{v}_1^{(l)} - H + G_n \cdot r_Y^{(l)}) \cdot {r_X^{(l)}}^{-1} \\
&= \overline{t}
\end{aligned}$$

Therefore, because $\overline{t}$ is identical across all transcripts, by the binding property of the commitment scheme, we have that t is identically derived regardless of the choice of transcripts. For similar reasons, because $\overline{a}$, $\overline{v}$, $\overline{c}$, and $\overline{f}$ are identical across transcripts we must have that a, v, c, and f are identically derived regardless of the choice of transcripts, and moreover that $t = t'$, $a = a'$, $v = v'$ and $c = c'$. Then, we must have that Equations (45), (46) (47) (48) hold for all $l \in [N]$.

Then, by Precondition (44), and the verifier's check in Step (5), we have for all $l \in [N]$

$$\left(\prod_{i\in[n]} (i + t_i \cdot r_X^{(l)} - r_Y^{(l)})\right) \cdot \left(\prod_{i\in[m]} (a_i + v_i \cdot r_X^{(l)} + (c_i + 1) \cdot {r_X^{(l)}}^2 - r_Y^{(l)})\right)$$
$$= \prod_{i\in[n]} v_{1,i}^{(l)} \cdot \prod_{i\in[m]} v_{2,i}^{(l)}$$
$$= p_1^{(l)} \cdot p_2^{(l)}$$
$$= p_3^{(l)} \cdot p_4^{(l)}$$
$$= \prod_{i\in[n]} v_{3,i}^{(l)} \cdot \prod_{i\in[m]} v_{4,i}^{(l)} =$$
$$\left(\prod_{i\in[m]} (a_i + v_i \cdot r_X^{(l)} + c_i \cdot {r_X^{(l)}}^2 - r_Y^{(l)})\right) \cdot \left(\prod_{i\in[n]} (i + t_i \cdot r_X^{(l)} + f_i \cdot {r_X^{(l)}}^2 - r_Y^{(l)})\right).$$

Therefore, by the precondition that N is sufficiently large, by interpolation, we have that

$$\left(\prod_{i\in[n]} (i + t_i \cdot X - Y)\right) \cdot \left(\prod_{i\in[m]} (a_i + v_i \cdot X + (c_i + 1) \cdot X^2 - Y)\right) =$$
$$\left(\prod_{i\in[m]} (a_i + v_i \cdot X + c_i \cdot X^2 - Y)\right) \cdot \left(\prod_{i\in[n]} (i + t_i \cdot X + f_i \cdot X^2 - Y)\right).$$

Then, by Lemma 10, and by the derivation that $\mathsf{Commit}(\mathsf{pp}, t) = \overline{t}$ (and the similar implied derivations for a and v), we have that

$$(\mathsf{pp}, (\overline{t}, \overline{a}, \overline{v}), (t, a, v)) \in \mathsf{LKP}.$$

□

B.5 Proof of Lemma 9 (From grand-product to zero-check)

Lemma 26 (Completeness). *Construction 5 is complete.*

Proof. Consider an arbitrary PPT adversary $\mathcal{A}$. For an arbitrary size parameter n, let $\mathsf{pp} \leftarrow \mathcal{G}(\lambda, n)$, and $(\mathsf{pk}, \mathsf{vk}, (F_{\mathsf{GP}}, G_{\mathsf{GP}})) \leftarrow \mathcal{K}(\mathsf{pp})$. Suppose now that the

adversary on input pp generates

$$((p, \overline{v}), v) \in \mathsf{GP}. \tag{49}$$

The prover and verifier on input $(\mathsf{pk}, \mathsf{vk}), (p, \overline{v}), v$ interactively produce output instance-witness pairs $(((\overline{v}, \overline{v}'), p), (v, v'))$. We must show that

$$((F_{\mathsf{GP}}, G_{\mathsf{GP}}), ((\overline{v}, \overline{v}'), p), (v, v')) \in \mathsf{ZC}. \tag{50}$$

Indeed, by Precondition 49 and the prover's computation in Step 1, we have that

$$\begin{aligned} \overline{v} &= \mathsf{Commit}(\mathsf{pp}, v) \\ \overline{v}' &= \mathsf{Commit}(\mathsf{pp}, v') \end{aligned}$$

By Lemma 8 we have that

$$\begin{aligned} f(0, x) &= \widetilde{v}(x) \\ f(1, \ldots, 1, 0) &= p, \end{aligned}$$

and that for all $x \in \{0,1\}^{\log n}$

$$f(1, x) = f(x, 0) \cdot f(x, 1).$$

Then, for all $x \in \{0,1\}^{\log n}$, we have that

$$\begin{aligned} 0 &= f(1, x) - f(x, 0) \cdot f(x, 1) \\ &= \widetilde{(v, v')}(1, x) - \widetilde{(v, v')}(x, 0) \cdot \widetilde{(v, v')}(x, 1) \\ &= (v, \widetilde{(v'_1, \ldots, p, v'_n)})(1, x) - (v, \widetilde{(v'_1, \ldots, p, v'_n)})(x, 0) \cdot (v, \widetilde{(v'_1, \ldots, p, v'_n)})(x, 1) \\ &= F_{\mathsf{GP}}(G_{\mathsf{GP}}((v, v'), p)(x)) \end{aligned}$$

Therefore, we have that Equation 50 holds. □

Lemma 27 (Knowledge soundness). *Construction 5 is knowledge-sound*

Proof. We prove knowledge soundness via tree extraction (Lemma 17). That is, we construct a PPT extractor χ, that can produce a satisfying input witness given as input a single transcript for public parameters pp and input instance $(p, \overline{v})$, and output instance-witness pair

$$((F_{\mathsf{GP}}, G_{\mathsf{GP}}), ((\overline{v}, \overline{v}'), p), (v, v')) \in \mathsf{ZC}. \tag{51}$$

Indeed, by Precondition (51), we have that for all $x \in \{0,1\}^{\log n}$

$$\begin{aligned} 0 &= F_{\mathsf{GP}}(G_{\mathsf{GP}}((v, v'), p)(x)) \\ &= (v, \widetilde{(v'_1, \ldots, p, v'_n)})(1, x) - (v, \widetilde{(v'_1, \ldots, p, v'_n)})(x, 0) \cdot (v, \widetilde{(v'_1, \ldots, p, v'_n)})(x, 1) \\ &= f(1, x) - f(x, 0) \cdot f(x, 1) \end{aligned}$$

for $f = (v, (\widetilde{v'_1, \ldots, p}, v'_n))$. But this means that $f(1, x) = f(x, 0) \cdot f(x, 1)$ for all $x \in \{0,1\}^{\log n}$, $f(1, \ldots, 1, 0) = p$, and $f(0, x) = \widetilde{v}(x)$. Then, by Lemma 8, we must have that $p = \prod_{i \in [n]} v_i$. Moreover, by Precondition (51), we have that $\mathsf{Commit}(\mathsf{pp}, v) = \overline{v}$. Therefore, we have that

$$((p, \overline{v}), v) \in \mathsf{GP}.$$

□

C Relationship with Protostar's verifier-check compression [20, §3.5]

The work of Bünz and Chen [20] also observes that the power-check relation can be enforced with $2 \cdot \sqrt{m}$ quadratic constraints and that they can be folded alongside the main instance. However, their specific approach requires specialized techniques with additional overheads to fold low-degree power-check constraints alongside the original high-degree constraints [20, Appendix A].

In contrast, our approach of representing these quadratic constraints as a zero-check instance (which is subsequently reduced to a sum-check instance) leads to a simpler and more efficient scheme. In particular, using SumFold to uniformly fold the power-check alongside the original constraints avoids having to commit to any cross-terms related to the power-check instance. Alternatively, ProtoStar uses Nova-style folding for low-degree constraints requiring commitments to cross terms. Moreover, the new power-check instance generated in every folding step can be folded alongside fresh power-check instances in subsequent folding steps. This is precisely enabled by SumFold's ability to fold multiple instances at once. In contrast, as discussed earlier (§1.1), Protostar does not support folding multiple instances at once.

D Optimizing the round complexity of Theorem 3

The zero-check folding scheme can be optimized down to $2 + \log k$ rounds of communication: First, as the commitment $\overline{e}$ to the powers of τ in Π_{ZCR} is checked independently of the interaction at a later point, the prover is free to send it as part of its first message in Π_{FNSC}. This allows the verifier to send the randomness τ in Π_{ZCR}, the randomness γ in Π_{NSC}, and the randomness ρ in the first round of SumFold all in one message, removing a round of communication. Second, in Π_{NSC}, instead of sending T_γ at the end of the underlying sum-check protocol and then again T and T_{pc}, the prover can directly send T and T_{pc} (in place of T_γ) in the final round of the underlying sum-check protocol. This, once again, removes a round of communication. In the case of folding a single zero-check instance with a single running instance (e.g., $k = 1$), this brings the overall number of rounds to 2.

E ZeroFold unrolled

In this section, we present the unrolled ZeroFold protocol.

Construction 8 (ZeroFold). We construct a reduction of knowledge of type

$$(\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}})^k \times \mathsf{ZC}^k \to (\mathsf{NSC} \times \mathsf{NSC}_{\mathsf{PC}} \times \mathsf{ZC}_{\mathsf{PC}}).$$

Let $\langle \mathcal{P}_{\mathsf{sc}}, \mathcal{V}_{\mathsf{sc}} \rangle$ be the sumcheck protocol (Lemma 4). Consider size bounds $\ell, d, s, t \in \mathbb{N}$ and let $\nu = \log k$. We index $i \in [k]$ and $i \in \{0,1\}^\nu$ interchangeably. Consider an arbitrary generator algorithm and underlying additively homomorphic commitment scheme. We define the encoder, prover, and verifier as follows.

$\underline{\mathcal{K}(\mathsf{pp}, (F, G))}$: Output $(\mathsf{pk}, \mathsf{vk}) \leftarrow ((F, G), \bot)$.

$\underline{\langle \mathcal{P}, \mathcal{V} \rangle((\mathsf{pk}, \mathsf{vk}), ((\mathsf{n}\vec{\mathsf{u}}, \mathsf{n}\vec{\mathsf{u}}_{\mathsf{pc}}, \vec{\mathsf{u}}_{\mathsf{pc}}), \vec{\mathsf{u}}), ((\mathsf{n}\vec{\mathsf{w}}, \mathsf{n}\vec{\mathsf{w}}_{\mathsf{pc}}, \vec{\mathsf{w}}_{\mathsf{pc}}), \vec{\mathsf{w}}))}$:

1. $\mathcal{P}, \mathcal{V}$: Parse for each $i \in [k]$,

$$(T_i, (\mathbf{u}_i^{\mathsf{nsc}}, \overline{e}_i), \mathsf{x}_i^{\mathsf{nsc}}) \leftarrow \mathsf{nu}_i \qquad (T_{\mathsf{pc},i}, (\mathbf{u}_{\mathsf{pc},i}^{\mathsf{nsc}}, \overline{e}_{\mathsf{pc},i}), \mathsf{x}_{\mathsf{pc},i}^{\mathsf{nsc}}) \leftarrow \mathsf{nu}_{\mathsf{pc},i}$$
$$(\mathbf{u}_i, \mathsf{x}_i) \leftarrow \mathsf{u}_i \qquad (\mathbf{u}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i}) \leftarrow \mathsf{u}_{\mathsf{pc},i}.$$

   The prover additionally parses for each $i \in [k]$,

$$(\mathbf{w}_i^{\mathsf{nsc}}, (e_{1,i}, e_{2,i})) \leftarrow \mathsf{nw}_i \qquad (\mathbf{w}_{\mathsf{pc},i}^{\mathsf{nsc}}, (e_{\mathsf{pc},1,i}, e_{\mathsf{pc},2,i})) \leftarrow \mathsf{nw}_{\mathsf{pc},i}$$
$$\mathbf{w}_i \leftarrow \mathsf{w}_i \qquad \mathbf{w}_{\mathsf{pc},i} \leftarrow \mathsf{w}_{\mathsf{pc},i}.$$

2. $\mathcal{V}$: Sample and send $\tau \xleftarrow{\$} \mathbb{F}$.
3. $\mathcal{P}$: Compute $((\overline{e}^\tau, \tau), e^\tau) \in \mathsf{ZC}_{\mathsf{PC}}$ with size parameter $m = 2^{\ell/2}$ and send $\overline{e}^\tau$.
4. $\mathcal{P}, \mathcal{V}$: Define

$$T \leftarrow (T_1, \ldots, T_k, 0, \ldots, 0) \qquad T_{\mathsf{pc}} \leftarrow (T_{\mathsf{pc},1}, \ldots, T_{\mathsf{pc},k}, 0, \ldots, 0)$$
$$\mathbf{u} \leftarrow (\mathbf{u}_1^{\mathsf{nsc}}, \ldots, \mathbf{u}_k^{\mathsf{nsc}}, \mathbf{u}_1, \ldots, \mathbf{u}_k) \qquad \mathbf{u}_{\mathsf{pc}} \leftarrow (\mathbf{u}_{\mathsf{pc},1}^{\mathsf{nsc}}, \ldots, \mathbf{u}_{\mathsf{pc},k}^{\mathsf{nsc}}, \mathbf{u}_{\mathsf{pc},1}, \ldots, \mathbf{u}_{\mathsf{pc},k})$$
$$\overline{e} \leftarrow (\overline{e}_1, \ldots, \overline{e}_k, \overline{e}^\tau, \ldots, \overline{e}^\tau) \qquad \overline{e}_{\mathsf{pc}} \leftarrow (\overline{e}_{\mathsf{pc},1}, \ldots, \overline{e}_{\mathsf{pc},k}, \overline{e}^\tau, \ldots, \overline{e}^\tau)$$
$$\mathsf{x} \leftarrow (\mathsf{x}_1^{\mathsf{nsc}}, \ldots, \mathsf{x}_k^{\mathsf{nsc}}, \mathsf{x}_1, \ldots, \mathsf{x}_k) \qquad \mathsf{x}_{\mathsf{pc}} \leftarrow (\mathsf{x}_{\mathsf{pc},1}^{\mathsf{nsc}}, \ldots, \mathsf{x}_{\mathsf{pc},k}^{\mathsf{nsc}}, \mathsf{x}_{\mathsf{pc},1}, \ldots, \mathsf{x}_{\mathsf{pc},k}).$$

   For $c \in \{1, 2\}$, the prover additionally defines

$$\mathbf{w} \leftarrow (\mathbf{w}_1^{\mathsf{nsc}}, \ldots, \mathbf{w}_k^{\mathsf{nsc}}, \mathbf{w}_1, \ldots, \mathbf{w}_k) \qquad \mathbf{w}_{\mathsf{pc}} \leftarrow (\mathbf{w}_{\mathsf{pc},1}^{\mathsf{nsc}}, \ldots, \mathbf{w}_{\mathsf{pc},k}^{\mathsf{nsc}}, \mathbf{w}_{\mathsf{pc},1}, \ldots, \mathbf{w}_{\mathsf{pc},k})$$
$$e_c \leftarrow (e_{c,1}, \ldots, e_{c,k}, e_c^\tau, \ldots, e_c^\tau) \qquad e_{\mathsf{pc},c} \leftarrow (e_{\mathsf{pc},c,1}, \ldots, e_{\mathsf{pc},c,k}, e_c^\tau, \ldots, e_c^\tau).$$

   where $(e_1^\tau, e_2^\tau) \leftarrow e^\tau$.

5. $\mathcal{V}$: Sample and send $\gamma \xleftarrow{\$} \mathbb{F}$ and $\rho \xleftarrow{\$} \mathbb{F}^{\nu+1}$.

6. $\mathcal{P}, \mathcal{V}$: Compute $(\sigma', r_b) \leftarrow \langle \mathcal{P}_{\mathsf{sc}}, \mathcal{V}_{\mathsf{sc}} \rangle((\mathsf{pk}, \mathsf{vk}), (\overline{Q}, \sigma), Q)$, where

$$\sigma \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(\rho, i) \cdot T_i + \gamma \cdot \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(\rho, i) \cdot T_{\mathsf{pc},i}$$

$$f_j(b, x) \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(b, i) \cdot g_{i,j}(x) \text{ where } \vec{g}_i \leftarrow G(\mathbf{w}_i, \mathsf{x}_i)$$

$$f_{\mathsf{pc},j}(b, x) \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(b, i) \cdot g_{\mathsf{pc},i,j}(x) \text{ where } \vec{g}_{\mathsf{pc},i} \leftarrow G_{\mathsf{PC}}(\mathbf{w}_{\mathsf{pc},i}, \mathsf{x}_{\mathsf{pc},i})$$

$$h_c(b, x_c) \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(b, i) \cdot \widetilde{e}_{c,i}(x_c) \quad \text{for } c \in \{1, 2\}$$

$$h_{\mathsf{pc},c}(b, x_c) \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(b, i) \cdot \widetilde{e}_{\mathsf{pc},c,i}(x_c) \quad \text{for } c \in \{1, 2\}$$

$$Q(b) \leftarrow \mathsf{eq}(\rho, b) \cdot \left(\sum_{x \in \{0,1\}^{\ell}} h_1(b, x_1) \cdot h_2(b, x_2) \cdot F(\vec{f}(b, x)) + \gamma \cdot h_{\mathsf{pc},1}(b, x_1) \cdot h_{\mathsf{pc},2}(b, x_2) \cdot F_{\mathsf{PC}}(\vec{f}_{\mathsf{pc}}(b, x)) \right)$$

$$\overline{Q} \leftarrow ((F, G), \rho, (\mathbf{u}_i, \mathsf{x}_i)_{i \in [k]}).$$

where $j \in [t]$.

7. $\mathcal{P}, \mathcal{V}$: Compute

$$S' \leftarrow \sigma' \cdot \mathsf{eq}(\rho, r_b)^{-1}$$

$$u_j \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot u_{i,j} \qquad u_{\mathsf{pc},j} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot u_{\mathsf{pc},i,j}$$

$$\overline{e} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot \overline{e}_i \qquad \overline{e}_{\mathsf{pc}} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot \overline{e}_{\mathsf{pc},i}$$

$$\mathsf{x} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot \mathsf{x}_i \qquad \mathsf{x}_{\mathsf{pc}} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot \mathsf{x}_{\mathsf{pc},i}.$$

for $j \in [s]$. The prover additionally computes for $j \in [s]$

$$w_j \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot w_{i,j} \qquad w_{\mathsf{pc},j} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot w_{\mathsf{pc},i,j}$$

$$e_c \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot e_{c,i} \qquad e_{\mathsf{pc},c} \leftarrow \sum_{i \in \{0,1\}^{\nu+1}} \mathsf{eq}(r_b, i) \cdot e_{\mathsf{pc},c,i} \quad \text{for } c \in \{1, 2\}.$$

8. $\mathcal{P}$: Compute $\vec{g} \leftarrow G((w_1, \ldots, w_s), \mathsf{x})$, $\vec{g}_{\mathsf{pc}} \leftarrow G_{\mathsf{PC}}((w_{\mathsf{pc},1}, \ldots, w_{\mathsf{pc},s}), \mathsf{x}_{\mathsf{pc}})$ and send to $\mathcal{V}$

$$S \leftarrow \sum_{x \in \{0,1\}^{\ell}} \widetilde{e}_1(x_1) \cdot \widetilde{e}_2(x_2) \cdot F(\vec{g}(x))$$

$$S_{\mathsf{pc}} \leftarrow \sum_{x \in \{0,1\}^{\ell}} \widetilde{e}_{\mathsf{pc},1}(x_1) \cdot \widetilde{e}_{\mathsf{pc},2}(x_2) \cdot F_{\mathsf{PC}}(\vec{g}_{\mathsf{pc}}(x)).$$

9. $\mathcal{V}$: Check that $S' = S + \gamma \cdot S_{\mathsf{pc}}$.
10. $\mathcal{P}, \mathcal{V}$: Define

$$\mathsf{u} \leftarrow ((u_1, \ldots, u_s), \overline{e}) \qquad \mathsf{u}_{\mathsf{pc}} \leftarrow ((u_{\mathsf{pc},1}, \ldots, u_{\mathsf{pc},s}), \overline{e}_{\mathsf{pc}}).$$

The prover additionally defines

$$\mathsf{w} \leftarrow ((w_1, \ldots, w_s), (e_1, e_2)) \qquad \mathsf{w}_{\mathsf{pc}} \leftarrow ((w_{\mathsf{pc},1}, \ldots, w_{\mathsf{pc},s}), (e_{\mathsf{pc},1}, e_{\mathsf{pc},2})).$$

11. $\mathcal{P}, \mathcal{V}$: Output the following instance-witness pairs (the verifier only outputs the instances).

$$((S, \mathsf{u}, \mathsf{x}), \mathsf{w}) \in \mathsf{NSC}$$
$$((S_{\mathsf{pc}}, \mathsf{u}_{\mathsf{pc}}, \mathsf{x}_{\mathsf{pc}}), \mathsf{w}_{\mathsf{pc}}) \in \mathsf{NSC}_{\mathsf{PC}}$$
$$((\overline{e}^{\tau}, \tau), e^{\tau}) \in \mathsf{ZC}_{\mathsf{PC}}$$

F AI Disclaimer

Parts of this manuscript were edited with assistance from an AI-based writing tool. In particular, we used AI support for improving spelling and wording, and for limited help with proofreading tasks such as formatting and consistency checks. We have produced and verified all technical content, definitions, statements, proofs, and experimental claims, and take full responsibility for the correctness and originality of the final text.